Western Union directional corridor analysis

Self-contained local Jupyter workspace for the 53,130 directed market pairs. The embedded data pack is extracted into a local runtime folder and loaded into clearly named pandas DataFrames. Use `select_pair(origin, destination)` with WU two-letter market codes.

Loaded tables

DataFrame	Rows	Columns
`corridor_universe`	53,130	30
`corridor_products`	52,022	24
`rank_stability`	8,572	17
`top10`	10	16
`markets`	301	5
`graph_corridors`	23,960	10
`graph_channels`	23,960	13
`q12_q13_summary`	10	16

Coverage: every pair has a `corridor_universe` row. Product, model, graph, and enrichment rows exist only where those layers have evidence. `null` means unavailable, not zero. The graph and Q12/Q13 tables are earlier or incomplete snapshots; use `corridor_universe`, `corridor_products`, and `top10` for the current Q10 view.

Data-source roadmap

These are the full source families for extending the analysis. They are tracked here separately from the eight tables currently loaded in the runtime.

#	Dataset	Raw granularity	Best uses
1	**World Bank Remittance Prices Worldwide (RPW), 2011–2025 Q3**	Provider × corridor × quarter × product	Pricing, WU presence/share-of-observations, corridor coverage, digital vs cash, exclusivity proxies, competitive intensity
2	**World Bank/KNOMAD bilateral remittance matrix**	Origin × destination country	Estimate corridor sizes, top-10 corridors, concentration, weight RPW observations economically
3	**World Bank WDI remittance inflows/outflows API**	Country × year	Global market size, 10/20/30-year growth, country concentration, 2025 reconstruction
4	**Global Findex respondent-level microdata**	Individual respondent	Q9 unbanked/underbanked senders/receivers; account/mobile-money status; demographic cuts
5	**Banxico remittance micro/tables + CEMLA series**	Mexico remittances by channel/time/geography	Cash vs account deposit, digital migration, payout method, receiver behavior
6	**WU + competitor location-locator data**	Individual physical agent/location	Q13 location density, nearest-store distance, corridor-side network density
7	**Census/ACS + population rasters / WorldPop**	Tract/grid population	Population-weight WU accessibility rather than naive store density
8	**OpenStreetMap + OSRM routing data**	Road graph / route matrix	Actual travel distance to WU/Ria/MoneyGram/etc., not straight-line distance
9	**FINABIEN Mexico outlet roster JSON**	Individual locations	Mexico-side benchmark/competitor network and access calculations

#	Dataset	Raw granularity	Best uses
10	**Regulator/MSB registries**	Licensed entity/location	Reconstruct agents and competitors where their own locator/API is incomplete
11	**UNCDF Nepal IME Pay dataset**	~51m transactions + 993 surveys + interviews	Mobile-wallet receipt behavior, cash-out, retention/use of remittances
12	**UNCDF Bangladesh/BRAC remittance dataset**	~4.2m transaction records + 507 surveys	Q11(b): what receivers actually do with digitally received remittances
13	**GSMA mobile-money datasets**	Country/provider/time transaction series	Q11(a), wallet penetration and international-remittance-to-wallet trends
14	**FinScope microdata**	Individual survey respondents	Remittance provider/channel choice; one Zambia wave identifies WU as a response
15	**Central-bank remittance datasets by country**	Month/channel/payment instrument	Fill World Bank gaps and build receiver-channel panels
16	**UN DESA International Migrant Stock**	Origin × destination × year	Fundamental corridor-volume model and migration elasticity
17	**Bilateral migration / diaspora matrices**	Country pair	Combine with bilateral remittance flows to estimate remittances per migrant
18	**Competitor filings/operating datasets**	Company × quarter/year	Transactions, principal, revenue, take rate, agent count; normalize WU vs Ria/Remitly/Intermex/MoneyGram
19	**Florida MSB registration downloads / other state registries**	Individual license/location	U.S. physical-location validation and network overlap
20	**EBA payment-institution register**	Institution × country/authorization	European competitor footprint, licensing, and potential corridor availability

In [1]:

```
#@title Load the 20-source roadmap pack and inspect bilateral flows
import pathlib, zipfile
import pandas as pd
from IPython.display import display

SOURCE_ARCHIVE = pathlib.Path.cwd() / "WU_Roadmap_Source_Pack.zip"
if not SOURCE_ARCHIVE.exists():
    raise FileNotFoundError("Place WU_Roadmap_Source_Pack.zip beside this notebook, then rerun this cell.")
SOURCE_DIR = pathlib.Path.cwd() / "local_runtime" / "wu_roadmap_sources"
SOURCE_DIR.mkdir(parents=True, exist_ok=True)
with zipfile.ZipFile(SOURCE_ARCHIVE) as zf:
    zf.extractall(SOURCE_DIR)

ROADMAP_DATA = {p.stem: pd.read_parquet(p) for p in sorted(SOURCE_DIR.glob("*.parquet"))}
globals().update(ROADMAP_DATA)
roadmap_loaded_tables = pd.DataFrame([
    {"DataFrame": name, "rows": len(frame), "columns": len(frame.columns)}
    for name, frame in ROADMAP_DATA.items()
]).sort_values("DataFrame", ignore_index=True)

print("ROADMAP AVAILABILITY")
status_order = pd.CategoricalDtype(["loaded", "partial", "reference", "missing"], ordered=True)
roadmap_status = source_manifest.copy()
roadmap_status["status"] = roadmap_status["status"].astype(status_order)
display(roadmap_status.sort_values(["status", "roadmap_item"]).style.format({"loaded_rows": "{:,}"}))
display(roadmap_status.groupby("status", observed=False).size().rename("roadmap_items").to_frame())

print("LOADED DATAFRAMES")
display(roadmap_loaded_tables)
print("Loaded", f"{sum(len(frame) for frame in ROADMAP_DATA.values()):,}", "records into", len(ROADMAP_DATA), "roadmap DataFrames")

print("BILATERAL REMITTANCE FLOWS USED IN CORRIDOR WEIGHTING")
bilateral_flows = bilateral_remittance_2024[[
    "y", "s", "r", "v", "source_country", "receive_country", "corridor", "neutral_rank", "source_url"
]].copy()
bilateral_flows["flow_usd"] = pd.to_numeric(bilateral_flows["v"], errors="coerce")
display(bilateral_flows.sort_values("flow_usd", ascending=False).head(20).style.format({"flow_usd": "USD {:,.0f}"}))
print("Directional corridors:", f"{len(bilateral_flows):,}")
print("Year(s):", sorted(bilateral_flows["y"].dropna().astype(str).unique()))
print("The flow column is the modeled bilateral remittance estimate. WU corridor shares are downstream model outputs, not disclosed company data.")

def roadmap_pair(origin_iso3="USA", destination_iso3="MEX"):
    return bilateral_flows.loc[
        bilateral_flows["s"].eq(origin_iso3.upper()) & bilateral_flows["r"].eq(destination_iso3.upper())
    ].copy()
```

```
US_MEX_BILATERAL = roadmap_pair("USA", "MEX")
```

```
ROADMAP AVAILABILITY
```

	roadmap_item	source_family	status	loaded_dataframes	loaded_rows	summed_columns	coverage_note
0	1	World Bank RPW, 2011–2025 Q3	loaded	rpw_history	253,960	49	Full observation-level workbook converted to Parquet; raw cell values retained as strings.
1	2	World Bank/KNOMAD bilateral remittance matrix	loaded	bilateral_remittance_2024	8,572	15	8,572 directional 2024 modeled remittance flows. Columns prefixed modeled_wu_* are downstream WU allocation outputs, not disclosed corridor revenue.
3	4	Global Findex respondent microdata	loaded	findex_microdata	144,090	199	144,090 respondent rows and 199 variables; survey weights retained.
8	9	FINABIEN Mexico outlet roster	loaded	finabien_roster	1,644	17	1,644 outlet records with coordinates and address fields.
9	10	Regulator/MSB registries	loaded	regulatory_agent_records, regulatory_agent_relationships, regulatory_outlet_relationships, regulatory_source_register	73,249	74	Seven-jurisdiction normalized registry evidence and relationship tables.
18	19	Florida MSB downloads / state registries	loaded	florida_msb_records	42,656	35	Florida records are also included in the broader regulatory tables.
2	3	World Bank WDI remittance inflows/outflows	partial	wdi_remittances_wld_lmy	132	11	Received-remittance history for WLD and LMY only; full country inflow/outflow panel is not in the local corpus.
4	5	Banxico remittance tables + CEMLA	partial	mexico_remittance_summary, source_documents	6	17	Structured Mexico summary plus frozen Banxico source documents; no CEMLA microdata table.
5	6	WU + competitor location-locator data	partial	wu_locations_global, regulatory_agent_records	507,068	67	Full collected WU location table plus registry-derived agent records; competitor locator coverage is uneven.
6	7	Census/ACS + population rasters / WorldPop	partial	miami_acs_demand_tracts, yucatan_population	3,333	298	Miami ACS tracts and Yucatán INEGI population points; no global WorldPop raster.
7	8	OpenStreetMap + OSRM routing	partial	miami_access_distribution, yucatan_routes	725	31	Computed Miami and Yucatán route outputs; no global road graph in the pack.
11	12	UNCDF Bangladesh/BRAC	partial	bangladesh_wallet_monthly	14	12	Monthly Bangladesh mobile-financial-services remittance series; the 4.2m transaction/507-survey microdata are not local.
14	15	Central-bank remittance datasets	partial	mexico_remittance_summary, colombia_remittances_monthly, india_rbi_country_panel	43	40	Structured Mexico, Colombia, and India/RBI extracts; not a global central-bank panel.
17	18	Competitor filings/operating datasets	partial	provider_benchmarks, source_documents	4	18	Normalized provider benchmarks plus frozen public filings/releases; company and period coverage varies.
12	13	GSMA mobile-money datasets	reference	source_documents	3	4	Frozen GSMA reports are local; no provider-country-time transaction microdata table was found.
10	11	UNCDF Nepal IME Pay	missing		0	0	Referenced in the research roadmap; transaction/survey microdata were not found locally.
13	14	FinScope microdata	missing		0	0	No FinScope respondent file was found locally.
15	16	UN DESA International Migrant Stock	missing		0	0	No standalone UN DESA migrant-stock file was found locally.
16	17	Bilateral migration / diaspora matrices	missing		0	0	No standalone bilateral migration matrix was found; the bilateral remittance flow file remains usable on its own.
19	20	EBA payment-institution register	missing		0	0	No EBA register extract was found locally.

	roadmap_items
status	
loaded	6
partial	8
reference	1
missing	5

LOADED DATAFRAMES

	DataFrame	rows	columns
0	bangladesh_wallet_monthly	14	12
1	bilateral_remittance_2024	8572	15
2	colombia_remittances_monthly	31	21
3	finabien_roster	1644	17
4	findex_microdata	144090	199
5	florida_msb_records	42656	35
6	india_rbi_country_panel	9	6
7	mexico_remittance_summary	3	13
8	miami_access_distribution	27	16
9	miami_acs_demand_tracts	642	12
10	provider_benchmarks	1	14
11	regulatory_agent_records	71533	35
12	regulatory_agent_relationships	573	13
13	regulatory_outlet_relationships	1121	13
14	regulatory_source_register	22	13
15	rpw_history	253960	49
16	source_documents	3	4
17	source_manifest	20	7
18	wdi_remittances_wld_lmy	132	11
19	wu_locations_global	435535	32
20	yucatan_population	2691	286
21	yucatan_routes	698	15

Loaded 963,977 records into 22 roadmap DataFrames
BILATERAL REMITTANCE FLOWS USED IN CORRIDOR WEIGHTING

	y	s	r	v	source_country	receive_country	corridor	neutral_rank	source_url	flow_usd
0	2024	USA	MEX	65855005077	United States of America	Mexico	United States of America -> Mexico	1.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 65,855,005,077
1	2024	USA	IND	27250393272	United States of America	India	United States of America -> India	2.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 27,250,393,272
4	2024	ARE	IND	26334542340	United Arab Emirates	India	United Arab Emirates -> India	5.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 26,334,542,340
2	2024	USA	GTM	19680528363	United States of America	Guatemala	United States of America -> Guatemala	3.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 19,680,528,363
6	2024	SAU	IND	15216172432	Saudi Arabia	India	Saudi Arabia -> India	7.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 15,216,172,432
3	2024	USA	PHL	14216968526	United States of America	Philippines	United States of America -> Philippines	4.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 14,216,968,526
17	2024	SAU	PAK	10565730481	Saudi Arabia	Pakistan	Saudi Arabia -> Pakistan	18.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 10,565,730,481
9	2024	GBR	IND	10443491625	United Kingdom	India	United Kingdom -> India	10.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 10,443,491,625
18	2024	KWT	IND	9506522122	Kuwait	India	Kuwait -> India	19.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 9,506,522,122
19	2024	SAU	EGY	9308422513	Saudi Arabia	Egypt	Saudi Arabia -> Egypt	20.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 9,308,422,513
5	2024	USA	DOM	8926733066	United States of America	Dominican Republic	United States of America -> Dominican Republic	6.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 8,926,733,066
29	2024	MYS	IDN	7883315434	Malaysia	Indonesia	Malaysia -> Indonesia	30.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 7,883,315,434
24	2024	SAU	BGD	7792655611	Saudi Arabia	Bangladesh	Saudi Arabia -> Bangladesh	25.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 7,792,655,611
7	2024	USA	SLV	7608901102	United States of America	El Salvador	United States of America -> El Salvador	8.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 7,608,901,102
8	2024	CAN	IND	7594507109	Canada	India	Canada -> India	9.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 7,594,507,109
10	2024	USA	CHN	7129152129	United States of America	China	United States of America -> China	11.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 7,129,152,129
11	2024	USA	VNM	7013237641	United States of America	Vietnam	United States of America -> Vietnam	12.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 7,013,237,641
12	2024	USA	HND	6970751264	United States of America	Honduras	United States of America -> Honduras	13.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 6,970,751,264

	y	s	r	v	source_country	receive_country	corridor	neutral_rank	source_url	flow_usd
35	2024	AUS	IND	6692824591	Australia	India	Australia -> India	36.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 6,692,824,591
36	2024	HKG	CHN	6635917075	Hong Kong	China	Hong Kong -> China	37.000000	https://www.sec.gov/Archives/edgar/data/1365135/000119312526061340/wu-20251231.htm; https://center-for-global-development.github.io/Bilateral-remittances-matrix-update/4-remittances-map.html; https://blogs.worldbank.org/en/peoplemove/bilateral-remittance-matrix-new	USD 6,635,917,075

```
Directional corridors: 8,572
Year(s): ['2024']
The flow column is the modeled bilateral remittance estimate. WU corridor shares are downstream model outputs, not disclosed company data.
```

In [2]:

```
#@title DataFrame catalog and 100-row sample browser
import pandas as pd
from IPython.display import display, clear_output
import ipywidgets as widgets

# Status describes coverage of the intended source family. Every local Parquet table is loaded in full.
meta = [
("bangladesh_wallet_monthly","12","One month of mobile-financial-services inward remittance activity","Bangladesh","2024-01 to 2025-02","Bangladesh Bank MFS and wage-remittance series","partial source family","Wallet receipt adoption and cash-out context"),
("bilateral_remittance_2024","2","One modeled sending-country to receiving-country remittance flow","Global; 8,572 directional country pairs","2024","World Bank / CGD bilateral remittances matrix update","fully loaded","Core: corridor size, rank, concentration, and economic weighting"),
("colombia_remittances_monthly","15","One month of national inward remittance value and underwriting fields","Colombia","2024-01 to 2026-07","Banco de la Republica series 15363","partial source family","Receiver-market trend and payment-rail context"),
("finabien_roster","9","One FINABIEN branch or outlet","Mexico; all 32 states","Roster retrieved 2026","Gobierno de Mexico / FINABIEN outlet roster","fully loaded","High: Mexico payout-network benchmark and access"),
("findex_microdata","4","One survey respondent with weight and financial-behavior variables","140 economies; 7 World Bank regions","2024","World Bank Global Findex respondent microdata","fully loaded","High: banked status, mobile money, digital payments, and remittance behavior"),
("florida_msb_records","19; also 10","One Florida principal-agent-location registry record","Florida, including limited out-of-state addresses","As of 2026-09-04","Florida Office of Financial Regulation MSB downloads","fully loaded; overlaps regulatory_agent_records","High: outlet validation and WU/competitor overlap"),
("india_rbi_country_panel","15","One sending-country share of remittances received by India","India; 8 named origins plus Other","FY2017, FY2021, FY2024","Reserve Bank of India survey extract","partial source family","India corridor calibration and origin mix"),
("mexico_remittance_summary","5; also 15","One annual Mexico total or one 2024 channel summary","Mexico","2023-2024","Banco de Mexico December 2024 release / CE80","partial source family","High for US-MX: flows, transactions, ticket, and payout channel"),
("miami_access_distribution","8","One provider x birth-country x routing-threshold access result","Miami-Dade; Mexico, Guatemala, Honduras-born populations","Derived 2026","ACS demand, provider locations, OpenStreetMap, OSRM","partial geographic extract","High for Miami case study: travel time and provider coverage"),
("miami_acs_demand_tracts","7","One tract with foreign-born demand counts and centroid","Miami-Dade County; 642 census tracts","ACS vintage not encoded","U.S. Census Bureau ACS B05006 extract","partial geographic extract","High for Miami case study: population-weighted access"),
("provider_benchmarks","18","One provider-period operating benchmark","Euronet group; Ria/Xe/Dandelion","FY2025","Euronet FY2025 public filing extract","partial source family","Competitor principal, transaction, ticket, and revenue normalization"),
("regulatory_agent_records","10","One normalized principal-agent-address registry record","Arkansas, Massachusetts, Florida, Jamaica, New Mexico, Dominican Republic","2025-12-31 to 2026-09-04","Official regulator and MSB registry downloads","fully loaded for collected jurisdictions","High: reconstruct agents and identify competing principals"),
("regulatory_agent_relationships","10","One legal-agent match linking WU with peer principal families","Arkansas, Florida, Jamaica, Massachusetts, New Mexico","2025-12-31 to 2026-09-04","Derived from official registry records","fully loaded for collected jurisdictions","High: agent multi-homing and exclusivity evidence"),
("regulatory_outlet_relationships","10","One outlet-address match linking WU with peer principal families","Arkansas, Florida, Jamaica, Massachusetts, New Mexico","2025-12-31 to 2026-09-04","Derived from official registry records","fully loaded for collected jurisdictions","High: same-location competition and network overlap"),
("regulatory_source_register","10","One acquired source file with URL, date, hash, and parse status","7 jurisdictions, including India source files","2025-12-31 to 2026-09-04","Official regulator/MSB source ledger","fully loaded","Audit: provenance and coverage boundaries"),
("rpw_history","1","One provider-product quote observation for a corridor and quarter","49 sending and 111 receiving economies; 382 observed corridor codes","2011 Q1 to 2025 Q3","World Bank Remittance Prices Worldwide workbook","fully loaded","Core: pricing, WU observation presence, channel mix, and competition"),
("source_documents","5, 13, 18","One document family retained as a local source reference","Mexico, global mobile money, and public-company coverage","2012-2026; varies","Banxico, GSMA, and public-company filings/releases","reference only","Audit/supporting: evidence not normalized as row-level data"),
("source_manifest","Control","One roadmap source family and its availability/limitation status","All 20 roadmap items","Current pack build","Generated from local source-pack inventory","fully loaded control table","Audit: identifies loaded, partial, reference, and missing sources"),
("wdi_remittances_wld_lmy","3","One aggregate-geography/year remittance-received observation","World and low/middle-income aggregate only","1960-2025","World Bank WDI API: BX.TRF.PWKR.CD.DT","partial source family","Long-run market growth; not usable for country ranking"),
("wu_locations_global","6","One WU location ID with market, address, coordinates, and service flags when available","211 WU market codes / 198 mapped ISO3 economies","Retrieved 2026-09-07","Western Union locations sitemap and locator collection","partial source family; WU table loaded","Core network footprint; detail/service coverage varies by location"),
("yucatan_population","7","One INEGI locality or municipality aggregate with demographic/housing fields","Yucatan; 106 municipalities plus aggregates","2020 census","INEGI 2020 locality census extract","partial geographic extract","High for Yucatan: population-weighted access and receiver density"),
("yucatan_routes","8","One locality-to-selected-site OSRM route result","Yucatan, Mexico","Derived 2026 from 2020 population and collected outlets","INEGI localities, WU/FINABIEN sites,
```

```
OpenStreetMap, OSRM","partial geographic extract","High for Yucatan: travel time and distance to payout sites")]
cols=["dataframe","roadmap_item","row_represents","geographic_coverage","year_or_period","source","coverage_status","western_union_use"]
DATAFRAME_CATALOG=pd.DataFrame(meta,columns=cols)
DATAFRAME_CATALOG["rows"]=DATAFRAME_CATALOG.dataframe.map(lambda n:len(ROADMAP_DATA[n]))
DATAFRAME_CATALOG["columns"]=DATAFRAME_CATALOG.dataframe.map(lambda n:len(ROADMAP_DATA[n].columns))
total=int(DATAFRAME_CATALOG.rows.sum())
DATAFRAME_CATALOG["volume_share"]=DATAFRAME_CATALOG.rows/total
DATAFRAME_CATALOG["sample_rows"]=DATAFRAME_CATALOG.rows.clip(upper=100)
DATAFRAME_CATALOG=DATAFRAME_CATALOG[["dataframe","rows","volume_share","sample_rows","columns","roadmap_item","row_represents","geographic_coverage","year_or_period","source","coverage_status"
,"western_union_use"]].sort_values("rows",ascending=False,ignore_index=True)
display(DATAFRAME_CATALOG.style.format({"rows":"{:,}","volume_share":"{:.2%}","sample_rows":"{:,}"}))
print(f"Catalog reconciles {total:,} rows across {len(DATAFRAME_CATALOG)} DataFrames.")
print("The raw total includes the Florida subset also present in regulatory_agent_records, plus control/reference rows.")
ROADMAP_SAMPLES={n:ROADMAP_DATA[n].head(100).copy() for n in DATAFRAME_CATALOG.dataframe}
print(f"Stored {sum(map(len,ROADMAP_SAMPLES.values())):,} sample rows in ROADMAP_SAMPLES.")
picker=widgets.Dropdown(options=DATAFRAME_CATALOG.dataframe.tolist(),value="bilateral_remittance_2024",description="DataFrame:",layout=widgets.Layout(width="650px"))
out=widgets.Output()
def show_sample(change=None):
 n=picker.value; m=DATAFRAME_CATALOG.set_index("dataframe").loc[n]
 with out:
  clear_output(wait=True)
  print(f"{n}: {int(m['rows']):,} total rows; showing {len(ROADMAP_SAMPLES[n]):,}")
  print(f"One row: {m['row_represents']}")
  print(f"Coverage: {m['geographic_coverage']} | Period: {m['year_or_period']} | Status: {m['coverage_status']}")
  print(f"WU use: {m['western_union_use']}")
  display(ROADMAP_SAMPLES[n])
picker.observe(show_sample,names="value")
display(picker,out); show_sample()
```

	dataframe	rows	volume_share	sample_rows	columns	roadmap_item	row_represents	geographic_coverage	year_or_period	source	coverage_status	western_union_use
0	wu_locations_global	435,535	45.18%	100	32	6	One WU location ID with market, address, coordinates, and service flags when available	211 WU market codes / 198 mapped ISO3 economies	Retrieved 2026-09-07	Western Union locations sitemap and locator collection	partial source family; WU table loaded	Core network footprint; detail/service coverage varies by location
1	rpw_history	253,960	26.35%	100	49	1	One provider-product quote observation for a corridor and quarter	49 sending and 111 receiving economies; 382 observed corridor codes	2011 Q1 to 2025 Q3	World Bank Remittance Prices Worldwide workbook	fully loaded	Core: pricing, WU observation presence, channel mix, and competition
2	findex_microdata	144,090	14.95%	100	199	4	One survey respondent with weight and financial-behavior variables	140 economies; 7 World Bank regions	2024	World Bank Global Findex respondent microdata	fully loaded	High: banked status, mobile money, digital payments, and remittance behavior
3	regulatory_agent_records	71,533	7.42%	100	35	10	One normalized principal-agent-address registry record	Arkansas, Massachusetts, Florida, Jamaica, New Mexico, Dominican Republic	2025-12-31 to 2026-09-04	Official regulator and MSB registry downloads	fully loaded for collected jurisdictions	High: reconstruct agents and identify competing principals
4	florida_msb_records	42,656	4.43%	100	35	19; also 10	One Florida principal-agent-location registry record	Florida, including limited out-of-state addresses	As of 2026-09-04	Florida Office of Financial Regulation MSB downloads	fully loaded; overlaps regulatory_agent_records	High: outlet validation and WU/competitor overlap
5	bilateral_remittance_2024	8,572	0.89%	100	15	2	One modeled sending-country to receiving-country remittance flow	Global; 8,572 directional country pairs	2024	World Bank / CGD bilateral remittances matrix update	fully loaded	Core: corridor size, rank, concentration, and economic weighting
6	yucatan_population	2,691	0.28%	100	286	7	One INEGI locality or municipality aggregate with demographic/housing fields	Yucatan; 106 municipalities plus aggregates	2020 census	INEGI 2020 locality census extract	partial geographic extract	High for Yucatan: population-weighted access and receiver density
7	finabien_roster	1,644	0.17%	100	17	9	One FINABIEN branch or outlet	Mexico; all 32 states	Roster retrieved 2026	Gobierno de Mexico / FINABIEN outlet roster	fully loaded	High: Mexico payout-network benchmark and access
8	regulatory_outlet_relationships	1,121	0.12%	100	13	10	One outlet-address match linking WU with peer principal families	Arkansas, Florida, Jamaica, Massachusetts, New Mexico	2025-12-31 to 2026-09-04	Derived from official registry records	fully loaded for collected jurisdictions	High: same-location competition and network overlap
9	yucatan_routes	698	0.07%	100	15	8	One locality-to-selected-site OSRM route result	Yucatan, Mexico	Derived 2026 from 2020 population and collected outlets	INEGI localities, WU/FINABIEN sites, OpenStreetMap, OSRM	partial geographic extract	High for Yucatan: travel time and distance to payout sites
10	miami_acs_demand_tracts	642	0.07%	100	12	7	One tract with foreign-born demand counts and centroid	Miami-Dade County; 642 census tracts	ACS vintage not encoded	U.S. Census Bureau ACS B05006 extract	partial geographic extract	High for Miami case study: population-weighted access
11	regulatory_agent_relationships	573	0.06%	100	13	10	One legal-agent match linking WU with peer principal families	Arkansas, Florida, Jamaica, Massachusetts, New Mexico	2025-12-31 to 2026-09-04	Derived from official registry records	fully loaded for collected jurisdictions	High: agent multi-homing and exclusivity evidence
12	wdi_remittances_wld_lmy	132	0.01%	100	11	3	One aggregate-geography/year remittance-received observation	World and low/middle-income aggregate only	1960-2025	World Bank WDI API: BX.TRF.PWKR.CD.DT	partial source family	Long-run market growth; not usable for country ranking
13	colombia_remittances_monthly	31	0.00%	31	21	15	One month of national inward remittance value and underwriting fields	Colombia	2024-01 to 2026-07	Banco de la Republica series 15363	partial source family	Receiver-market trend and payment-rail context
14	miami_access_distribution	27	0.00%	27	16	8	One provider x birth-country x routing-threshold access result	Miami-Dade; Mexico, Guatemala, Honduras-born populations	Derived 2026	ACS demand, provider locations, OpenStreetMap, OSRM	partial geographic extract	High for Miami case study: travel time and provider coverage
15	regulatory_source_register	22	0.00%	22	13	10	One acquired source file with URL, date, hash, and parse status	7 jurisdictions, including India source files	2025-12-31 to 2026-09-04	Official regulator/MSB source ledger	fully loaded	Audit: provenance and coverage boundaries
16	source_manifest	20	0.00%	20	7	Control	One roadmap source family and its availability/limitation status	All 20 roadmap items	Current pack build	Generated from local source-pack inventory	fully loaded control table	Audit: identifies loaded, partial, reference, and missing sources
17	bangladesh_wallet_monthly	14	0.00%	14	12	12	One month of mobile-financial-services inward remittance activity	Bangladesh	2024-01 to 2025-02	Bangladesh Bank MFS and wage-remittance series	partial source family	Wallet receipt adoption and cash-out context
18	india_rbi_country_panel	9	0.00%	9	6	15	One sending-country share of remittances received by India	India; 8 named origins plus Other	FY2017, FY2021, FY2024	Reserve Bank of India survey extract	partial source family	India corridor calibration and origin mix

	dataframe	rows	volume_share	sample_rows	columns	roadmap_item	row_represents	geographic_coverage	year_or_period	source	coverage_status	western_union_use
19	mexico_remittance_summary	3	0.00%	3	13	5; also 15	One annual Mexico total or one 2024 channel summary	Mexico	2023-2024	Banco de Mexico December 2024 release / CE80	partial source family	High for US-MX: flows, transactions, ticket, and payout channel
20	source_documents	3	0.00%	3	4	5, 13, 18	One document family retained as a local source reference	Mexico, global mobile money, and public-company coverage	2012-2026; varies	Banxico, GSMA, and public-company filings/releases	reference only	Audit/supporting: evidence not normalized as row-level data
21	provider_benchmarks	1	0.00%	1	14	18	One provider-period operating benchmark	Euronet group; Ria/Xe/Dandelion	FY2025	Euronet FY2025 public filing extract	partial source family	Competitor principal, transaction, ticket, and revenue normalization

```
Catalog reconciles 963,977 rows across 22 DataFrames.
The raw total includes the Florida subset also present in regulatory_agent_records, plus control/reference rows.
Stored 1,430 sample rows in ROADMAP_SAMPLES.
Dropdown(description='DataFrame:', index=5, layout=Layout(width='650px'), options=('wu_locations_global', 'rpw…
Output()
```

In [3]:

```
#@title 1. Load the local data pack (run first)
import json, pathlib, zipfile
import pandas as pd
from IPython.display import display

ARCHIVE = pathlib.Path.cwd() / "WU_Corridor_Analysis_Data.zip"
if not ARCHIVE.exists():
    raise FileNotFoundError("Place WU_Corridor_Analysis_Data.zip beside this notebook, then rerun this cell.")
DATA_DIR = pathlib.Path.cwd() / "local_runtime" / "wu_corridor_data"
DATA_DIR.mkdir(parents=True, exist_ok=True)
with zipfile.ZipFile(ARCHIVE) as zf:
    zf.extractall(DATA_DIR)

DATA = {p.stem: pd.read_parquet(p) for p in sorted(DATA_DIR.glob("*.parquet"))}
globals().update(DATA)
with open(DATA_DIR / "US_MX_raw_quote.json", encoding="utf-8") as handle:
    US_MX_raw_quote = json.load(handle)
with open(DATA_DIR / "US_MX_pair_dossier.json", encoding="utf-8") as handle:
    US_MX_pair_dossier = json.load(handle)

loaded_tables = pd.DataFrame([
    {"DataFrame": name, "rows": len(frame), "columns": len(frame.columns)}
    for name, frame in DATA.items()
]).sort_values("DataFrame", ignore_index=True)
display(loaded_tables)
print("Loaded", f"{sum(len(frame) for frame in DATA.values()):,}", "tabular records into", len(DATA), "DataFrames")
```

	DataFrame	rows	columns
0	corridor_products	52022	24
1	corridor_universe	53130	30
2	graph_channels	23960	13
3	graph_corridors	23960	10
4	markets	301	5
5	q12_q13_summary	10	16
6	rank_stability	8572	17
7	top10	10	16

```
Loaded 161,965 tabular records into 8 DataFrames
```

In [4]:

```python
#@title 2. Pair selector and reusable helpers
def _pair_rows(frame, origin, destination):
    origin, destination = origin.upper(), destination.upper()
    corridor_id = f"{origin}->{destination}"
    if {"origin_wu_code", "destination_wu_code"}.issubset(frame.columns):
        return frame.loc[(frame.origin_wu_code == origin) & (frame.destination_wu_code == destination)].copy()
    if "corridor_id" in frame.columns:
        return frame.loc[frame.corridor_id == corridor_id].copy()
    if "corridor" in frame.columns:
        return frame.loc[frame.corridor == corridor_id].copy()
    if "market_id" in frame.columns:
        return frame.loc[frame.market_id.isin([origin, destination])].copy()
    return frame.iloc[0:0].copy()

def select_pair(origin="US", destination="MX"):
    """Return every matching record, organized by dataset, for one directed pair."""
    result = {name: _pair_rows(frame, origin, destination) for name, frame in DATA.items()}
    result["record_counts"] = pd.DataFrame([
        {"dataset": name, "records": len(frame), "fields": len(frame.columns)}
        for name, frame in result.items()
    ])
    return result

def show_pair(pair):
    display(pair["record_counts"])
    for name, frame in pair.items():
        if name != "record_counts" and len(frame):
            print(f"\n{name}: {len(frame)} record(s)")
            display(frame)
```

In [5]:

```python
#@title 3. Worked example: United States → Mexico
PAIR = select_pair("US", "MX")
show_pair(PAIR)

# Convenient aliases for analysis
PAIR_UNIVERSE = PAIR["corridor_universe"]
PAIR_PRODUCTS = PAIR["corridor_products"]
PAIR_STABILITY = PAIR["rank_stability"]
PAIR_TOP10 = PAIR["top10"]
```

	dataset	records	fields
0	corridor_products	3	24
1	corridor_universe	1	30
2	graph_channels	1	13
3	graph_corridors	1	10
4	markets	2	5
5	q12_q13_summary	1	16
6	rank_stability	1	17
7	top10	1	16

```
corridor_products: 3 record(s)
```

	corridor_id	origin_wu_code	destination_wu_code	send_currency	payout_currency	product_row_id	service_id	service_name	fund_out	fund_out_mnemonic	...	wallet_payout	funding_methods	payout_methods	speed	request_hash	response_hash	payload_fingerprint	timestamp	http_status	raw_evidence_path
4937	US->MX	US	MX	USD	MXN	2ba6d69e14b824ca5249e42b04d64e6a9ebb2783930bb8...	000	Money In Minutes	000	MM	...	None	["AC","AD","AP","AR","CA","CC","DC","GP"]	["000","MM"]	0|0-4	2ba6d69e14b824ca5249e42b04d64e6a9ebb2783930bb8...	700b8a5ba131c024f385dc5b081b03933eb15e20fd080b...	df47f4ea8360150aaaeb6c2ff79aea2320fababd726f73...	2026-09-07T13:18:52+00:00	NaN	raw\quotes\2ba6d69e14b824ca5249e42b04d64e6a9eb...
4937	US->MX	US	MX	USD	MXN	2ba6d69e14b824ca5249e42b04d64e6a9ebb2783930bb8...	500	Direct to Bank	500	DB	...	None	["AC","AD","AP","AR","CA","CC","DC","GP"]	["500","DB"]	0|0-1|0-4	2ba6d69e14b824ca5249e42b04d64e6a9ebb2783930bb8...	700b8a5ba131c024f385dc5b081b03933eb15e20fd080b...	df47f4ea8360150aaaeb6c2ff79aea2320fababd726f73...	2026-09-07T13:18:52+00:00	NaN	raw\quotes\2ba6d69e14b824ca5249e42b04d64e6a9eb...
4938	US->MX	US	MX	USD	MXN	2ba6d69e14b824ca5249e42b04d64e6a9ebb2783930bb8...	800	Mobile Money Transfer	800	MB	...	True	["AC","AD","AP","AR","CC","DC","GP"]	["800","MB"]	0|0-4	2ba6d69e14b824ca5249e42b04d64e6a9ebb2783930bb8...	700b8a5ba131c024f385dc5b081b03933eb15e20fd080b...	df47f4ea8360150aaaeb6c2ff79aea2320fababd726f73...	2026-09-07T13:18:52+00:00	NaN	raw\quotes\2ba6d69e14b824ca5249e42b04d64e6a9eb...

3 rows × 24 columns

corridor_universe: 1 record(s)

	corridor_id	origin_wu_code	destination_wu_code	origin_name	destination_name	origin_iso2_secondary	destination_iso2_secondary	origin_iso3_secondary	destination_iso3_secondary	origin_mapping_status	...	payout_currencies	request_hashes	response_hashes	latest_timestamp	digital	retail	cash_pickup	bank_payout	wallet_payout	channel_evidence_basis
4866	US->MX	US	MX	United States	Mexico	US	MX	USA	MEX	direct_code	...	["MXN"]	["2ba6d69e14b824ca5249e42b04d64e6a9ebb2783930b...	["700b8a5ba131c024f385dc5b081b03933eb15e20fd08...	2026-09-07T13:18:52+00:00	true	true	true	true	true	pair_product_response

1 rows × 30 columns

graph_channels: 1 record(s)

	corridor_id	origin_wu_code	destination_wu_code	digital	retail	cash_pickup	account_payout	wallet_payout	funding_methods	payout_methods	products	product_status	channel_evidence_source
22713	US->MX	US	MX	true	true	true	true	unknown	["AC","AD","AP","AR","CA","CC","DC","GP"]	["000","500","800","DB","MB","MM"]	["Direct to Bank","Mobile Money Transfer","Mon...	PRODUCT_CONFIRMED	dcaas_country_list|price_corridors|smo_config

graph_corridors: 1 record(s)

	corridor_id	origin_wu_code	destination_wu_code	origin_name	destination_name	origin_canonical_iso2	destination_canonical_iso2	website_destination_observed	corridor_confirmed	product_status
22713	US->MX	US	MX	United States	Mexico	US	MX	True	true	PRODUCT_CONFIRMED

markets: 2 record(s)

	market_id	wu_market_name	iso2_secondary	iso3_secondary	normalization_notes
136	MX	Mexico	MX	MEX	direct_code
211	US	United States	US	USA	direct_code

q12_q13_summary: 1 record(s)

	corridor	modeled_revenue_share	send_wu_sites	receive_wu_sites	send_confirmed_multibrand_pct	receive_confirmed_multibrand_pct	send_wu_p50_miles	send_wu_p90_miles	receive_wu_p50_miles	receive_wu_p90_miles	send_moneygram_p50_miles	send_ria_p50_miles	receive_moneygram_p50_miles	receive_ria_p50_miles	wu_relative_access_advantage	service_exclusivity_evidence_quality
0	US->MX	0.081486	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	send:NO_WU_AFFIRMATIVE_CONTRACT_EVIDENCE;recei...

rank_stability: 1 record(s)

	corridor	send_iso3	receive_iso3	origin_wu_code	destination_wu_code	rank_min	rank_max	median_rank	top10_frequency	modeled_share_min	modeled_share_max	rank_stability	product_confirmed	product_status	digital	retail	scenario_count
0	United States of America -> Mexico	USA	MEX	US	MX	1	1	1.0	1.0	0.044779	0.122174	HIGH	True	PRODUCT_CONFIRMED	true	true	12

top10: 1 record(s)

	rank	origin_wu_code	destination_wu_code	origin_name	destination_name	modeled_revenue_share	share_perimeter	product_confirmed	digital	retail	cash_pickup	bank_payout	wallet_payout	rank_stability	model_evidence_strength	limitations
0	1	US	MX	United States	Mexico	0.078937	FY2025 consolidated revenue; cross-border CMT ...	True	True	True	True	True	true	HIGH	HIGH	Modeled allocation; WU does not disclose corri...

Current findings

- **30,932 of 53,130** directed pairs have validated products; **14,488** have an explicit no-product response. The remainder are untestable currency configurations or semantic ambiguity.
- Among confirmed pairs, retail-only evidence dominates. Both digital and retail are confirmed for **9,893** pairs.
- The final ten modeled corridors account for **21.39%** of the modeled revenue allocation. US→Mexico ranks first at **7.89%**.
- Only **4 of the final 10** remain in the top ten across all 12 sensitivity scenarios.
- The US→Mexico raw quote resolves to three services and retains individual funding methods, fees, FX rates, speeds, limits, and payout amounts.

Run the cells below to reproduce and explore each finding.

In [6]:
```
#@title 4. Coverage funnel and evidence states
status_order = ["PRODUCT_CONFIRMED", "EXPLICIT_NO_PRODUCT", "UNTESTABLE_CURRENCY", "SEMANTIC_AMBIGUITY"]
coverage_funnel = (
    corridor_universe["product_status"].value_counts()
    .reindex(status_order, fill_value=0).rename_axis("product_status").reset_index(name="pairs")
)
coverage_funnel["share_of_53,130"] = coverage_funnel.pairs / len(corridor_universe)
display(coverage_funnel.style.format({"share_of_53,130": "{:.1%}"}))
print("Confirmed among evaluable pairs:", f"{coverage_funnel.loc[0, 'pairs'] / coverage_funnel.loc[:1, 'pairs'].sum():.1%}")
```

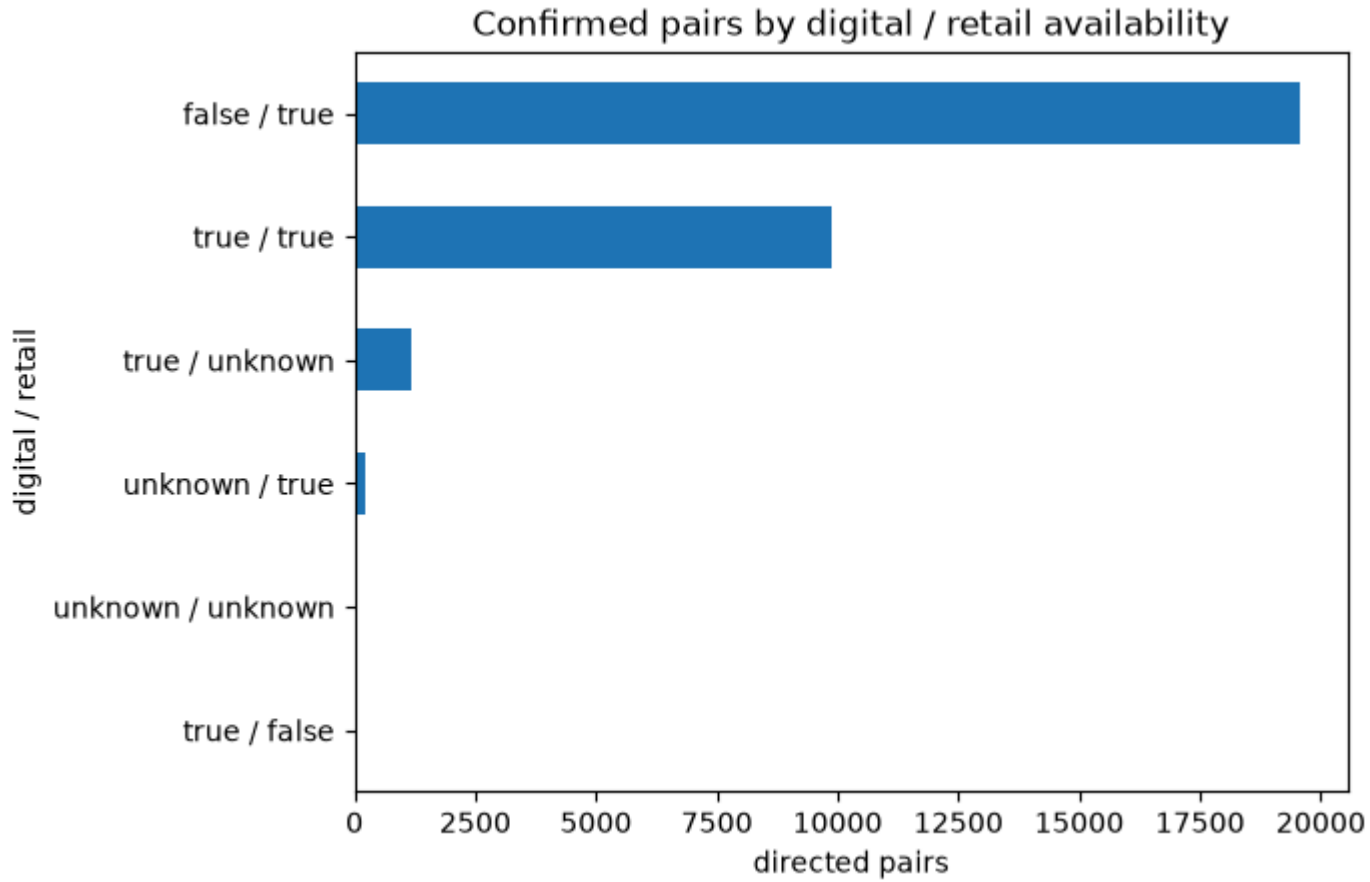
	product_status	pairs	share_of_53,130
0	PRODUCT_CONFIRMED	30932	58.2%
1	EXPLICIT_NO_PRODUCT	14488	27.3%
2	UNTESTABLE_CURRENCY	7302	13.7%
3	SEMANTIC_AMBIGUITY	408	0.8%

```
Confirmed among evaluable pairs: 68.1%
```

In [7]:
```
#@title 5. Channel combinations across confirmed pairs
channel_counts = (
    corridor_universe.loc[corridor_universe.product_status.eq("PRODUCT_CONFIRMED")]
    .groupby(["digital", "retail"], dropna=False)
    .size().rename("pairs").reset_index().sort_values("pairs", ascending=False)
)
display(channel_counts)
ax = channel_counts.assign(combination=channel_counts.digital.astype(str) + " / " + channel_counts.retail.astype(str)).plot.barh(
    x="combination", y="pairs", legend=False, title="Confirmed pairs by digital / retail availability"
)
ax.invert_yaxis(); ax.set_xlabel("directed pairs"); ax.set_ylabel("digital / retail")
```

	digital	retail	pairs
0	false	true	19568
2	true	true	9893
3	true	unknown	1194
4	unknown	true	240
5	unknown	unknown	24
1	true	false	13

Out[7]: Text(0, 0.5, 'digital / retail')



In [8]:
```
#@title 6. Product and service mix
service_counts = (
    corridor_products.groupby(["service_id", "service_name"], dropna=False).size()
    .rename("product_rows").reset_index().sort_values("product_rows", ascending=False)
)
display(service_counts.head(25))
```

```
payout_mix = corridor_products[["cash_pickup", "bank_payout", "wallet_payout"]].apply(lambda c: c.eq(True).sum()).rename("product_rows")
display(payout_mix.to_frame())
```

	service_id	service_name	product_rows
0	000	Money In Minutes	38937
12	500	Direct to Bank	7533
23	800	Mobile Money Transfer	2431
8	300	Next Day/Delayed Services	548
5	200	Direct To Card	496
10	500	DIRECT TO BANK	373
7	202	Direct To Card	264
6	201	Direct To Card	237
21	800	MOBILE MONEY TRANSFER	195
22	800	MOBILE WALLET	185
14	501	Direct to Bank	153
9	400	Money In Minutes	120
19	600	PREPAID	63
11	500	DIRECT TO BANK ACCOUNT	57
17	50A	BREB	55
27	801	WECHAT WALLET	54
24	801	MOBILE MONEY TRANSFER	51
28	D00	Direct to Bank	50
37	W00	Mobile Money Transfer	50
13	501	DAVIVIENDA BANK/DEPOSITO BANCO	46
30	M00	Money In Minutes	41
2	012	Next Day/Delayed Services	26
20	700	HOME DELIVERY	18
26	801	WAVE WALLET	7
3	099	Money In Minutes	5

	product_rows
cash_pickup	39113
bank_payout	8287
wallet_payout	2975

In [9]:
```
#@title 7. Currency-configuration complexity
configuration_distribution = corridor_universe.configuration_count.value_counts().sort_index().rename("pairs").to_frame()
configuration_distribution["share"] = configuration_distribution.pairs / len(corridor_universe)
display(configuration_distribution.style.format({"share": "{:.1%}"}))
print("Maximum exact currency configurations observed for one pair:", int(corridor_universe.configuration_count.max()))
```

configuration_count	pairs	share
0	7302	13.7%
1	29255	55.1%
2	11472	21.6%
3	3680	6.9%
4	1063	2.0%
6	311	0.6%
8	27	0.1%
9	18	0.0%
12	2	0.0%

```
Maximum exact currency configurations observed for one pair: 12
```

In [10]:

```
#@title 8. Reconciled top-ten modeled corridors
top10_view = top10[[
    "rank", "origin_name", "destination_name", "modeled_revenue_share", "digital", "retail",
    "cash_pickup", "bank_payout", "wallet_payout", "rank_stability", "model_evidence_strength", "limitations"
]].copy()
display(top10_view.style.format({"modeled_revenue_share": "{:.2%}"}))
print("Top-ten modeled share:", f"{top10.modeled_revenue_share.sum():.2%}")
print("Reminder: this is a modeled allocation; WU does not disclose corridor revenue.")
```

	rank	origin_name	destination_name	modeled_revenue_share	digital	retail	cash_pickup	bank_payout	wallet_payout	rank_stability	model_evidence_strength	limitations
0	1	United States	Mexico	7.89%	True	True	True	True	true	HIGH	HIGH	Modeled allocation; WU does not disclose corridor revenue
1	2	United States	India	3.27%	True	True	True	True	unknown	HIGH	HIGH	Modeled allocation; WU does not disclose corridor revenue
2	3	United States	Guatemala	2.36%	True	True	True	True	true	HIGH	HIGH	Modeled allocation; WU does not disclose corridor revenue
3	4	United States	Philippines	1.70%	True	True	True	True	true	HIGH	HIGH	Modeled allocation; WU does not disclose corridor revenue
4	5	United Arab Emirates	India	1.54%	False	True	True	True	unknown	MEDIUM	MEDIUM	Modeled allocation; WU does not disclose corridor revenue; medium rank stability
5	6	United States	Dominican Republic	1.07%	True	True	True	True	unknown	MEDIUM	HIGH	Modeled allocation; WU does not disclose corridor revenue; medium rank stability
6	7	United States	El Salvador	0.91%	True	True	True	True	unknown	MEDIUM	HIGH	Modeled allocation; WU does not disclose corridor revenue; medium rank stability
7	8	United Kingdom	India	0.90%	True	True	True	True	unknown	MEDIUM	MEDIUM	Modeled allocation; WU does not disclose corridor revenue; medium rank stability
8	9	Saudi Arabia	India	0.89%	False	True	True	True	unknown	MEDIUM	MEDIUM	Modeled allocation; WU does not disclose corridor revenue; medium rank stability
9	10	United States	China	0.85%	True	True	True	True	true	MEDIUM	HIGH	Modeled allocation; WU does not disclose corridor revenue; medium rank stability

```
Top-ten modeled share: 21.39%
Reminder: this is a modeled allocation; WU does not disclose corridor revenue.
```

In [11]:

```
#@title 9. Rank and share stability for the final top ten
stability_keyed = rank_stability.dropna(subset=["origin_wu_code", "destination_wu_code"]).drop_duplicates(
    ["origin_wu_code", "destination_wu_code"]
)
stability_view = top10[["rank", "origin_wu_code", "destination_wu_code", "origin_name", "destination_name"]].merge(
    stability_keyed[["origin_wu_code", "destination_wu_code", "rank_min", "rank_max", "median_rank", "top10_frequency", "modeled_share_min", "modeled_share_max", "scenario_count"]],
    on=["origin_wu_code", "destination_wu_code"], how="left", validate="one_to_one"
)
```

```
display(stability_view.style.format({
    "top10_frequency": "{:.0%}", "modeled_share_min": "{:.2%}", "modeled_share_max": "{:.2%}", "median_rank": "{:.1f}"
}))
print("Final top-ten corridors present in all 12 scenarios:", int(stability_view.top10_frequency.eq(1).sum()), "of 10")
```

	rank	origin_wu_code	destination_wu_code	origin_name	destination_name	rank_min	rank_max	median_rank	top10_frequency	modeled_share_min	modeled_share_max	scenario_count
0	1	US	MX	United States	Mexico	1	1	1.0	100%	4.48%	12.22%	12
1	2	US	IN	United States	India	2	2	2.0	100%	2.31%	4.05%	12
2	3	US	GT	United States	Guatemala	3	4	3.0	100%	1.81%	2.70%	12
3	4	US	PH	United States	Philippines	4	6	4.0	100%	1.42%	1.80%	12
4	5	AE	IN	United Arab Emirates	India	3	12	5.0	67%	0.75%	2.74%	12
5	6	US	DO	United States	Dominican Republic	5	11	6.0	83%	0.92%	1.15%	12
6	7	US	SV	United States	El Salvador	6	14	7.5	67%	0.75%	1.02%	12
7	8	GB	IN	United Kingdom	India	5	23	9.0	67%	0.42%	1.69%	12
8	9	SA	IN	Saudi Arabia	India	7	18	8.5	67%	0.50%	1.38%	12
9	10	US	CN	United States	China	7	17	10.5	50%	0.69%	0.97%	12

```
Final top-ten corridors present in all 12 scenarios: 4 of 10
```

In [12]:

```
#@title 10. US → Mexico detailed quote offers
quote_rows = []
for service in US_MX_raw_quote["response"].get("services_groups", []):
    service_fields = {key: value for key, value in service.items() if key != "pay_groups"}
    for offer in service.get("pay_groups", []):
        quote_rows.append({**service_fields, **offer})
US_MX_QUOTE_OFFERS = pd.DataFrame(quote_rows)
offer_columns = [
    "service", "service_name", "fund_out", "fund_out_mnem", "fund_in", "send_amount", "receive_amount",
    "gross_fee", "fx_rate", "fund_in_speed", "fund_out_speed", "speed_indicator", "min_amount", "max_amount", "max_payout"
]
display(US_MX_QUOTE_OFFERS[[c for c in offer_columns if c in US_MX_QUOTE_OFFERS]].sort_values(["service", "gross_fee", "fund_in"]))
print("Detailed offers:", len(US_MX_QUOTE_OFFERS))
print("Raw response status:", US_MX_raw_quote["response"]["response_status"])
```

	service	service_name	fund_out	fund_out_mnem	fund_in	send_amount	receive_amount	gross_fee	fx_rate	fund_in_speed	fund_out_speed	speed_indicator	min_amount	max_amount	max_payout
2	000	Money In Minutes	000	MM	CA	100	1624	3.00	16.2357	0	0	0	0	1.000000e+10	126759.18
1	000	Money In Minutes	000	MM	AC	100	1664	3.99	16.6330	4	0	0-4	0	1.000000e+10	126759.18
4	000	Money In Minutes	000	MM	AD	100	1664	3.99	16.6330	0	0	0	0	1.000000e+09	126759.18
7	000	Money In Minutes	000	MM	AP	100	1664	3.99	16.6330	0	0	0	0	1.000000e+10	126759.18
6	000	Money In Minutes	000	MM	DC	100	1664	3.99	16.6330	0	0	0	0	1.000000e+10	126759.18
3	000	Money In Minutes	000	MM	AR	100	1664	6.99	16.6330	0	0	0	0	1.000000e+09	126759.18
0	000	Money In Minutes	000	MM	CC	100	1664	6.99	16.6330	0	0	0	0	1.000000e+10	126759.18
5	000	Money In Minutes	000	MM	GP	100	1664	6.99	16.6330	0	0	0	0	1.000000e+10	126759.18
9	500	Direct to Bank	500	DB	AC	100	1655	1.99	16.5485	4	0	0-4	0	1.000000e+10	126759.18
12	500	Direct to Bank	500	DB	AD	100	1655	1.99	16.5485	0	0	0	0	1.000000e+09	126759.18
15	500	Direct to Bank	500	DB	AP	100	1655	1.99	16.5485	0	0	0	0	1.000000e+10	126759.18
14	500	Direct to Bank	500	DB	DC	100	1655	1.99	16.5485	0	0	0	0	1.000000e+10	126759.18
10	500	Direct to Bank	500	DB	CA	100	1647	3.00	16.4639	0	0	0-1	0	1.000000e+10	126759.18
11	500	Direct to Bank	500	DB	AR	100	1655	4.99	16.5485	0	0	0	0	1.000000e+09	126759.18
8	500	Direct to Bank	500	DB	CC	100	1655	4.99	16.5485	0	0	0	0	1.000000e+10	126759.18
13	500	Direct to Bank	500	DB	GP	100	1655	4.99	16.5485	0	0	0	0	1.000000e+10	126759.18
18	800	Mobile Money Transfer	800	MB	AC	100	1655	1.99	16.5485	4	0	0-4	0	1.000000e+10	84500.49
19	800	Mobile Money Transfer	800	MB	AD	100	1655	1.99	16.5485	0	0	0	0	1.000000e+09	84500.49
22	800	Mobile Money Transfer	800	MB	AP	100	1655	1.99	16.5485	0	0	0	0	1.000000e+10	84500.49
21	800	Mobile Money Transfer	800	MB	DC	100	1655	1.99	16.5485	0	0	0	0	1.000000e+10	84500.49
17	800	Mobile Money Transfer	800	MB	AR	100	1655	4.99	16.5485	0	0	0	0	1.000000e+09	84500.49
16	800	Mobile Money Transfer	800	MB	CC	100	1655	4.99	16.5485	0	0	0	0	1.000000e+10	84500.49
20	800	Mobile Money Transfer	800	MB	GP	100	1655	4.99	16.5485	0	0	0	0	1.000000e+10	84500.49

```
Detailed offers: 23
Raw response status: {'code': 'P0000', 'long_message': 'NO ERRORS--NO WARNINGS.6313493.2026-09-07 13:18:56.218_2026-09-07 13:18:56.340', 'message': 'OK.SUCCESSFUL PROCESSING', 'status': 0}
```

In [13]:

```python
#@title 11. Field inventory by dataset
field_inventory = pd.concat([
    pd.DataFrame({"dataset": name, "field": frame.columns, "dtype": [str(dtype) for dtype in frame.dtypes]})
    for name, frame in DATA.items()
], ignore_index=True)
display(field_inventory)
print("Tabular fields cataloged:", len(field_inventory))
print("US→MX full dossier sections:", list(US_MX_pair_dossier["records"]))
```

	dataset	field	dtype
0	corridor_products	corridor_id	str
1	corridor_products	origin_wu_code	str
2	corridor_products	destination_wu_code	str
3	corridor_products	send_currency	str
4	corridor_products	payout_currency	str
...	...	...	...
126	top10	bank_payout	bool
127	top10	wallet_payout	str
128	top10	rank_stability	str
129	top10	model_evidence_strength	str
130	top10	limitations	str

131 rows × 3 columns

```
Tabular fields cataloged: 131
US→MX full dossier sections: ['corridor_universe', 'corridor_products', 'rank_stability', 'reconciled_top10', 'graph_markets', 'graph_corridors', 'graph_corridor_channels', 'graph_edges',
'q12_q13_corridor_summary']
```

In [14]:
```
#@title 12. Scratchpad — choose another directional pair
ORIGIN = "US"
DESTINATION = "IN"
CUSTOM_PAIR = select_pair(ORIGIN, DESTINATION)
show_pair(CUSTOM_PAIR)
```

	dataset	records	fields
0	corridor_products	2	24
1	corridor_universe	1	30
2	graph_channels	1	13
3	graph_corridors	1	10
4	markets	2	5
5	q12_q13_summary	1	16
6	rank_stability	1	17
7	top10	1	16

```
corridor_products: 2 record(s)
```

	corridor_id	origin_wu_code	destination_wu_code	send_currency	payout_currency	product_row_id	service_id	service_name	fund_out	fund_out_mnemonic	...	wallet_payout	funding_methods	payout_methods	speed	request_hash	response_hash	payload_fingerprint	timestamp	http_status	raw_evidence_path
49273	US->IN	US	IN	USD	INR	a3f2e4c81ffbce55bf5d6197be4114c2113b2045a4dd70...	000	Money In Minutes	000	MM	...	None	["AC","AD","AP","AR","CA","CC","DC","GP"]	["000", "MM"]	0|0-4	a3f2e4c81ffbce55bf5d6197be4114c2113b2045a4dd70...	39641a8110edfea7a1bce9b61a828045ba2fae6a4afeff...	9d8af152622bd2193e2a9a0898664d2fe0f4b0ff25699e...	2026-09-07T13:18:52+00:00	NaN	raw\quotes\a3f2e4c81ffbce55bf5d6197be4114c2113...
49274	US->IN	US	IN	USD	INR	a3f2e4c81ffbce55bf5d6197be4114c2113b2045a4dd70...	500	Direct to Bank	500	DB	...	None	["AC","AD","AP","AR","CA","CC","DC","GP"]	["500", "DB"]	0|0-1|0-4	a3f2e4c81ffbce55bf5d6197be4114c2113b2045a4dd70...	39641a8110edfea7a1bce9b61a828045ba2fae6a4afeff...	9d8af152622bd2193e2a9a0898664d2fe0f4b0ff25699e...	2026-09-07T13:18:52+00:00	NaN	raw\quotes\a3f2e4c81ffbce55bf5d6197be4114c2113...

2 rows × 24 columns

```
corridor_universe: 1 record(s)
```

	corridor_id	origin_wu_code	destination_wu_code	origin_name	destination_name	origin_iso2_secondary	destination_iso2_secondary	origin_iso3_secondary	destination_iso3_secondary	origin_mapping_status	...	payout_currencies	request_hashes	response_hashes	latest_timestamp	digital	retail	cash_pickup	bank_payout	wallet_payout	channel_evidence_basis
48621	US->IN	US	IN	United States	India	US	IN	USA	IND	direct_code	...	["INR"]	["a3f2e4c81ffbce55bf5d6197be4114c2113b2045a4dd...	["39641a8110edfea7a1bce9b61a828045ba2fae6a4afe...	2026-09-07T13:18:52+00:00	true	true	true	true	unknown	pair_product_response

1 rows × 30 columns

graph_channels: 1 record(s)

	corridor_id	origin_wu_code	destination_wu_code	digital	retail	cash_pickup	account_payout	wallet_payout	funding_methods	payout_methods	products	product_status	channel_evidence_source
22665	US->IN	US	IN	true	true	true	true	unknown	["AC","AD","AP","AR","CA","CC","DC","GP"]	["000","500","DB","MM"]	["Direct to Bank","Money In Minutes"]	PRODUCT_CONFIRMED	dcaas_country_list|price_corridors|smo_config

graph_corridors: 1 record(s)

	corridor_id	origin_wu_code	destination_wu_code	origin_name	destination_name	origin_canonical_iso2	destination_canonical_iso2	website_destination_observed	corridor_confirmed	product_status
22665	US->IN	US	IN	United States	India	US	IN	True	true	PRODUCT_CONFIRMED

markets: 2 record(s)

	market_id	wu_market_name	iso2_secondary	iso3_secondary	normalization_notes
91	IN	India	IN	IND	direct_code
211	US	United States	US	USA	direct_code

q12_q13_summary: 1 record(s)

	corridor	modeled_revenue_share	send_wu_sites	receive_wu_sites	send_confirmed_multibrand_pct	receive_confirmed_multibrand_pct	send_wu_p50_miles	send_wu_p90_miles	receive_wu_p50_miles	receive_wu_p90_miles	send_moneygram_p50_miles	send_ria_p50_miles	receive_moneygram_p50_miles	receive_ria_p50_miles	wu_relative_access_advantage	service_exclusivity_evidence_quality
1	US->IN	0.033718	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	send:NO_WU_AFFIRMATIVE_CONTRACT_EVIDENCE;recei...

rank_stability: 1 record(s)

	corridor	send_iso3	receive_iso3	origin_wu_code	destination_wu_code	rank_min	rank_max	median_rank	top10_frequency	modeled_share_min	modeled_share_max	rank_stability	product_confirmed	product_status	digital	retail	scenario_count
1	United States of America -> India	USA	IND	US	IN	2	2	2.0	1.0	0.023103	0.040547	HIGH	True	PRODUCT_CONFIRMED	true	true	12

top10: 1 record(s)

	rank	origin_wu_code	destination_wu_code	origin_name	destination_name	modeled_revenue_share	share_perimeter	product_confirmed	digital	retail	cash_pickup	bank_payout	wallet_payout	rank_stability	model_evidence_strength	limitations
1	2	US	IN	United States	India	0.032664	FY2025 consolidated revenue; cross-border CMT ...	True	True	True	True	True	unknown	HIGH	HIGH	Modeled allocation; WU does not disclose corri...

RPW → Western Union competitive-footprint reconstruction

This section recreates and improves the Statista-style Western Union sender/receiver "market share" charts directly from the World Bank Remittance Prices Worldwide (`rpw_history`) raw observations. These metrics are **shares of observed RPW provider/product quotes, not transaction-value market share**.

Outputs:

- corridor-level WU presence, quote share, provider share, competitive intensity and price rank;
- sender-country and receiver-country WU footprint tables;
- cash vs digital WU footprint where channel fields exist;
- latest-period and historical trend views;
- merge to `bilateral_remittance_2024` for economic weighting;
- diagnostics that flag schema mismatches and coverage limitations.

The cells are schema-tolerant because RPW workbook column names can vary by vintage. Run the roadmap-pack loader first so `rpw_history` and `bilateral_remittance_2024` exist.

In [15]:
```
#@title 13. RPW schema discovery and normalization
import re
import numpy as np
import pandas as pd
from IPython.display import display

if "rpw_history" not in globals():
    raise NameError("Run the roadmap-pack loader first; rpw_history is not loaded.")

RPW = rpw_history.copy()
print(f"rpw_history: {len(RPW):,} rows × {RPW.shape[1]} columns")
display(pd.DataFrame({"column": RPW.columns, "dtype": RPW.dtypes.astype(str).values}))

# Flexible column resolver. It first prefers exact normalized names, then substring matches.
def _norm_name(x):
    return re.sub(r"[^a-z0-9]+", "_", str(x).strip().lower()).strip("_")

_norm_to_original = {_norm_name(c): c for c in RPW.columns}

def pick_col(*candidates, required=False):
    norms = [_norm_name(x) for x in candidates]
    for n in norms:
        if n in _norm_to_original:
            return _norm_to_original[n]
    for n in norms:
        hits = [orig for nn, orig in _norm_to_original.items() if n in nn or nn in n]
        if len(hits) == 1:
            return hits[0]
    if required:
        raise KeyError(f"Could not resolve any of {candidates}. Available columns: {list(RPW.columns)}")
    return None

COL = {
    "send": pick_col("source_name", "send_country", "sending_country", "source_country", "origin_country", "send economy", required=True),
    "receive": pick_col("destination_name", "receive_country", "receiving_country", "destination_country", "recipient_country", "receive economy", required=True),
    "provider": pick_col("firm", "provider", "service_provider", "mto", required=True),
    "period": pick_col("period", "quarter", "survey_period", "date", "year_quarter", "time"),
    "year": pick_col("year"),
    "quarter": pick_col("quarter", "qtr"),
    "product": pick_col("product", "product_name", "service", "service_name"),
    "channel": pick_col("access_point", "channel", "send_channel", "access channel"),
```

```
    "payment": pick_col("payment_instrument", "payment_method", "funding_method"),
    "receive_method": pick_col("pickup_method", "receiving_method", "receive_method", "payout_method", "delivery_method"),
    "cost": pick_col("cc1_total_cost", "cc1 total cost %", "total_cost_percent", "total_cost", "total_cost_percentage", "cost_percent", "total cost %"),
    "fee": pick_col("cc1_lcu_fee", "cc1 lcu fee", "fee", "transfer_fee", "fee_percent"),
    "fx_margin": pick_col("cc1_fx_margin", "cc1 fx margin", "exchange_rate_margin", "fx_margin", "exchange_rate_margin_percent"),
    "speed": pick_col("speed_actual", "speed actual", "speed", "transfer_speed", "time"),
}
print("Resolved columns:")
display(pd.Series(COL, name="RPW column").to_frame())

# Canonical normalized fields.
r = pd.DataFrame(index=RPW.index)
r["send_country"] = RPW[COL["send"]].astype(str).str.strip()
r["receive_country"] = RPW[COL["receive"]].astype(str).str.strip()
r["provider"] = RPW[COL["provider"]].astype(str).str.strip()
r["provider_norm"] = r["provider"].str.lower().str.replace(r"[^a-z0-9]+", " ", regex=True).str.strip()
r["is_wu"] = r["provider_norm"].str.contains(r"\bwestern\s+union\b|\bwesternunion\b", regex=True, na=False)
r["corridor"] = r["send_country"] + " → " + r["receive_country"]

for key in ["product","channel","payment","receive_method","speed"]:
    r[key] = RPW[COL[key]].astype(str).str.strip() if COL[key] else pd.NA
for key in ["cost","fee","fx_margin"]:
    r[key] = pd.to_numeric(RPW[COL[key]], errors="coerce") if COL[key] else np.nan

# Build a sortable period key.
if COL["period"]:
    r["period_raw"] = RPW[COL["period"]].astype(str)
elif COL["year"]:
    q = RPW[COL["quarter"]].astype(str) if COL["quarter"] else ""
    r["period_raw"] = RPW[COL["year"]].astype(str) + (" Q" + q if COL["quarter"] else "")
else:
    r["period_raw"] = "all"

def period_sort_key(x):
    s = str(x)
    y = re.search(r"(20\d{2}|19\d{2})", s)
    q = re.search(r"(?:Q|quarter\s*)([1-4])", s, flags=re.I)
    return (int(y.group(1)) if y else -1, int(q.group(1)) if q else 0, s)

periods = sorted(r["period_raw"].dropna().unique(), key=period_sort_key)
LATEST_RPW_PERIOD = periods[-1] if periods else "all"
print("Detected latest RPW period:", LATEST_RPW_PERIOD)
print("WU rows:", f"{int(r.is_wu.sum()):,}", f"({r.is_wu.mean():.2%} of raw RPW rows)")
print("Observed corridors:", f"{r.corridor.nunique():,}")
```

```
rpw_history: 253,960 rows × 49 columns
```

	column	dtype
0	id	str
1	period	str
2	source_code	str
3	source_name	str
4	source_region	str
5	source_income	str
6	source_lending	str
7	source_G8G20	str
8	destination_code	str
9	destination_name	str
10	destination_region	str
11	destination_income	str
12	destination_lending	str
13	destination_G8G20	str
14	firm	str
15	firm_type	str
16	product	str
17	payment instrument	str
18	access point	str
19	sending location	str
20	speed actual	str
21	cc1 lcu amount	str
22	cc1 denomination amount	str
23	cc1 lcu code	str
24	cc1 lcu fee	str
25	cc1 lcu fx rate	str
26	cc1 fx margin	str
27	cc1 total cost %	str
28	cc2 lcu amount	str
29	cc2 denomination amount	str
30	cc2 lcu code	str
31	cc2 lcu fee	str
32	cc2 lcu fx rate	str
33	cc2 fx margin	str
34	cc2 total cost %	str
35	inter lcu bank fx	str
36	transparent	str
37	Standard Note	str
38	note1	str
39	note2	str
40	receiving network coverage	str
41	coverage	str
42	pickup location	str

	column	dtype
43	pickup method	str
44	pick-up method	str
45	date	str
46	corridor	str
47	source_sheet	str
48	source_row	str

```
Resolved columns:
```

	RPW column
send	source_name
receive	destination_name
provider	firm
period	period
year	NaN
quarter	NaN
product	product
channel	access point
payment	payment instrument
receive_method	pickup method
cost	cc1 total cost %
fee	cc1 lcu fee
fx_margin	cc1 fx margin
speed	speed actual

```
Detected latest RPW period: 2025_3Q
WU rows: 44,368 (17.47% of raw RPW rows)
Observed corridors: 388
```

In [16]:

```python
#@title 14. Latest-period WU footprint by corridor
LATEST = r.loc[r["period_raw"].eq(LATEST_RPW_PERIOD)].copy()
if LATEST.empty:
    raise ValueError("Latest-period filter returned zero rows; inspect period_raw values above.")

# One provider count per corridor; quote/product rows remain observation-level.
def corridor_metrics(g):
    providers = g["provider_norm"].dropna()
    wu = g[g["is_wu"]]
    provider_count = providers.nunique()
    return pd.Series({
        "rpw_quotes": len(g),
        "wu_quotes": len(wu),
        "wu_quote_share": len(wu) / len(g) if len(g) else np.nan,
        "providers": provider_count,
        "wu_present": bool(len(wu)),
        "wu_provider_share": (1 / provider_count) if len(wu) and provider_count else 0.0,
        "wu_products": wu["product"].nunique(dropna=True) if "product" in wu else np.nan,
        "all_products": g["product"].nunique(dropna=True) if "product" in g else np.nan,
        "median_cost_all": g["cost"].median(),
        "median_cost_wu": wu["cost"].median(),
        "min_cost_all": g["cost"].min(),
        "min_cost_wu": wu["cost"].min(),
    })
```

```python
RPW_WU_CORRIDOR_LATEST = (
    LATEST.groupby(["send_country","receive_country","corridor"], dropna=False)
    .apply(corridor_metrics, include_groups=False)
    .reset_index()
)
RPW_WU_CORRIDOR_LATEST["wu_cost_premium_vs_median"] = (
    RPW_WU_CORRIDOR_LATEST["median_cost_wu"] - RPW_WU_CORRIDOR_LATEST["median_cost_all"]
)
RPW_WU_CORRIDOR_LATEST["wu_cost_premium_pct"] = (
    RPW_WU_CORRIDOR_LATEST["median_cost_wu"] / RPW_WU_CORRIDOR_LATEST["median_cost_all"] - 1
)

# Corridor-level cost percentile for WU: 0 = cheapest, 1 = most expensive.
def wu_cost_percentile(g):
    d = g.dropna(subset=["cost"]).copy()
    if d.empty or not d["is_wu"].any():
        return np.nan
    d["pct_rank"] = d["cost"].rank(method="average", pct=True)
    return d.loc[d.is_wu, "pct_rank"].median()

cost_pct = (
    LATEST.groupby(["send_country","receive_country"], dropna=False)
    .apply(wu_cost_percentile, include_groups=False)
    .rename("wu_cost_percentile")
    .reset_index()
)
RPW_WU_CORRIDOR_LATEST = RPW_WU_CORRIDOR_LATEST.merge(cost_pct, on=["send_country","receive_country"], how="left")

print("Latest-period corridor footprint")
display(
    RPW_WU_CORRIDOR_LATEST.sort_values(["wu_present","wu_quote_share","rpw_quotes"], ascending=[False,False,False])
    .head(40)
    .style.format({"wu_quote_share":"{:.1%}","wu_provider_share":"{:.1%}","wu_cost_premium_pct":"{:.1%}","wu_cost_percentile":"{:.0%}"})
)
```

```
Latest-period corridor footprint
```

	send_country	receive_country	corridor	rpw_quotes	wu_quotes	wu_quote_share	providers	wu_present	wu_provider_share	wu_products	all_products	median_cost_all	median_cost_wu	min_cost_all	min_cost_wu	wu_cost_premium_vs_median	wu_cost_premium_pct	wu_cost_percentile
200	Saudi Arabia	South Sudan	Saudi Arabia → South Sudan	1	1	100.0%	1	True	100.0%	0	0	4.830000	4.830000	4.830000	4.830000	0.000000	0.0%	100%
203	Saudi Arabia	Syrian Arab Republic	Saudi Arabia → Syrian Arab Republic	1	1	100.0%	1	True	100.0%	0	0	8.880000	8.880000	8.880000	8.880000	0.000000	0.0%	100%
298	United Kingdom	South Sudan	United Kingdom → South Sudan	10	7	70.0%	4	True	25.0%	0	0	7.060000	6.230000	4.750000	4.750000	-0.830000	-11.8%	45%
186	Qatar	Sudan	Qatar → Sudan	3	2	66.7%	2	True	50.0%	0	0	33.690000	33.690000	2.230000	33.690000	0.000000	0.0%	83%
196	Saudi Arabia	Myanmar	Saudi Arabia → Myanmar	3	2	66.7%	3	True	33.3%	0	0	4.930000	4.655000	4.380000	4.380000	-0.275000	-5.6%	50%
269	United Arab Emirates	South Sudan	United Arab Emirates → South Sudan	3	2	66.7%	2	True	50.0%	0	0	2.970000	3.685000	2.700000	2.700000	0.715000	24.1%	67%
32	Brazil	Paraguay	Brazil → Paraguay	10	6	60.0%	4	True	25.0%	0	0	6.430000	5.485000	2.740000	2.740000	-0.945000	-14.7%	40%
72	Germany	Afghanistan	Germany → Afghanistan	5	3	60.0%	2	True	50.0%	0	0	7.730000	7.730000	6.680000	7.730000	0.000000	0.0%	70%
83	Germany	Lebanon	Germany → Lebanon	5	3	60.0%	2	True	50.0%	0	0	4.320000	4.320000	3.000000	4.320000	0.000000	0.0%	70%
246	Switzerland	Hungary	Switzerland → Hungary	13	7	53.8%	4	True	25.0%	0	0	3.120000	2.820000	0.630000	0.630000	-0.300000	-9.6%	46%
158	New Zealand	China	New Zealand → China	28	14	50.0%	9	True	11.1%	0	0	3.920000	3.750000	1.000000	1.420000	-0.170000	-4.3%	50%
61	France	Haiti	France → Haiti	8	4	50.0%	5	True	20.0%	0	0	4.695000	4.780000	4.100000	4.490000	0.085000	1.8%	59%
73	Germany	Albania	Germany → Albania	6	3	50.0%	3	True	33.3%	0	0	4.580000	4.580000	3.210000	4.580000	0.000000	0.0%	58%
184	Qatar	Philippines	Qatar → Philippines	6	3	50.0%	4	True	25.0%	0	0	2.615000	2.290000	0.810000	0.810000	-0.325000	-12.4%	42%
0	Angola	Namibia	Angola → Namibia	2	1	50.0%	2	True	50.0%	0	0	13.290000	9.940000	9.940000	9.940000	-3.350000	-25.2%	50%
34	Cameroon	Nigeria	Cameroon → Nigeria	2	1	50.0%	2	True	50.0%	0	0	4.380000	4.130000	4.130000	4.130000	-0.250000	-5.7%	50%
188	Saudi Arabia	Afghanistan	Saudi Arabia → Afghanistan	2	1	50.0%	2	True	50.0%	0	0	5.755000	4.860000	4.860000	4.860000	-0.895000	-15.6%	50%
195	Saudi Arabia	Lebanon	Saudi Arabia → Lebanon	2	1	50.0%	2	True	50.0%	0	0	5.570000	5.620000	5.520000	5.620000	0.050000	0.9%	100%
65	France	Mali	France → Mali	9	4	44.4%	6	True	16.7%	0	0	2.860000	2.945000	1.230000	2.570000	0.085000	3.0%	58%
248	Switzerland	Serbia	Switzerland → Serbia	9	4	44.4%	3	True	33.3%	0	0	5.780000	3.545000	3.250000	3.250000	-2.235000	-38.7%	25%
256	Thailand	Indonesia	Thailand → Indonesia	16	7	43.8%	8	True	12.5%	0	0	12.470000	4.470000	2.450000	2.450000	-8.000000	-64.2%	22%
274	United Kingdom	Albania	United Kingdom → Albania	30	13	43.3%	6	True	16.7%	0	0	4.920000	4.410000	1.770000	1.770000	-0.510000	-10.4%	38%
87	Germany	North Macedonia	Germany → North Macedonia	7	3	42.9%	3	True	33.3%	0	0	3.560000	3.560000	0.000000	3.560000	0.000000	0.0%	50%
245	Switzerland	Albania	Switzerland → Albania	7	3	42.9%	4	True	25.0%	0	0	7.710000	7.710000	5.360000	7.710000	0.000000	0.0%	64%
231	Spain	China	Spain → China	12	5	41.7%	5	True	20.0%	0	0	5.135000	5.870000	3.750000	5.870000	0.735000	14.3%	75%

	send_country	receive_country	corridor	rpw_quotes	wu_quotes	wu_quote_share	providers	wu_present	wu_provider_share	wu_products	all_products	median_cost_all	median_cost_wu	min_cost_all	min_cost_wu	wu_cost_premium_vs_median	wu_cost_premium_pct	wu_cost_percentile
89	Germany	Serbia	Germany → Serbia	17	7	41.2%	7	True	14.3%	0	0	1.780000	3.560000	0.000000	2.660000	1.780000	100.0%	82%
249	Switzerland	Sri Lanka	Switzerland → Sri Lanka	17	7	41.2%	5	True	20.0%	0	0	4.790000	5.440000	2.280000	4.790000	0.650000	13.6%	76%
254	Thailand	China	Thailand → China	17	7	41.2%	10	True	10.0%	0	0	17.230000	4.530000	2.510000	2.510000	-12.700000	-73.7%	21%
259	Thailand	Vietnam	Thailand → Vietnam	22	9	40.9%	11	True	9.1%	0	0	12.320000	4.750000	2.300000	2.300000	-7.570000	-61.4%	23%
284	United Kingdom	Jamaica	United Kingdom → Jamaica	20	8	40.0%	6	True	16.7%	0	0	4.860000	5.060000	-1.270000	-1.270000	0.200000	4.1%	70%
177	Portugal	Mozambique	Portugal → Mozambique	15	6	40.0%	4	True	25.0%	0	0	3.070000	4.280000	1.800000	3.070000	1.210000	39.4%	60%
181	Qatar	Jordan	Qatar → Jordan	5	2	40.0%	4	True	25.0%	0	0	3.620000	2.460000	2.460000	2.460000	-1.160000	-32.0%	30%
165	New Zealand	Vietnam	New Zealand → Vietnam	28	11	39.3%	11	True	9.1%	0	0	5.735000	5.580000	1.390000	3.190000	-0.155000	-2.7%	48%
79	Germany	Hungary	Germany → Hungary	18	7	38.9%	5	True	20.0%	0	0	6.070000	13.300000	2.050000	6.720000	7.230000	119.1%	89%
84	Germany	Moldova	Germany → Moldova	13	5	38.5%	5	True	20.0%	0	0	3.500000	3.130000	2.760000	2.790000	-0.370000	-10.6%	42%
255	Thailand	India	Thailand → India	21	8	38.1%	11	True	9.1%	0	0	11.760000	4.685000	0.720000	0.720000	-7.075000	-60.2%	31%
149	Malaysia	Vietnam	Malaysia → Vietnam	16	6	37.5%	7	True	14.3%	0	0	2.800000	2.635000	2.060000	2.380000	-0.165000	-5.9%	34%
257	Thailand	Lao PDR	Thailand → Lao PDR	16	6	37.5%	10	True	10.0%	0	0	15.520000	3.525000	2.740000	2.740000	-11.995000	-77.3%	23%
31	Brazil	Bolivia	Brazil → Bolivia	8	3	37.5%	6	True	16.7%	0	0	18.685000	17.870000	5.790000	16.830000	-0.815000	-4.4%	50%
42	Canada	Lebanon	Canada → Lebanon	8	3	37.5%	4	True	25.0%	0	0	6.050000	6.050000	2.100000	6.050000	0.000000	0.0%	56%

In [17]:

```
#@title 15. Recreate Statista-style WU footprint by sender and receiver
# IMPORTANT: these are RPW observation/provider-presence metrics, NOT actual remittance market shares.

def country_footprint(df, side):
    other = "receive_country" if side == "send_country" else "send_country"
    rows = []
    for country, g in df.groupby(side, dropna=False):
        corridors = g[[side, other]].drop_duplicates()
        wu_corridors = g.loc[g.is_wu, [side, other]].drop_duplicates()
        providers_by_corr = g.groupby(other)["provider_norm"].nunique()
        wu_by_corr = g[g.is_wu].groupby(other).size().gt(0)
        provider_share = wu_by_corr.reindex(providers_by_corr.index, fill_value=False).astype(float).div(providers_by_corr)
        rows.append({
            side: country,
            "rpw_quotes": len(g),
            "wu_quotes": int(g.is_wu.sum()),
            "wu_quote_share": g.is_wu.mean(),
            "corridors_observed": len(corridors),
            "wu_corridors_present": len(wu_corridors),
            "wu_corridor_presence_rate": len(wu_corridors)/len(corridors) if len(corridors) else np.nan,
            "avg_wu_provider_share_across_corridors": provider_share.mean() if len(provider_share) else np.nan,
            "median_wu_cost": g.loc[g.is_wu,"cost"].median(),
            "median_market_cost": g["cost"].median(),
```

```
        })
    out = pd.DataFrame(rows)
    out["wu_cost_premium_pct"] = out["median_wu_cost"] / out["median_market_cost"] - 1
    return out

RPW_WU_BY_SENDER = country_footprint(LATEST, "send_country").sort_values("wu_quote_share", ascending=False, ignore_index=True)
RPW_WU_BY_RECEIVER = country_footprint(LATEST, "receive_country").sort_values("wu_quote_share", ascending=False, ignore_index=True)

print("Statista-style reconstruction — sender countries")
display(RPW_WU_BY_SENDER.head(30).style.format({c:"{:.1%}" for c in ["wu_quote_share","wu_corridor_presence_rate","avg_wu_provider_share_across_corridors","wu_cost_premium_pct"]}))
print("Statista-style reconstruction — receiver countries")
display(RPW_WU_BY_RECEIVER.head(30).style.format({c:"{:.1%}" for c in ["wu_quote_share","wu_corridor_presence_rate","avg_wu_provider_share_across_corridors","wu_cost_premium_pct"]}))
```

```
Statista-style reconstruction — sender countries
```

	send_country	rpw_quotes	wu_quotes	wu_quote_share	corridors_observed	wu_corridors_present	wu_corridor_presence_rate	avg_wu_provider_share_across_corridors	median_wu_cost	median_market_cost	wu_cost_premium_pct
0	Angola	2	1	50.0%	1	1	100.0%	50.0%	9.940000	13.290000	-25.2%
1	Cameroon	2	1	50.0%	1	1	100.0%	50.0%	4.130000	4.380000	-5.7%
2	Brazil	28	12	42.9%	3	3	100.0%	20.6%	6.045000	6.245000	-3.2%
3	Switzerland	57	24	42.1%	5	5	100.0%	24.0%	5.115000	5.440000	-6.0%
4	Portugal	32	12	37.5%	3	3	100.0%	27.8%	4.850000	4.135000	17.3%
5	Thailand	128	47	36.7%	7	7	100.0%	10.1%	4.470000	14.045000	-68.2%
6	Israel	3	1	33.3%	1	1	100.0%	33.3%	16.980000	13.470000	26.1%
7	Dominican Republic	3	1	33.3%	1	1	100.0%	33.3%	7.910000	7.910000	0.0%
8	Qatar	49	16	32.7%	9	9	100.0%	25.7%	2.715000	3.130000	-13.3%
9	Oman	42	12	28.6%	6	6	100.0%	19.9%	3.300000	3.310000	-0.3%
10	New Zealand	225	64	28.4%	8	8	100.0%	9.6%	3.700000	4.510000	-18.0%
11	Poland	22	6	27.3%	1	1	100.0%	14.3%	3.010000	3.010000	0.0%
12	France	256	67	26.2%	16	15	93.8%	15.7%	4.440000	3.800000	16.8%
13	Belgium	64	15	23.4%	4	3	75.0%	9.8%	3.690000	3.530000	4.5%
14	Czechia	31	7	22.6%	2	2	100.0%	13.4%	3.880000	4.160000	-6.7%
15	Germany	491	101	20.6%	24	23	95.8%	18.8%	4.340000	3.500000	24.0%
16	Spain	216	44	20.4%	13	13	100.0%	14.2%	4.640000	4.095000	13.3%
17	Canada	264	53	20.1%	15	15	100.0%	15.6%	6.010000	4.365000	37.7%
18	Sweden	59	11	18.6%	4	3	75.0%	14.7%	6.730000	4.510000	49.2%
19	United Kingdom	791	144	18.2%	33	33	100.0%	14.9%	4.800000	3.710000	29.4%
20	Türkiye	11	2	18.2%	1	1	100.0%	11.1%	117.975000	113.630000	3.8%
21	Italy	418	72	17.2%	19	19	100.0%	10.5%	3.380000	3.565000	-5.2%
22	Austria	130	22	16.9%	6	6	100.0%	13.6%	3.920000	4.110000	-4.6%
23	South Africa	279	43	15.4%	13	13	100.0%	11.9%	32.770000	10.280000	218.8%
24	Saudi Arabia	120	18	15.0%	17	17	100.0%	33.8%	4.210000	3.860000	9.1%
25	United Arab Emirates	149	22	14.8%	12	12	100.0%	16.9%	3.760000	3.170000	18.6%
26	United States	1010	149	14.8%	42	40	95.2%	13.9%	4.460000	4.375000	1.9%
27	Chile	7	1	14.3%	1	1	100.0%	14.3%	4.510000	4.620000	-2.4%
28	Rwanda	7	1	14.3%	1	1	100.0%	20.0%	15.190000	15.190000	0.0%

	send_country	rpw_quotes	wu_quotes	wu_quote_share	corridors_observed	wu_corridors_present	wu_corridor_presence_rate	avg_wu_provider_share_across_corridors	median_wu_cost	median_market_cost	wu_cost_premium_pct
29	Côte d'Ivoire	7	1	14.3%	1	1	100.0%	20.0%	3.230000	3.230000	0.0%

Statista-style reconstruction — receiver countries

	receive_country	rpw_quotes	wu_quotes	wu_quote_share	corridors_observed	wu_corridors_present	wu_corridor_presence_rate	avg_wu_provider_share_across_corridors	median_wu_cost	median_market_cost	wu_cost_premium_pct
0	Paraguay	10	6	60.0%	1	1	100.0%	25.0%	5.485000	6.430000	-14.7%
1	Namibia	2	1	50.0%	1	1	100.0%	50.0%	9.940000	13.290000	-25.2%
2	Afghanistan	16	7	43.8%	4	4	100.0%	45.8%	7.800000	7.765000	0.5%
3	North Macedonia	7	3	42.9%	1	1	100.0%	33.3%	3.560000	3.560000	0.0%
4	Albania	60	25	41.7%	4	4	100.0%	21.5%	4.580000	5.330000	-14.1%
5	Sudan	10	4	40.0%	3	3	100.0%	41.7%	33.150000	32.480000	2.1%
6	Lao PDR	16	6	37.5%	1	1	100.0%	10.0%	3.525000	15.520000	-77.3%
7	Comoros	11	4	36.4%	1	1	100.0%	20.0%	3.050000	3.140000	-2.9%
8	Madagascar	14	5	35.7%	1	1	100.0%	20.0%	4.630000	4.630000	0.0%
9	Hungary	50	17	34.0%	3	3	100.0%	19.2%	6.720000	3.970000	69.3%
10	Myanmar	24	8	33.3%	3	3	100.0%	23.1%	1.130000	2.635000	-57.1%
11	Congo, Dem. Rep.	15	5	33.3%	1	1	100.0%	16.7%	3.540000	3.260000	8.6%
12	Serbia	66	21	31.8%	5	5	100.0%	21.2%	3.840000	4.890000	-21.5%
13	Togo	26	8	30.8%	2	2	100.0%	22.5%	2.650000	2.880000	-8.0%
14	Angola	13	4	30.8%	1	1	100.0%	12.5%	32.780000	22.210000	47.6%
15	Cabo Verde	20	6	30.0%	2	2	100.0%	29.2%	4.450000	4.805000	-7.4%
16	Bolivia	20	6	30.0%	2	2	100.0%	15.5%	17.140000	17.140000	0.0%
17	Haiti	40	12	30.0%	4	4	100.0%	21.1%	5.020000	5.000000	0.4%
18	Jamaica	55	15	27.3%	3	3	100.0%	14.5%	6.360000	5.060000	25.7%
19	Cambodia	22	6	27.3%	1	1	100.0%	9.1%	2.790000	5.100000	-45.3%
20	China	274	74	27.0%	16	16	100.0%	13.2%	5.015000	4.570000	9.7%
21	Moldova	30	8	26.7%	2	2	100.0%	15.0%	2.990000	3.530000	-15.3%
22	Bosnia and Herzegovina	39	10	25.6%	2	2	100.0%	16.7%	4.105000	4.690000	-12.5%
23	Tonga	43	11	25.6%	2	2	100.0%	9.4%	5.140000	6.610000	-22.2%
24	Cuba	4	1	25.0%	1	1	100.0%	33.3%	17.000000	7.750000	119.4%
25	Tunisia	41	10	24.4%	2	2	100.0%	10.4%	5.145000	4.270000	20.5%
26	South Sudan	46	11	23.9%	5	4	80.0%	37.0%	4.830000	6.230000	-22.5%
27	Vietnam	276	65	23.6%	12	12	100.0%	11.5%	4.320000	4.155000	4.0%
28	Nigeria	193	44	22.8%	10	9	90.0%	16.9%	3.835000	2.870000	33.6%

	receive_country	rpw_quotes	wu_quotes	wu_quote_share	corridors_observed	wu_corridors_present	wu_corridor_presence_rate	avg_wu_provider_share_across_corridors	median_wu_cost	median_market_cost	wu_cost_premium_pct
29	Senegal	44	10	22.7%	2	2	100.0%	12.9%	3.070000	2.835000	8.3%

In [18]:
```
#@title 16. Cash vs digital footprint (best-effort channel classification)
text_fields = [c for c in ["channel","payment","receive_method","product"] if c in LATEST.columns]
channel_text = LATEST[text_fields].fillna("").astype(str).agg(" | ".join, axis=1).str.lower()
LATEST_CH = LATEST.copy()
LATEST_CH["digital_flag"] = channel_text.str.contains(r"online|internet|mobile|app|wallet|bank account|account deposit|debit|credit", regex=True)
LATEST_CH["cash_flag"] = channel_text.str.contains(r"cash|agent|branch|retail|counter|pickup|pick-up", regex=True)
LATEST_CH["channel_bucket"] = np.select(
    [LATEST_CH.cash_flag & ~LATEST_CH.digital_flag, LATEST_CH.digital_flag & ~LATEST_CH.cash_flag, LATEST_CH.cash_flag & LATEST_CH.digital_flag],
    ["cash/retail", "digital/account", "hybrid/ambiguous"],
    default="unclassified"
)

RPW_WU_CHANNEL_MIX = (
    LATEST_CH.groupby("channel_bucket")
    .agg(rpw_quotes=("is_wu","size"), wu_quotes=("is_wu","sum"), corridors=("corridor","nunique"))
    .reset_index()
)
RPW_WU_CHANNEL_MIX["wu_quote_share"] = RPW_WU_CHANNEL_MIX.wu_quotes / RPW_WU_CHANNEL_MIX.rpw_quotes
print("Heuristic classification only; inspect the underlying RPW fields before using this as a hard statistic.")
display(RPW_WU_CHANNEL_MIX.style.format({"wu_quote_share":"{:.1%}"}))
```

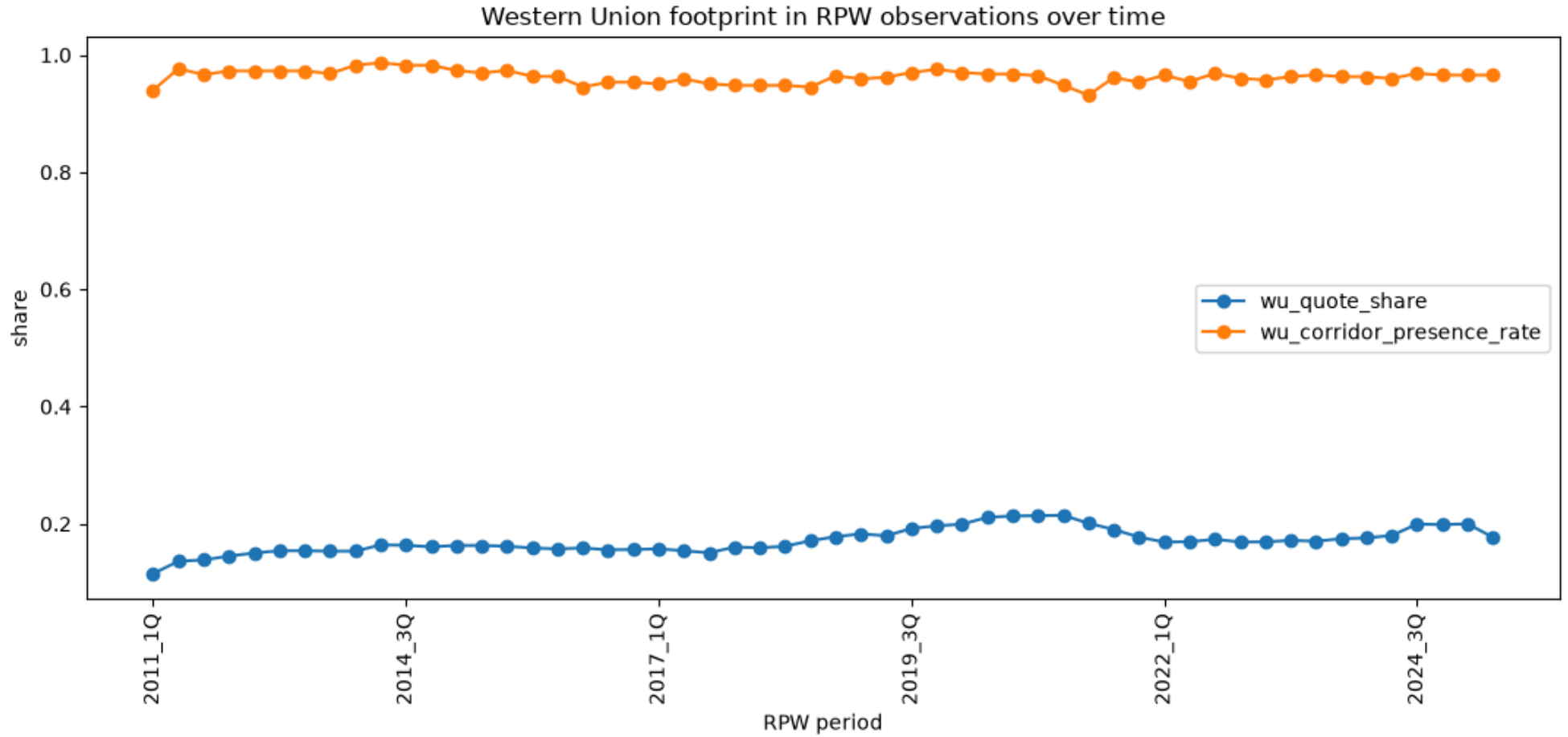
```
Heuristic classification only; inspect the underlying RPW fields before using this as a hard statistic.
```

	channel_bucket	rpw_quotes	wu_quotes	corridors	wu_quote_share
0	cash/retail	887	273	317	30.8%
1	digital/account	2385	215	294	9.0%
2	hybrid/ambiguous	3198	656	334	20.5%

In [19]:
```
#@title 17. Historical trend in WU RPW footprint
RPW_WU_TREND = (
    r.groupby("period_raw", dropna=False)
    .agg(
        rpw_quotes=("is_wu","size"),
        wu_quotes=("is_wu","sum"),
        corridors=("corridor","nunique"),
        wu_corridors=("corridor", lambda x: x[r.loc[x.index,"is_wu"]].nunique()),
        providers=("provider_norm","nunique")
    )
    .reset_index()
)
RPW_WU_TREND["wu_quote_share"] = RPW_WU_TREND.wu_quotes / RPW_WU_TREND.rpw_quotes
RPW_WU_TREND["wu_corridor_presence_rate"] = RPW_WU_TREND.wu_corridors / RPW_WU_TREND.corridors
RPW_WU_TREND["_sort"] = RPW_WU_TREND.period_raw.map(period_sort_key)
RPW_WU_TREND = RPW_WU_TREND.sort_values("_sort").drop(columns="_sort").reset_index(drop=True)
display(RPW_WU_TREND.style.format({"wu_quote_share":"{:.1%}","wu_corridor_presence_rate":"{:.1%}"}))

ax = RPW_WU_TREND.plot(x="period_raw", y=["wu_quote_share","wu_corridor_presence_rate"], figsize=(13,5), marker="o", title="Western Union footprint in RPW observations over time")
ax.set_ylabel("share"); ax.set_xlabel("RPW period"); ax.tick_params(axis="x", rotation=90)
```

	period_raw	rpw_quotes	wu_quotes	corridors	wu_corridors	providers	wu_quote_share	wu_corridor_presence_rate
0	2011_1Q	2603	299	199	187	351	11.5%	94.0%
1	2011_3Q	2668	363	213	208	352	13.6%	97.7%
2	2012_1Q	2638	365	208	201	368	13.8%	96.6%
3	2012_3Q	2907	421	219	213	375	14.5%	97.3%
4	2013_1Q	2924	439	220	214	372	15.0%	97.3%
5	2013_2Q	2948	454	220	214	374	15.4%	97.3%
6	2013_3Q	2941	453	220	214	374	15.4%	97.3%
7	2013_4Q	2904	445	220	213	385	15.3%	96.8%
8	2014_1Q	2983	457	226	222	403	15.3%	98.2%
9	2014_2Q	2951	484	226	223	406	16.4%	98.7%
10	2014_3Q	2890	472	226	222	399	16.3%	98.2%
11	2014_4Q	2881	463	226	222	395	16.1%	98.2%
12	2015_1Q	2821	460	226	220	396	16.3%	97.3%
13	2015_2Q	2794	455	227	220	395	16.3%	96.9%
14	2015_3Q	2831	458	227	221	397	16.2%	97.4%
15	2015_4Q	3397	539	300	289	447	15.9%	96.3%
16	2016_1Q	3410	536	300	289	447	15.7%	96.3%
17	2016_2Q	3905	620	365	345	484	15.9%	94.5%
18	2016_3Q	3958	616	365	348	490	15.6%	95.3%
19	2016_4Q	3932	614	365	348	494	15.6%	95.3%
20	2017_1Q	3954	622	365	347	490	15.7%	95.1%
21	2017_2Q	4093	630	365	350	491	15.4%	95.9%
22	2017_3Q	4174	628	365	347	501	15.0%	95.1%
23	2017_4Q	4185	671	365	346	506	16.0%	94.8%
24	2018_1Q	4217	671	365	346	502	15.9%	94.8%
25	2018_2Q	4270	690	365	346	494	16.2%	94.8%
26	2018_3Q	4423	757	365	345	473	17.1%	94.5%
27	2018_4Q	4637	825	365	352	479	17.8%	96.4%
28	2019_1Q	4757	870	367	352	486	18.3%	95.9%
29	2019_2Q	4952	889	367	353	487	18.0%	96.2%
30	2019_3Q	5123	984	367	356	483	19.2%	97.0%
31	2019_4Q	5176	1017	367	358	479	19.6%	97.5%
32	2020_1Q	5197	1036	367	356	482	19.9%	97.0%
33	2020_2Q	4878	1029	367	355	464	21.1%	96.7%
34	2020_3Q	4912	1048	367	355	461	21.3%	96.7%
35	2020_4Q	4897	1047	367	354	460	21.4%	96.5%
36	2021_1Q	4871	1045	367	348	457	21.5%	94.8%
37	2021_2Q	5507	1106	364	339	419	20.1%	93.1%
38	2021_3Q	5746	1094	363	349	424	19.0%	96.1%
39	2021_4Q	5986	1061	364	347	433	17.7%	95.3%
40	2022_1Q	6034	1019	348	336	415	16.9%	96.6%
41	2022_2Q	5892	998	351	335	404	16.9%	95.4%
42	2022_3Q	5704	990	349	338	398	17.4%	96.8%

	period_raw	rpw_quotes	wu_quotes	corridors	wu_corridors	providers	wu_quote_share	wu_corridor_presence_rate
43	2022_4Q	7608	1286	350	336	397	16.9%	96.0%
44	2023_1Q	7582	1282	351	336	389	16.9%	95.7%
45	2023_2Q	7581	1302	351	338	390	17.2%	96.3%
46	2023_3Q	7537	1284	351	339	390	17.0%	96.6%
47	2023_4Q	7588	1324	350	337	393	17.4%	96.3%
48	2024_1Q	7467	1313	349	336	390	17.6%	96.3%
49	2024_2Q	7318	1318	348	334	389	18.0%	96.0%
50	2024_3Q	6636	1324	348	337	395	20.0%	96.8%
51	2024_4Q	6655	1322	348	336	396	19.9%	96.6%
52	2025_1Q	6647	1329	348	336	391	20.0%	96.6%
53	2025_3Q	6470	1144	348	336	400	17.7%	96.6%



In [20]:
```
#@title 18. Merge RPW footprint with 2024 bilateral remittance flows
if "bilateral_flows" not in globals():
```

```
    bilateral_flows = bilateral_remittance_2024.copy()
    bilateral_flows["flow_usd"] = pd.to_numeric(bilateral_flows.get("v"), errors="coerce")

# Country-name merge first. This is intentionally conservative: unmatched rows are retained for review.
b = bilateral_flows.copy()
for c in ["source_country","receive_country"]:
    if c in b.columns:
        b[c + "_key"] = b[c].astype(str).str.strip().str.lower().str.replace(r"[^a-z0-9]+", " ", regex=True).str.strip()

m = RPW_WU_CORRIDOR_LATEST.copy()
m["send_key"] = m.send_country.astype(str).str.lower().str.replace(r"[^a-z0-9]+", " ", regex=True).str.strip()
m["receive_key"] = m.receive_country.astype(str).str.lower().str.replace(r"[^a-z0-9]+", " ", regex=True).str.strip()

if {"source_country_key","receive_country_key"}.issubset(b.columns):
    RPW_BILATERAL_2024 = b.merge(
        m,
        left_on=["source_country_key","receive_country_key"],
        right_on=["send_key","receive_key"],
        how="left",
        suffixes=("_bilateral","_rpw")
    )
else:
    RPW_BILATERAL_2024 = b.copy()
    print("Bilateral table lacks source_country / receive_country names; use ISO mapping for a stronger merge.")

if "flow_usd" in RPW_BILATERAL_2024:
    matched = RPW_BILATERAL_2024.wu_present.notna() if "wu_present" in RPW_BILATERAL_2024 else pd.Series(False, index=RPW_BILATERAL_2024.index)
    print("Bilateral corridors matched to an RPW latest-period corridor:", f"{matched.sum():,} / {len(RPW_BILATERAL_2024):,}")
    print("Share of modeled bilateral dollars covered by matched RPW corridors:", f"{RPW_BILATERAL_2024.loc[matched,'flow_usd'].sum()/RPW_BILATERAL_2024.flow_usd.sum():.1%}")
    display(RPW_BILATERAL_2024.sort_values("flow_usd", ascending=False).head(30)[[
        c for c in ["source_country","receive_country","flow_usd","wu_present","wu_quote_share","providers","wu_cost_percentile","median_cost_wu","median_cost_all"] if c in
RPW_BILATERAL_2024.columns
    ]].style.format({"flow_usd":"${:,.0f}","wu_quote_share":"{:.1%}","wu_cost_percentile":"{:.0%}"}))
```

```
Bilateral corridors matched to an RPW latest-period corridor: 241 / 8,572
Share of modeled bilateral dollars covered by matched RPW corridors: 34.9%
```

	source_country	flow_usd	wu_present	wu_quote_share	providers	wu_cost_percentile	median_cost_wu	median_cost_all
0	United States of America	$65,855,005,077	nan	nan%	nan	nan%	nan	nan
1	United States of America	$27,250,393,272	nan	nan%	nan	nan%	nan	nan
4	United Arab Emirates	$26,334,542,340	True	8.3%	12.000000	52%	2.430000	3.045000
2	United States of America	$19,680,528,363	nan	nan%	nan	nan%	nan	nan
6	Saudi Arabia	$15,216,172,432	True	8.3%	8.000000	75%	4.140000	3.555000
3	United States of America	$14,216,968,526	nan	nan%	nan	nan%	nan	nan
17	Saudi Arabia	$10,565,730,481	True	8.3%	8.000000	8%	1.440000	3.060000
9	United Kingdom	$10,443,491,625	True	10.7%	12.000000	50%	2.010000	2.060000
18	Kuwait	$9,506,522,122	True	6.7%	8.000000	68%	2.510000	2.420000
19	Saudi Arabia	$9,308,422,513	nan	nan%	nan	nan%	nan	nan
5	United States of America	$8,926,733,066	nan	nan%	nan	nan%	nan	nan
29	Malaysia	$7,883,315,434	True	5.6%	22.000000	33%	2.635000	3.400000
24	Saudi Arabia	$7,792,655,611	True	11.1%	7.000000	56%	8.290000	8.290000
7	United States of America	$7,608,901,102	nan	nan%	nan	nan%	nan	nan
8	Canada	$7,594,507,109	True	15.6%	13.000000	52%	2.580000	2.580000
10	United States of America	$7,129,152,129	nan	nan%	nan	nan%	nan	nan
11	United States of America	$7,013,237,641	nan	nan%	nan	nan%	nan	nan
12	United States of America	$6,970,751,264	nan	nan%	nan	nan%	nan	nan
35	Australia	$6,692,824,591	True	8.3%	22.000000	50%	3.040000	3.430000
36	Hong Kong	$6,635,917,075	nan	nan%	nan	nan%	nan	nan
13	United States of America	$6,574,714,680	nan	nan%	nan	nan%	nan	nan
30	Qatar	$6,551,456,432	True	28.6%	5.000000	21%	1.330000	2.980000
23	Russia	$6,168,840,055	nan	nan%	nan	nan%	nan	nan
33	India	$5,983,054,213	nan	nan%	nan	nan%	nan	nan
34	Oman	$5,977,112,744	True	27.3%	8.000000	45%	3.150000	3.150000
26	Kazakhstan	$5,722,263,345	nan	nan%	nan	nan%	nan	nan
16	United States of America	$5,708,009,646	nan	nan%	nan	nan%	nan	nan
27	Russia	$5,503,374,184	nan	nan%	nan	nan%	nan	nan
39	United Arab Emirates	$5,395,099,333	nan	nan%	nan	nan%	nan	nan
40	United Arab Emirates	$5,361,259,319	True	11.8%	11.000000	85%	4.695000	0.630000

In [21]:

```
#@title 19. Economically weighted WU footprint proxies and top corridors
if "RPW_BILATERAL_2024" not in globals() or "flow_usd" not in RPW_BILATERAL_2024.columns:
    raise NameError("Run the bilateral merge cell first.")

w = RPW_BILATERAL_2024.dropna(subset=["flow_usd"]).copy()
# These are proxies, not estimated WU principal shares. Keep the names explicit.
w["flow_x_wu_quote_share"] = w["flow_usd"] * w["wu_quote_share"].fillna(0)
w["flow_x_wu_provider_share"] = w["flow_usd"] * w["wu_provider_share"].fillna(0)
w["flow_x_wu_presence"] = w["flow_usd"] * w["wu_present"].fillna(False).astype(float)

WU_FLOW_WEIGHTED_PROXY = w.sort_values("flow_x_wu_quote_share", ascending=False).copy()
cols = [c for c in ["source_country","receive_country","flow_usd","wu_present","wu_quote_share","wu_provider_share","providers","wu_cost_percentile","flow_x_wu_quote_share"] if c in w.columns]
display(WU_FLOW_WEIGHTED_PROXY[cols].head(40).style.format({
    "flow_usd":"${:,.0f}", "wu_quote_share":"{:.1%}", "wu_provider_share":"{:.1%}", "wu_cost_percentile":"{:.0%}", "flow_x_wu_quote_share":"${:,.0f}"
}))

covered = w[w.wu_quote_share.notna()]
```

```
if len(covered):
    flow_weighted_quote_share = np.average(covered.wu_quote_share, weights=covered.flow_usd)
    flow_weighted_provider_share = np.average(covered.wu_provider_share, weights=covered.flow_usd)
    print("Flow-weighted WU RPW quote share across matched corridors:", f"{flow_weighted_quote_share:.2%}")
    print("Flow-weighted WU provider-share proxy across matched corridors:", f"{flow_weighted_provider_share:.2%}")
print("Do NOT label either number as WU transaction/principal market share; they are economically weighted RPW footprint proxies.")
```

	source_country	flow_usd	wu_present	wu_quote_share	wu_provider_share	providers	wu_cost_percentile	flow_x_wu_quote_share
4	United Arab Emirates	$26,334,542,340	True	8.3%	8.3%	12.000000	52%	$2,194,545,195
30	Qatar	$6,551,456,432	True	28.6%	20.0%	5.000000	21%	$1,871,844,695
34	Oman	$5,977,112,744	True	27.3%	12.5%	8.000000	45%	$1,630,121,657
6	Saudi Arabia	$15,216,172,432	True	8.3%	12.5%	8.000000	75%	$1,268,014,369
37	United Kingdom	$4,326,654,913	True	29.0%	11.1%	9.000000	79%	$1,256,125,620
8	Canada	$7,594,507,109	True	15.6%	7.7%	13.000000	52%	$1,186,641,736
9	United Kingdom	$10,443,491,625	True	10.7%	8.3%	12.000000	50%	$1,118,945,531
75	Germany	$2,454,010,999	True	38.9%	20.0%	5.000000	89%	$954,337,611
17	Saudi Arabia	$10,565,730,481	True	8.3%	12.5%	8.000000	8%	$880,477,540
83	Germany	$2,138,284,877	True	41.2%	14.3%	7.000000	82%	$880,470,243
24	Saudi Arabia	$7,792,655,611	True	11.1%	14.3%	7.000000	56%	$865,850,623
31	United Kingdom	$4,849,609,279	True	17.2%	12.5%	8.000000	24%	$836,139,531
49	Spain	$3,585,228,881	True	22.7%	11.1%	9.000000	77%	$814,824,746
59	Germany	$3,069,189,631	True	26.3%	14.3%	7.000000	76%	$807,681,482
68	United Arab Emirates	$3,524,606,413	True	22.2%	14.3%	7.000000	44%	$783,245,870
104	Oman	$2,305,604,044	True	33.3%	14.3%	7.000000	61%	$768,534,681
66	Canada	$1,984,574,827	True	35.7%	16.7%	6.000000	46%	$708,776,724
162	Cameroon	$1,367,768,172	True	50.0%	50.0%	2.000000	50%	$683,884,086
18	Kuwait	$9,506,522,122	True	6.7%	12.5%	8.000000	68%	$633,768,141
40	United Arab Emirates	$5,361,259,319	True	11.8%	9.1%	11.000000	85%	$630,736,390
44	France	$3,780,143,709	True	15.4%	5.9%	17.000000	65%	$581,560,571
196	Qatar	$1,145,166,409	True	50.0%	25.0%	4.000000	42%	$572,583,204
35	Australia	$6,692,824,591	True	8.3%	4.5%	22.000000	50%	$557,735,383
139	Qatar	$1,626,026,187	True	33.3%	25.0%	4.000000	92%	$542,008,729
73	Germany	$2,517,511,003	True	20.0%	12.5%	8.000000	57%	$503,502,201
98	Poland	$1,832,158,053	True	27.3%	14.3%	7.000000	52%	$499,679,469
52	Spain	$3,537,198,867	True	13.6%	10.0%	10.000000	59%	$482,345,300
81	Spain	$2,272,086,995	True	20.0%	11.1%	9.000000	67%	$454,417,399
88	France	$2,000,199,436	True	22.2%	12.5%	8.000000	81%	$444,488,764
29	Malaysia	$7,883,315,434	True	5.6%	4.5%	22.000000	33%	$437,961,969
55	Saudi Arabia	$4,314,987,161	True	10.0%	16.7%	6.000000	70%	$431,498,716
112	Japan	$2,515,067,902	True	16.7%	9.1%	11.000000	75%	$419,177,984
127	Japan	$2,072,868,419	True	20.0%	11.1%	9.000000	30%	$414,573,684
147	France	$1,133,643,442	True	35.3%	16.7%	6.000000	79%	$400,109,450
192	Saudi Arabia	$1,165,690,710	True	33.3%	33.3%	3.000000	33%	$388,563,570
80	Singapore	$3,512,575,439	True	10.7%	7.1%	14.000000	84%	$376,347,368
20	Canada	$4,598,007,234	True	8.1%	7.1%	14.000000	45%	$372,811,397
204	United Arab Emirates	$1,102,952,476	True	33.3%	20.0%	5.000000	58%	$367,650,825
155	United Arab Emirates	$1,458,853,324	True	25.0%	16.7%	6.000000	25%	$364,713,331
164	Italy	$1,032,182,258	True	35.3%	11.1%	9.000000	41%	$364,299,620

```
Flow-weighted WU RPW quote share across matched corridors: 15.74%
Flow-weighted WU provider-share proxy across matched corridors: 11.99%
Do NOT label either number as WU transaction/principal market share; they are economically weighted RPW footprint proxies.
```

In [22]:

```
#@title 20. Validation checks and known-management corridor lookups
checks = []
checks.append(("RPW rows > 0", len(r) > 0, len(r)))
checks.append(("WU identified", int(r.is_wu.sum()) > 0, int(r.is_wu.sum())))
checks.append(("Latest period rows > 0", len(LATEST) > 0, len(LATEST)))
checks.append(("Corridor shares bounded", RPW_WU_CORRIDOR_LATEST.wu_quote_share.between(0,1).all(), None))
checks.append(("Provider shares bounded", RPW_WU_CORRIDOR_LATEST.wu_provider_share.between(0,1).all(), None))
VALIDATION = pd.DataFrame(checks, columns=["check","passed","detail"])
display(VALIDATION)

# Search corridors management has discussed; this is a lookup aid, not a calibration by itself.
management_pairs = [
    ("United States","Mexico"),
    ("United States","India"),
    ("United States","Guatemala"),
    ("United States","Philippines"),
    ("United States","Dominican Republic"),
    ("United Arab Emirates","India"),
    ("Saudi Arabia","India"),
    ("United Kingdom","India"),
    ("Italy","Morocco"),
    ("France","Cameroon"),
    ("Kuwait","Bangladesh"),
]
lookup_rows=[]
for s,d in management_pairs:
    z=RPW_WU_CORRIDOR_LATEST[
        RPW_WU_CORRIDOR_LATEST.send_country.str.contains(s, case=False, regex=False, na=False) &
        RPW_WU_CORRIDOR_LATEST.receive_country.str.contains(d, case=False, regex=False, na=False)
    ]
    if len(z): lookup_rows.append(z.assign(requested_pair=f"{s} → {d}"))
MANAGEMENT_CORRIDOR_RPW = pd.concat(lookup_rows, ignore_index=True) if lookup_rows else pd.DataFrame()
if len(MANAGEMENT_CORRIDOR_RPW):
    display(MANAGEMENT_CORRIDOR_RPW[[c for c in
["requested_pair","send_country","receive_country","wu_present","wu_quote_share","providers","wu_provider_share","wu_cost_percentile","median_cost_wu","median_cost_all"] if c in
MANAGEMENT_CORRIDOR_RPW.columns]].style.format({"wu_quote_share":"{:.1%}","wu_provider_share":"{:.1%}","wu_cost_percentile":"{:.0%}"}))
else:
    print("No exact management-pair name matches. Inspect country naming conventions and add aliases if needed.")
```

	check	passed	detail
0	RPW rows > 0	True	253960.0
1	WU identified	True	44368.0
2	Latest period rows > 0	True	6470.0
3	Corridor shares bounded	True	NaN
4	Provider shares bounded	True	NaN

	requested_pair	send_country	receive_country	wu_present	wu_quote_share	providers	wu_provider_share	wu_cost_percentile	median_cost_wu	median_cost_all
0	United States → Mexico	United States	Mexico	True	13.0%	15	6.7%	79%	5.900000	4.485000
1	United States → India	United States	India	True	19.6%	13	7.7%	65%	3.830000	2.750000
2	United States → Guatemala	United States	Guatemala	True	3.3%	11	9.1%	80%	5.030000	3.975000
3	United States → Philippines	United States	Philippines	True	12.1%	14	7.1%	62%	4.460000	3.960000
4	United States → Dominican Republic	United States	Dominican Republic	True	14.8%	12	8.3%	80%	5.810000	4.100000
5	United Arab Emirates → India	United Arab Emirates	India	True	8.3%	12	8.3%	52%	2.430000	3.045000
6	Saudi Arabia → India	Saudi Arabia	India	True	8.3%	8	12.5%	75%	4.140000	3.555000
7	United Kingdom → India	United Kingdom	India	True	10.7%	12	8.3%	50%	2.010000	2.060000
8	Italy → Morocco	Italy	Morocco	True	20.7%	14	7.1%	57%	3.620000	3.390000
9	France → Cameroon	France	Cameroon	True	15.0%	6	16.7%	78%	4.440000	2.315000
10	Kuwait → Bangladesh	Kuwait	Bangladesh	True	7.1%	7	14.3%	77%	6.870000	6.530000

In [23]:
```python
#@title 21. Export analysis tables to one Excel workbook
from pathlib import Path
OUT = Path.cwd() / 'local_outputs' / 'WU_RPW_Footprint_Analysis.xlsx'
OUT.parent.mkdir(parents=True, exist_ok=True)
with pd.ExcelWriter(OUT, engine='openpyxl') as xw:
    RPW_WU_CORRIDOR_LATEST.to_excel(xw, sheet_name='corridor_latest', index=False)
    RPW_WU_BY_SENDER.to_excel(xw, sheet_name='sender_footprint', index=False)
    RPW_WU_BY_RECEIVER.to_excel(xw, sheet_name='receiver_footprint', index=False)
    RPW_WU_CHANNEL_MIX.to_excel(xw, sheet_name='channel_mix', index=False)
    RPW_WU_TREND.to_excel(xw, sheet_name='historical_trend', index=False)
    if 'RPW_BILATERAL_2024' in globals():
        RPW_BILATERAL_2024.to_excel(xw, sheet_name='bilateral_2024_merge', index=False)
    if 'WU_FLOW_WEIGHTED_PROXY' in globals():
        WU_FLOW_WEIGHTED_PROXY.to_excel(xw, sheet_name='flow_weighted_proxy', index=False)
    VALIDATION.to_excel(xw, sheet_name='validation', index=False)
print('Saved:', OUT)
```

```
Saved: C:\Users\micha\OneDrive\Documents\ChatGPT\alphasensecheck\python_analysis\outputs\colab\local_outputs\WU_RPW_Footprint_Analysis.xlsx
```

Notes

- Pair direction matters: `US→MX` and `MX→US` are separate records.
- `top10.modeled_revenue_share` is a modeled allocation because Western Union does not disclose corridor revenue.
- The embedded US→MX dossier and raw quote are available as `US_MX_pair_dossier` and `US_MX_raw_quote`.
- Change only the two codes in `select_pair()` to inspect another corridor.

Extended provider concentration, pricing, and writeup analysis

This section turns RPW + 2024 bilateral flows into **comparative provider exposure proxies**, not reported revenues. It is designed to support Questions 2, 3, 5, 8 and 10 directly, with secondary evidence for Questions 6, 12 and 13.

Three deliberately separate concepts are reported:

1. **RPW footprint share** — provider presence / quote share in observed corridors.
2. **Flow-weighted principal proxy** — bilateral remittance flow × provider footprint share.
3. **Revenue-intensity proxy** — flow-weighted principal proxy × observed RPW total cost (%). This is a rough within-provider allocation proxy, not accounting revenue.

Use the revenue-intensity proxy primarily for **concentration within a provider** (top-10 corridors, top countries, HHI), not for comparing absolute revenue dollars across providers.

In [24]:
```
#@title 22. Build all-provider latest-period corridor panel
from math import log

# Provider aliases: preserve raw provider but consolidate obvious branding variants.
def canonical_provider(x):
    s = str(x).strip()
    n = re.sub(r"[^a-z0-9]+", " ", s.lower()).strip()
    rules = [
        (r"western\s*union|westernunion", "Western Union"),
        (r"moneygram", "MoneyGram"),
        (r"remitly", "Remitly"),
        (r"wise|transferwise", "Wise"),
        (r"ria\b|ria money", "Ria"),
        (r"worldremit", "WorldRemit"),
        (r"xoom", "Xoom"),
        (r"small world", "Small World"),
        (r"taptap send|taptapsend", "Taptap Send"),
        (r"sendwave", "Sendwave"),
        (r"revolut", "Revolut"),
        (r"xe money|xe\.com|\bxe\b", "XE"),
    ]
    for pat, label in rules:
        if re.search(pat, n): return label
    return re.sub(r"\s+", " ", s).strip()

LATEST_ALL = LATEST.copy()
LATEST_ALL["provider_canon"] = LATEST_ALL["provider"].map(canonical_provider)
LATEST_ALL["corridor"] = LATEST_ALL["send_country"] + " → " + LATEST_ALL["receive_country"]

# Provider-corridor quote metrics.
pc = (LATEST_ALL.groupby(["provider_canon","send_country","receive_country","corridor"], dropna=False)
      .agg(quotes=("provider_canon","size"),
           products=("product","nunique"),
           median_cost=("cost","median"),
           mean_cost=("cost","mean"),
           min_cost=("cost","min"),
           median_fee=("fee","median"),
           median_fx_margin=("fx_margin","median"))
      .reset_index())

# Corridor denominators.
cd = (LATEST_ALL.groupby(["send_country","receive_country","corridor"], dropna=False)
      .agg(total_quotes=("provider_canon","size"), providers=("provider_canon","nunique"),
           corridor_median_cost=("cost","median"), corridor_min_cost=("cost","min"))
```

```
      .reset_index())

PROVIDER_CORRIDOR_LATEST = pc.merge(cd, on=["send_country","receive_country","corridor"], how="left")
PROVIDER_CORRIDOR_LATEST["quote_share"] = PROVIDER_CORRIDOR_LATEST["quotes"] / PROVIDER_CORRIDOR_LATEST["total_quotes"]
PROVIDER_CORRIDOR_LATEST["equal_provider_share"] = 1 / PROVIDER_CORRIDOR_LATEST["providers"].replace(0,np.nan)
PROVIDER_CORRIDOR_LATEST["cost_vs_corridor_median_pp"] = PROVIDER_CORRIDOR_LATEST["median_cost"] - PROVIDER_CORRIDOR_LATEST["corridor_median_cost"]
PROVIDER_CORRIDOR_LATEST["cost_premium_pct"] = PROVIDER_CORRIDOR_LATEST["median_cost"] / PROVIDER_CORRIDOR_LATEST["corridor_median_cost"] - 1

print(f"Latest period: {LATEST_RPW_PERIOD}")
print(f"Providers: {PROVIDER_CORRIDOR_LATEST.provider_canon.nunique():,}; corridors: {PROVIDER_CORRIDOR_LATEST.corridor.nunique():,}")
display(PROVIDER_CORRIDOR_LATEST.sort_values(["provider_canon","corridor"]).head(30))
```

```
Latest period: 2025_3Q
Providers: 381; corridors: 348
```

	provider_canon	send_country	receive_country	corridor	quotes	products	median_cost	mean_cost	min_cost	median_fee	median_fx_margin	total_quotes	providers	corridor_median_cost	corridor_min_cost	quote_share	equal_provider_share	cost_vs_corridor_median_pp	cost_premium_pct
0	A-Express Remit	Singapore	Indonesia	Singapore → Indonesia	1	0	2.550	2.550	2.55	6.00	0.240	31	19	2.620	0.56	0.032258	0.052632	-0.070	-0.026718
1	A-Express Remit	Singapore	Philippines	Singapore → Philippines	2	0	2.225	2.225	2.13	4.75	0.400	38	19	2.315	0.76	0.052632	0.052632	-0.090	-0.038877
2	ABC Bank	Kenya	South Sudan	Kenya → South Sudan	3	0	11.810	11.810	11.81	1800.00	1.810	24	10	15.140	2.46	0.125000	0.100000	-3.330	-0.219947
3	ABC Bank	Kenya	Tanzania	Kenya → Tanzania	3	0	11.810	11.810	11.81	1800.00	1.810	26	11	13.475	0.46	0.115385	0.090909	-1.665	-0.123562
4	ABC Bank	Kenya	Uganda	Kenya → Uganda	3	0	11.810	11.810	11.81	1800.00	1.810	27	12	11.810	2.85	0.111111	0.083333	0.000	0.000000
5	ABN AMRO Bank	Netherlands	Morocco	Netherlands → Morocco	1	0	25.180	25.180	25.18	35.00	0.180	25	9	4.210	0.98	0.040000	0.111111	20.970	4.980998
6	ABN AMRO Bank	Netherlands	Suriname	Netherlands → Suriname	2	0	19.285	19.285	13.57	27.00	0.000	15	6	5.680	3.84	0.133333	0.166667	13.605	2.395246
7	ABN AMRO Bank	Netherlands	Türkiye	Netherlands → Türkiye	2	0	32.220	32.220	32.22	45.00	0.080	23	9	5.000	2.50	0.086957	0.111111	27.220	5.444000
8	ABSA	South Africa	Angola	South Africa → Angola	2	0	28.050	28.050	22.21	370.00	1.040	13	6	22.210	6.12	0.153846	0.166667	5.840	0.262945
9	ABSA	South Africa	Botswana	South Africa → Botswana	2	0	31.115	31.115	25.00	365.00	4.470	27	8	10.290	5.57	0.074074	0.125000	20.825	2.023810
10	ABSA	South Africa	China	South Africa → China	1	0	37.690	37.690	37.69	450.00	4.840	11	6	31.950	3.76	0.090909	0.166667	5.740	0.179656
11	ABSA	South Africa	Eswatini	South Africa → Eswatini	2	0	5.290	5.290	0.00	72.50	0.000	9	6	10.950	0.00	0.222222	0.166667	-5.660	-0.516895
12	ABSA	South Africa	Kenya	South Africa → Kenya	2	0	28.850	28.850	22.55	365.00	2.205	33	9	7.050	3.44	0.060606	0.111111	21.800	3.092199
13	ABSA	South Africa	Lesotho	South Africa → Lesotho	2	0	5.290	5.290	0.00	72.50	0.000	11	7	10.580	0.00	0.181818	0.142857	-5.290	-0.500000
14	ABSA	South Africa	Nigeria	South Africa → Nigeria	1	0	34.600	34.600	34.60	450.00	1.750	17	7	8.260	3.46	0.058824	0.142857	26.340	3.188862
15	ABSA	South Africa	Tanzania	South Africa → Tanzania	2	0	28.050	28.050	22.21	370.00	1.040	16	6	13.740	6.13	0.125000	0.166667	14.310	1.041485
16	ABSA	South Africa	Zambia	South Africa → Zambia	2	0	28.800	28.800	22.51	365.00	2.155	34	10	11.240	4.87	0.058824	0.100000	17.560	1.562278
17	ABSA	South Africa	Zimbabwe	South Africa → Zimbabwe	2	0	28.050	28.050	22.21	370.00	1.040	40	12	10.280	5.15	0.050000	0.083333	17.770	1.728599
18	ADCB	United Arab Emirates	India	United Arab Emirates → India	1	0	21.060	21.060	21.06	126.00	3.920	24	12	3.045	0.77	0.041667	0.083333	18.015	5.916256
19	ANZ Bank	Australia	Fiji	Australia → Fiji	2	0	4.440	4.440	2.69	3.50	2.690	26	11	3.690	-3.17	0.076923	0.090909	0.750	0.203252
20	ANZ Bank	Australia	India	Australia → India	2	0	8.305	8.305	8.19	9.00	3.805	36	22	3.430	-2.22	0.055556	0.045455	4.875	1.421283
21	ANZ Bank	Australia	Indonesia	Australia → Indonesia	1	0	7.840	7.840	7.84	9.00	3.340	19	12	2.870	0.70	0.052632	0.083333	4.970	1.731707
22	ANZ Bank	Australia	Lebanon	Australia → Lebanon	2	0	7.840	7.840	7.84	9.00	3.340	33	15	7.840	2.39	0.060606	0.066667	0.000	0.000000
23	ANZ Bank	Australia	Malaysia	Australia → Malaysia	2	0	7.840	7.840	7.84	9.00	3.340	20	9	3.055	0.80	0.100000	0.111111	4.785	1.566285

	provider_canon	send_country	receive_country	corridor	quotes	products	median_cost	mean_cost	min_cost	median_fee	median_fx_margin	total_quotes	providers	corridor_median_cost	corridor_min_cost	quote_share	equal_provider_share	cost_vs_corridor_median_pp	cost_premium_pct
24	ANZ Bank	Australia	Samoa	Australia → Samoa	2	0	6.630	6.630	4.95	3.50	4.880	15	8	5.880	-0.54	0.133333	0.125000	0.750	0.127551
25	ANZ Bank	Australia	Sri Lanka	Australia → Sri Lanka	1	0	8.030	8.030	8.03	9.00	3.530	23	11	3.040	1.00	0.043478	0.090909	4.990	1.641447
26	ANZ Bank	Australia	Thailand	Australia → Thailand	2	0	8.095	8.095	7.90	9.00	3.595	19	9	3.680	1.52	0.105263	0.111111	4.415	1.199728
27	ANZ Bank	Australia	Tonga	Australia → Tonga	2	0	5.100	5.100	3.14	3.50	3.350	16	9	6.615	0.92	0.125000	0.111111	-1.515	-0.229025
28	ANZ Bank	Australia	Vanuatu	Australia → Vanuatu	2	0	7.335	7.335	5.72	3.50	5.585	14	7	8.630	5.72	0.142857	0.142857	-1.295	-0.150058
29	ANZ Bank	Australia	Vietnam	Australia → Vietnam	2	0	7.840	7.840	7.84	9.00	3.340	18	11	3.955	0.17	0.111111	0.090909	3.885	0.982301

In [25]:

```
#@title 23. Merge every provider to 2024 bilateral flows
# Reuse conservative country-name normalization from the prior merge.
def country_key(x):
    return re.sub(r"[^a-z0-9]+", " ", str(x).lower()).strip()

if "bilateral_flows" not in globals():
    bilateral_flows = bilateral_remittance_2024.copy()
    bilateral_flows["flow_usd"] = pd.to_numeric(bilateral_flows.get("v"), errors="coerce")

bf = bilateral_flows.copy()
if "flow_usd" not in bf.columns:
    bf["flow_usd"] = pd.to_numeric(bf.get("v"), errors="coerce")

src_col = next((c for c in ["source_country","send_country","origin_country"] if c in bf.columns), None)
rec_col = next((c for c in ["receive_country","destination_country","recipient_country"] if c in bf.columns), None)
if not src_col or not rec_col:
    raise KeyError(f"Could not identify bilateral source/receive country columns: {bf.columns.tolist()}")

bf["send_key"] = bf[src_col].map(country_key)
bf["receive_key"] = bf[rec_col].map(country_key)
pp = PROVIDER_CORRIDOR_LATEST.copy()
pp["send_key"] = pp.send_country.map(country_key)
pp["receive_key"] = pp.receive_country.map(country_key)

PROVIDER_FLOW_2024 = pp.merge(
    bf[["send_key","receive_key","flow_usd"]].drop_duplicates(["send_key","receive_key"]),
    on=["send_key","receive_key"], how="left"
)
PROVIDER_FLOW_2024["principal_proxy_quote"] = PROVIDER_FLOW_2024["flow_usd"] * PROVIDER_FLOW_2024["quote_share"]
PROVIDER_FLOW_2024["principal_proxy_equal_provider"] = PROVIDER_FLOW_2024["flow_usd"] * PROVIDER_FLOW_2024["equal_provider_share"]
# RPW total cost is percent; divide by 100. Revenue-intensity proxy is deliberately not called revenue.
PROVIDER_FLOW_2024["revenue_intensity_proxy"] = PROVIDER_FLOW_2024["principal_proxy_quote"] * PROVIDER_FLOW_2024["median_cost"] / 100

coverage = PROVIDER_FLOW_2024.groupby("provider_canon").agg(
    corridors_rpw=("corridor","nunique"),
    corridors_with_flow=("flow_usd",lambda x:x.notna().sum()),
    bilateral_flow_covered=("flow_usd","sum")
).reset_index()
coverage["flow_merge_rate"] = coverage.corridors_with_flow / coverage.corridors_rpw
PROVIDER_FLOW_COVERAGE = coverage.sort_values("bilateral_flow_covered",ascending=False)
display(PROVIDER_FLOW_COVERAGE.head(30).style.format({"bilateral_flow_covered":"${:,.0f}","flow_merge_rate":"{:.1%}"}))
```

	provider_canon	corridors_rpw	corridors_with_flow	bilateral_flow_covered	flow_merge_rate
368	Western Union	336	233	$306,617,501,430	69.3%
236	MoneyGram	306	211	$288,174,527,146	69.0%
287	Remitly	159	105	$154,519,997,406	66.0%
289	Ria	197	130	$146,285,306,566	66.0%
370	Wise	127	88	$118,203,098,964	69.3%
372	WorldRemit	108	81	$104,494,225,171	75.0%
26	Al-Rajhi Bank	8	7	$45,590,055,852	87.5%
298	STC Pay	15	13	$43,887,317,912	86.7%
18	Al Fardan Exchange	11	9	$41,604,960,063	81.8%
216	Lari	11	9	$41,604,960,063	81.8%
16	Al Ansari	10	8	$41,545,914,593	80.0%
336	TeleMoney	8	7	$41,462,918,164	87.5%
61	Bank Albilad	7	6	$40,659,451,768	85.7%
147	GCC Exchange	6	6	$38,984,108,793	100.0%
373	Xoom	58	23	$38,975,941,253	39.7%
364	Wall St Exchange	7	5	$35,459,502,380	71.4%
117	DirectRemit (NBD)	5	4	$35,332,734,484	80.0%
303	Saudi British Bank	4	4	$34,974,807,326	100.0%
141	Fawri	9	7	$34,031,936,811	77.8%
122	Dubai Islamic Bank	2	2	$31,695,801,659	100.0%
313	Skrill	17	14	$31,691,754,182	82.4%
325	State Bank of India	10	8	$26,818,527,703	80.0%
130	Emirates NBD	1	1	$26,334,542,340	100.0%
4	ADCB	1	1	$26,334,542,340	100.0%
267	Paysend	29	20	$26,262,542,365	69.0%
9	Ace Money Transfer	19	14	$24,191,960,595	73.7%
168	ICICI Bank	3	3	$20,566,194,933	100.0%
179	InstaReM	14	10	$18,506,166,690	71.4%
304	ScotiaBank	8	8	$17,553,939,894	100.0%
257	Orbit Remit	13	13	$15,534,900,331	100.0%

In [26]:
```
#@title 24. Provider concentration statistics: top corridors, HHI, Gini, effective N

def gini_nonneg(x):
    a=np.asarray(pd.Series(x).dropna(),dtype=float)
    a=a[a>=0]
    if len(a)==0 or a.sum()==0:return np.nan
    a=np.sort(a); n=len(a)
    return (2*np.dot(np.arange(1,n+1),a)/(n*a.sum())) - (n+1)/n

def conc_stats(g, value_col):
    x=g[["corridor",value_col]].dropna().groupby("corridor")[value_col].sum().sort_values(ascending=False)
    if x.sum()<=0:return pd.Series(dtype=float)
    s=x/x.sum()
    hhi=(s.pow(2).sum())
    entropy=-(s[s>0]*np.log(s[s>0])).sum()
    return pd.Series({
        "corridors":len(x), "top1_share":s.iloc[:1].sum(), "top5_share":s.iloc[:5].sum(),
```

```
        "top10_share":s.iloc[:10].sum(), "top20_share":s.iloc[:20].sum(),
        "hhi_0_1":hhi, "hhi_10000":hhi*10000, "effective_corridors_1_hhi":1/hhi if hhi else np.nan,
        "gini":gini_nonneg(x.values), "entropy":entropy, "effective_corridors_entropy":np.exp(entropy)
    })

eligible = PROVIDER_FLOW_2024.groupby("provider_canon").filter(lambda g:g.flow_usd.notna().sum()>=5)
frames=[]
for metric in ["principal_proxy_quote","principal_proxy_equal_provider","revenue_intensity_proxy"]:
    z=eligible.groupby("provider_canon").apply(lambda g:conc_stats(g,metric), include_groups=False).reset_index()
    z["metric"] = metric
    frames.append(z)
PROVIDER_CONCENTRATION = pd.concat(frames,ignore_index=True)

# Writeup-friendly primary view.
PROVIDER_CONCENTRATION_PRIMARY = PROVIDER_CONCENTRATION.query("metric == 'revenue_intensity_proxy'").sort_values("top10_share",ascending=False)
display(PROVIDER_CONCENTRATION_PRIMARY.head(25).style.format({
    "top1_share":"{:.1%}","top5_share":"{:.1%}","top10_share":"{:.1%}","top20_share":"{:.1%}",
    "hhi_0_1":"{:.3f}","hhi_10000":"{:,.0f}","effective_corridors_1_hhi":"{:.1f}","gini":"{:.2f}",
    "entropy":"{:.2f}","effective_corridors_entropy":"{:.1f}"
}))
```

	provider_canon	corridors	top1_share	top5_share	top10_share	top20_share	hhi_0_1	hhi_10000	effective_corridors_1_hhi	gini	entropy	effective_corridors_entropy	metric
171	Al Ansari	8.000000	58.6%	100.0%	100.0%	100.0%	0.397	3,969	2.5	0.60	1.28	3.6	revenue_intensity_proxy
172	Al Dar Exchange	7.000000	44.1%	96.2%	100.0%	100.0%	0.288	2,880	3.5	0.44	1.48	4.4	revenue_intensity_proxy
210	Kyodai	6.000000	28.2%	94.1%	100.0%	100.0%	0.200	2,002	5.0	0.24	1.69	5.4	revenue_intensity_proxy
190	Commerzbank AG	6.000000	31.7%	97.6%	100.0%	100.0%	0.251	2,511	4.0	0.39	1.51	4.5	revenue_intensity_proxy
232	Sendwave	5.000000	83.4%	100.0%	100.0%	100.0%	0.712	7,118	1.4	0.59	0.66	1.9	revenue_intensity_proxy
208	KlickEx Pacific	6.000000	46.0%	107.2%	100.0%	100.0%	0.381	3,808	2.6	0.43	1.28	3.6	revenue_intensity_proxy
242	TransferGo	11.000000	54.8%	88.8%	100.0%	100.0%	0.336	3,361	3.0	0.65	1.55	4.7	revenue_intensity_proxy
228	Royal Bank of Canada	9.000000	42.4%	93.7%	100.0%	100.0%	0.268	2,684	3.7	0.59	1.58	4.8	revenue_intensity_proxy
185	Bank of Baroda	5.000000	79.7%	100.0%	100.0%	100.0%	0.649	6,490	1.5	0.65	0.74	2.1	revenue_intensity_proxy
184	Bank Albilad	6.000000	32.4%	97.1%	100.0%	100.0%	0.250	2,500	4.0	0.39	1.51	4.5	revenue_intensity_proxy
181	Banca Intesa SanPaolo	8.000000	25.2%	80.0%	100.0%	100.0%	0.162	1,621	6.2	0.29	1.94	7.0	revenue_intensity_proxy
212	Mama Money	8.000000	66.9%	97.8%	100.0%	100.0%	0.479	4,793	2.1	0.63	1.17	3.2	revenue_intensity_proxy
189	Citibank	5.000000	63.0%	100.0%	100.0%	100.0%	0.450	4,500	2.2	0.54	1.07	2.9	revenue_intensity_proxy
200	Fawri	7.000000	45.5%	96.0%	100.0%	100.0%	0.304	3,042	3.3	0.54	1.44	4.2	revenue_intensity_proxy
187	Bank of New Zealand	7.000000	42.9%	93.1%	100.0%	100.0%	0.261	2,612	3.8	0.46	1.58	4.9	revenue_intensity_proxy
175	Al Muzaini Exchange	5.000000	58.7%	100.0%	100.0%	100.0%	0.400	3,998	2.5	0.49	1.19	3.3	revenue_intensity_proxy
176	Al-Rajhi Bank	7.000000	33.8%	96.2%	100.0%	100.0%	0.262	2,620	3.8	0.49	1.49	4.5	revenue_intensity_proxy
179	Aussie Forex & Finance	5.000000	31.1%	100.0%	100.0%	100.0%	0.233	2,327	4.3	0.21	1.53	4.6	revenue_intensity_proxy
173	Al Fardan Exchange	9.000000	75.8%	96.5%	100.0%	100.0%	0.588	5,876	1.7	0.75	0.96	2.6	revenue_intensity_proxy
174	Al Mulla Exchange	5.000000	67.4%	100.0%	100.0%	100.0%	0.557	5,570	1.8	0.46	1.00	2.7	revenue_intensity_proxy
219	Nedbank	8.000000	60.1%	96.7%	100.0%	100.0%	0.443	4,427	2.3	0.70	1.10	3.0	revenue_intensity_proxy
233	Seven Bank	7.000000	52.0%	91.8%	100.0%	100.0%	0.314	3,145	3.2	0.47	1.51	4.5	revenue_intensity_proxy
240	Toronto Dominion Bank	5.000000	62.4%	100.0%	100.0%	100.0%	0.497	4,968	2.0	0.62	0.85	2.3	revenue_intensity_proxy
239	TeleMoney	7.000000	35.9%	96.1%	100.0%	100.0%	0.254	2,541	3.9	0.48	1.54	4.7	revenue_intensity_proxy
246	Wall St Exchange	5.000000	86.5%	100.0%	100.0%	100.0%	0.762	7,618	1.3	0.67	0.48	1.6	revenue_intensity_proxy

In [27]:
```
#@title 25. Top-10 corridor table for Western Union + major competitors
# Major competitors = providers with broad RPW coverage; always include WU.
provider_scale = (PROVIDER_FLOW_2024.groupby("provider_canon")
```

```
                  .agg(corridors=("corridor","nunique"), flow_covered=("flow_usd","sum"), quotes=("quotes","sum"))
                  .sort_values(["corridors","flow_covered"],ascending=False))
majors = provider_scale.head(12).index.tolist()
if "Western Union" not in majors: majors=["Western Union"]+majors[:11]
print("Providers shown:", majors)

def top_corridors(provider, metric="revenue_intensity_proxy", n=10):
    z=PROVIDER_FLOW_2024[(PROVIDER_FLOW_2024.provider_canon==provider)&PROVIDER_FLOW_2024[metric].notna()].copy()
    z=z.sort_values(metric,ascending=False)
    total=z[metric].sum()
    z["within_provider_share"] = z[metric]/total if total else np.nan
    z["cum_share"] = z.within_provider_share.cumsum()
    return z[["provider_canon","corridor","flow_usd","quote_share","providers","median_cost","cost_vs_corridor_median_pp",metric,"within_provider_share","cum_share"]].head(n)

TOP10_PROVIDER_CORRIDORS = pd.concat([top_corridors(p) for p in majors],ignore_index=True)
display(TOP10_PROVIDER_CORRIDORS.style.format({
    "flow_usd":"${:,.0f}","quote_share":"{:.1%}","median_cost":"{:.2f}%","cost_vs_corridor_median_pp":"{:+.2f} pp",
    "revenue_intensity_proxy":"${:,.0f}","within_provider_share":"{:.1%}","cum_share":"{:.1%}"
}))
```

```
Providers shown: ['Western Union', 'MoneyGram', 'Ria', 'Remitly', 'Wise', 'WorldRemit', 'Xoom', 'Walmart2World', 'Paysend', 'Ace Money Transfer', 'Dahabshiil', 'ANZ Bank']
```

	provider_canon	corridor	flow_usd	quote_share	providers	median_cost	cost_vs_corridor_median_pp	revenue_intensity_proxy	within_provider_share	cum_share
0	Western Union	Germany → Hungary	$2,454,010,999	38.9%	5	13.30%	+7.23 pp	$126,926,902	5.7%	5.7%
1	Western Union	South Africa → Zimbabwe	$2,259,410,507	10.0%	12	31.81%	+21.53 pp	$71,871,848	3.2%	8.9%
2	Western Union	Saudi Arabia → Bangladesh	$7,792,655,611	11.1%	7	8.29%	+0.00 pp	$71,779,017	3.2%	12.2%
3	Western Union	Oman → Bangladesh	$2,305,604,044	33.3%	7	8.22%	+0.00 pp	$63,173,551	2.8%	15.0%
4	Western Union	United Arab Emirates → India	$26,334,542,340	8.3%	12	2.43%	-0.61 pp	$53,327,448	2.4%	17.4%
5	Western Union	United Arab Emirates → Bangladesh	$3,524,606,413	22.2%	7	6.79%	-0.37 pp	$53,143,232	2.4%	19.8%
6	Western Union	Saudi Arabia → India	$15,216,172,432	8.3%	8	4.14%	+0.58 pp	$52,495,795	2.4%	22.2%
7	Western Union	Oman → India	$5,977,112,744	27.3%	8	3.15%	+0.00 pp	$51,348,832	2.3%	24.5%
8	Western Union	United Kingdom → Nigeria	$4,326,654,913	29.0%	8	3.84%	+2.83 pp	$48,235,224	2.2%	26.6%
9	Western Union	Israel → Morocco	$723,622,134	33.3%	3	16.98%	+3.51 pp	$40,957,013	1.8%	28.5%
10	MoneyGram	Saudi Arabia → Bangladesh	$7,792,655,611	11.1%	7	8.66%	+0.37 pp	$74,982,664	6.4%	6.4%
11	MoneyGram	Saudi Arabia → India	$15,216,172,432	8.3%	8	3.57%	+0.02 pp	$45,268,113	3.8%	10.2%
12	MoneyGram	Oman → India	$5,977,112,744	18.2%	8	3.82%	+0.67 pp	$41,513,765	3.5%	13.7%
13	MoneyGram	Saudi Arabia → Pakistan	$10,565,730,481	8.3%	8	4.24%	+1.18 pp	$37,332,248	3.2%	16.9%
14	MoneyGram	Saudi Arabia → Ethiopia	$1,165,690,710	33.3%	3	8.66%	+0.00 pp	$33,649,605	2.9%	19.7%
15	MoneyGram	Cameroon → Nigeria	$1,367,768,172	50.0%	2	4.63%	+0.25 pp	$31,663,833	2.7%	22.4%
16	MoneyGram	United Arab Emirates → India	$26,334,542,340	4.2%	12	2.48%	-0.56 pp	$27,212,360	2.3%	24.7%
17	MoneyGram	South Africa → Zimbabwe	$2,259,410,507	7.5%	12	15.43%	+5.15 pp	$26,147,028	2.2%	26.9%
18	MoneyGram	United Arab Emirates → Bangladesh	$3,524,606,413	11.1%	7	6.23%	-0.92 pp	$24,398,109	2.1%	29.0%
19	MoneyGram	Dominican Republic → Haiti	$820,590,987	33.3%	3	8.42%	+0.51 pp	$23,031,254	2.0%	30.9%
20	Ria	Germany → Hungary	$2,454,010,999	22.2%	5	6.07%	+0.00 pp	$33,101,882	6.2%	6.2%
21	Ria	Germany → Croatia	$2,517,511,003	15.0%	7	6.07%	+2.51 pp	$22,921,938	4.3%	10.5%
22	Ria	Germany → Ukraine	$3,069,189,631	26.3%	6	2.50%	-0.29 pp	$20,192,037	3.8%	14.3%
23	Ria	Malaysia → Indonesia	$7,883,315,434	8.3%	22	3.06%	-0.34 pp	$20,102,454	3.8%	18.1%
24	Ria	United Kingdom → India	$10,443,491,625	7.1%	12	2.51%	+0.45 pp	$18,723,689	3.5%	21.7%
25	Ria	Germany → Romania	$2,251,610,730	19.6%	13	3.61%	+0.82 pp	$15,937,872	3.0%	24.7%
26	Ria	Canada → India	$7,594,507,109	6.2%	13	3.01%	+0.43 pp	$14,287,166	2.7%	27.3%
27	Ria	Spain → Morocco	$3,585,228,881	9.1%	9	3.67%	-0.13 pp	$11,945,331	2.2%	29.6%
28	Ria	Malaysia → Philippines	$1,362,275,811	18.9%	20	4.51%	+1.76 pp	$11,623,526	2.2%	31.8%
29	Ria	Germany → Bosnia and Herzegovina	$752,321,950	31.8%	5	4.35%	+0.00 pp	$10,412,820	2.0%	33.7%
30	Remitly	United Arab Emirates → India	$26,334,542,340	8.3%	12	2.04%	-1.00 pp	$44,768,722	6.4%	6.4%
31	Remitly	United Kingdom → Pakistan	$4,849,609,279	20.7%	8	3.20%	+0.69 pp	$32,107,758	4.6%	11.0%
32	Remitly	Spain → Colombia	$3,537,198,867	18.2%	10	4.93%	+0.31 pp	$31,706,164	4.5%	15.5%
33	Remitly	Germany → Romania	$2,251,610,730	23.5%	13	3.58%	+0.79 pp	$18,940,020	2.7%	18.2%
34	Remitly	France → Tunisia	$2,000,199,436	22.2%	7	4.25%	+0.17 pp	$18,890,772	2.7%	20.9%
35	Remitly	Spain → Morocco	$3,585,228,881	18.2%	9	2.69%	-1.11 pp	$17,502,436	2.5%	23.4%
36	Remitly	Singapore → India	$3,512,575,439	17.9%	14	2.77%	+0.06 pp	$17,374,704	2.5%	25.9%
37	Remitly	Canada → Philippines	$4,598,007,234	8.1%	14	4.61%	+1.29 pp	$17,186,605	2.5%	28.4%
38	Remitly	Canada → India	$7,594,507,109	6.2%	13	3.49%	+0.91 pp	$16,565,519	2.4%	30.8%
39	Remitly	Australia → China	$1,707,738,154	27.8%	10	3.47%	+0.17 pp	$16,460,698	2.4%	33.1%
40	Wise	United Arab Emirates → Bangladesh	$3,524,606,413	22.2%	7	10.71%	+3.55 pp	$83,846,470	22.3%	22.3%
41	Wise	United Arab Emirates → Pakistan	$5,361,259,319	11.8%	11	5.30%	+4.67 pp	$33,460,566	8.9%	31.2%
42	Wise	France → Morocco	$3,780,143,709	12.8%	14	4.92%	+1.32 pp	$23,843,983	6.3%	37.5%

	provider_canon	corridor	flow_usd	quote_share	providers	median_cost	cost_vs_corridor_median_pp	revenue_intensity_proxy	within_provider_share	cum_share
43	Wise	United Kingdom → India	$10,443,491,625	10.7%	12	1.72%	-0.34 pp	$19,245,863	5.1%	42.6%
44	Wise	United Arab Emirates → Indonesia	$1,458,853,324	25.0%	6	5.05%	+0.14 pp	$18,436,259	4.9%	47.5%
45	Wise	United Arab Emirates → Philippines	$3,130,443,828	9.5%	9	5.03%	+1.60 pp	$14,981,410	4.0%	51.5%
46	Wise	Germany → Ukraine	$3,069,189,631	10.5%	6	3.77%	+0.98 pp	$12,179,837	3.2%	54.8%
47	Wise	United Kingdom → Pakistan	$4,849,609,279	10.3%	8	2.19%	-0.32 pp	$10,986,873	2.9%	57.7%
48	Wise	Italy → Morocco	$1,551,494,958	10.3%	11	5.70%	+2.31 pp	$9,148,470	2.4%	60.1%
49	Wise	Poland → Ukraine	$1,832,158,053	13.6%	7	3.39%	+0.38 pp	$8,469,567	2.3%	62.4%
50	WorldRemit	Spain → Colombia	$3,537,198,867	27.3%	10	4.24%	-0.38 pp	$40,902,881	10.3%	10.3%
51	WorldRemit	United Kingdom → Pakistan	$4,849,609,279	20.7%	8	2.96%	+0.45 pp	$29,699,676	7.5%	17.9%
52	WorldRemit	Canada → Pakistan	$1,380,198,956	31.6%	7	6.10%	+1.74 pp	$26,586,990	6.7%	24.6%
53	WorldRemit	United Kingdom → India	$10,443,491,625	10.7%	12	2.29%	+0.23 pp	$25,623,853	6.5%	31.1%
54	WorldRemit	South Africa → Zimbabwe	$2,259,410,507	15.0%	12	5.97%	-4.30 pp	$20,249,967	5.1%	36.2%
55	WorldRemit	Spain → Morocco	$3,585,228,881	13.6%	9	3.88%	+0.08 pp	$18,969,120	4.8%	41.0%
56	WorldRemit	Spain → Dominican Republic	$1,117,647,335	23.1%	7	6.49%	+1.40 pp	$16,738,918	4.2%	45.2%
57	WorldRemit	Canada → Philippines	$4,598,007,234	16.2%	14	1.97%	-1.35 pp	$14,688,769	3.7%	48.9%
58	WorldRemit	Malaysia → Nepal	$1,993,511,584	14.3%	9	4.75%	+2.44 pp	$13,527,400	3.4%	52.3%
59	WorldRemit	Canada → India	$7,594,507,109	9.4%	13	1.74%	-0.84 pp	$12,388,540	3.1%	55.5%
60	Xoom	United Kingdom → India	$10,443,491,625	10.7%	12	3.07%	+1.01 pp	$34,351,628	18.9%	18.9%
61	Xoom	Canada → India	$7,594,507,109	9.4%	13	3.88%	+1.30 pp	$27,625,020	15.2%	34.2%
62	Xoom	France → Morocco	$3,780,143,709	10.3%	14	4.59%	+0.99 pp	$17,795,753	9.8%	44.0%
63	Xoom	Canada → Sri Lanka	$590,352,343	33.3%	5	6.10%	+0.68 pp	$12,003,831	6.6%	50.6%
64	Xoom	Canada → China	$1,984,574,827	21.4%	6	2.50%	-3.08 pp	$10,631,651	5.9%	56.5%
65	Xoom	Canada → Pakistan	$1,380,198,956	15.8%	7	4.36%	+0.00 pp	$9,501,580	5.2%	61.7%
66	Xoom	Canada → Vietnam	$774,872,054	15.8%	10	7.67%	+3.52 pp	$9,384,108	5.2%	66.9%
67	Xoom	United Kingdom → Bangladesh	$1,261,507,171	9.4%	10	7.38%	+2.31 pp	$8,728,053	4.8%	71.7%
68	Xoom	Italy → Romania	$1,665,973,101	9.7%	10	4.34%	+1.44 pp	$6,997,087	3.9%	75.6%
69	Xoom	Canada → Philippines	$4,598,007,234	8.1%	14	1.83%	-1.49 pp	$6,822,449	3.8%	79.3%
70	Paysend	Germany → Ukraine	$3,069,189,631	10.5%	6	2.12%	-0.67 pp	$6,849,139	21.9%	21.9%
71	Paysend	United Kingdom → India	$10,443,491,625	3.6%	12	1.20%	-0.86 pp	$4,475,782	14.3%	36.2%
72	Paysend	Brazil → Bolivia	$66,561,984	12.5%	6	51.76%	+33.07 pp	$4,306,560	13.7%	49.9%
73	Paysend	Netherlands → Indonesia	$485,604,518	26.1%	7	2.36%	-0.89 pp	$2,989,635	9.5%	59.5%
74	Paysend	Poland → Ukraine	$1,832,158,053	9.1%	7	1.54%	-1.47 pp	$2,565,021	8.2%	67.6%
75	Paysend	United Kingdom → Nigeria	$4,326,654,913	6.5%	8	0.64%	-0.37 pp	$1,786,490	5.7%	73.3%
76	Paysend	Italy → Moldova	$349,880,986	11.8%	8	2.78%	-0.78 pp	$1,144,317	3.7%	77.0%
77	Paysend	Brazil → Peru	$54,418,129	40.0%	5	4.92%	-0.43 pp	$1,070,949	3.4%	80.4%
78	Paysend	Netherlands → Morocco	$605,420,495	8.0%	9	2.10%	-2.11 pp	$1,017,106	3.2%	83.7%
79	Paysend	Italy → Ukraine	$505,910,359	7.7%	9	2.16%	-2.93 pp	$840,590	2.7%	86.4%
80	Ace Money Transfer	Germany → Romania	$2,251,610,730	17.6%	13	4.45%	+1.66 pp	$17,681,767	36.1%	36.1%
81	Ace Money Transfer	Australia → China	$1,707,738,154	11.1%	10	4.12%	+0.82 pp	$7,817,646	16.0%	52.1%
82	Ace Money Transfer	United Kingdom → Pakistan	$4,849,609,279	10.3%	8	1.30%	-1.21 pp	$6,521,888	13.3%	65.5%
83	Ace Money Transfer	France → Morocco	$3,780,143,709	5.1%	14	2.56%	-1.04 pp	$4,962,650	10.1%	75.6%
84	Ace Money Transfer	Netherlands → Suriname	$123,205,835	40.0%	6	5.67%	-0.00 pp	$2,796,772	5.7%	81.3%
85	Ace Money Transfer	Australia → India	$6,692,824,591	2.8%	22	1.41%	-2.02 pp	$2,621,356	5.4%	86.7%

	provider_canon	corridor	flow_usd	quote_share	providers	median_cost	cost_vs_corridor_median_pp	revenue_intensity_proxy	within_provider_share	cum_share
86	Ace Money Transfer	United Kingdom → Sri Lanka	$898,685,534	13.8%	7	1.89%	-0.22 pp	$2,342,780	4.8%	91.5%
87	Ace Money Transfer	United Kingdom → Kenya	$1,521,598,271	7.4%	10	1.05%	-1.60 pp	$1,183,465	2.4%	93.9%
88	Ace Money Transfer	United Kingdom → Ghana	$600,335,606	5.4%	12	3.09%	+1.71 pp	$1,002,723	2.0%	95.9%
89	Ace Money Transfer	France → India	$549,188,388	7.1%	10	1.45%	-2.02 pp	$568,802	1.2%	97.1%
90	Dahabshiil	United Kingdom → Somalia	$322,464,729	11.8%	6	8.55%	+2.18 pp	$3,241,719	70.1%	70.1%
91	Dahabshiil	Kenya → Uganda	$856,701,360	3.7%	12	2.85%	-8.96 pp	$904,296	19.5%	89.6%
92	Dahabshiil	United Arab Emirates → Sudan	$59,045,470	25.0%	4	2.97%	-29.89 pp	$438,413	9.5%	99.1%
93	Dahabshiil	Australia → Somalia	$20,532,368	9.1%	5	1.59%	-1.38 pp	$29,679	0.6%	99.7%
94	Dahabshiil	Kenya → Rwanda	$5,127,974	8.3%	6	2.85%	-2.52 pp	$12,179	0.3%	100.0%
95	ANZ Bank	Australia → India	$6,692,824,591	5.6%	22	8.30%	+4.88 pp	$30,879,949	34.5%	34.5%
96	ANZ Bank	New Zealand → India	$1,153,490,901	9.1%	13	13.18%	+9.82 pp	$13,820,918	15.4%	49.9%
97	ANZ Bank	Australia → Vietnam	$1,291,443,739	11.1%	11	7.84%	+3.88 pp	$11,249,910	12.6%	62.4%
98	ANZ Bank	New Zealand → Philippines	$612,911,601	10.3%	11	12.60%	+11.10 pp	$7,988,986	8.9%	71.3%
99	ANZ Bank	Australia → Thailand	$904,479,428	10.5%	9	8.09%	+4.41 pp	$7,707,117	8.6%	79.9%
100	ANZ Bank	Australia → Lebanon	$770,124,346	6.1%	15	7.84%	+0.00 pp	$3,659,257	4.1%	84.0%
101	ANZ Bank	New Zealand → Tonga	$147,354,606	11.1%	13	14.97%	+8.77 pp	$2,450,998	2.7%	86.7%
102	ANZ Bank	Australia → Sri Lanka	$668,987,306	4.3%	11	8.03%	+4.99 pp	$2,335,638	2.6%	89.4%
103	ANZ Bank	New Zealand → Fiji	$167,259,670	9.4%	14	12.88%	+8.10 pp	$2,019,661	2.3%	91.6%
104	ANZ Bank	Australia → Indonesia	$485,911,990	5.3%	12	7.84%	+4.97 pp	$2,005,026	2.2%	93.8%

In [28]:

```
#@title 26. Country concentration: sender and receiver, top 10 countries
rows=[]
for p,g in eligible.groupby("provider_canon"):
    for side,col in [("send","send_country"),("receive","receive_country")]:
        x=g.dropna(subset=["revenue_intensity_proxy"]).groupby(col)["revenue_intensity_proxy"].sum().sort_values(ascending=False)
        if x.sum()<=0: continue
        s=x/x.sum(); hhi=s.pow(2).sum()
        rows.append({"provider_canon":p,"side":side,"countries":len(s),"top1_country_share":s.iloc[:1].sum(),
                     "top5_country_share":s.iloc[:5].sum(),"top10_country_share":s.iloc[:10].sum(),"hhi_10000":hhi*10000,
                     "effective_countries":1/hhi if hhi else np.nan,"gini":gini_nonneg(x.values)})
PROVIDER_COUNTRY_CONCENTRATION=pd.DataFrame(rows)

WU_COUNTRY_CONCENTRATION=PROVIDER_COUNTRY_CONCENTRATION.query("provider_canon=='Western Union'")
display(WU_COUNTRY_CONCENTRATION.style.format({"top1_country_share":"{:.1%}","top5_country_share":"{:.1%}","top10_country_share":"{:.1%}","hhi_10000":"{:,.0f}","effective_countries":"
{:.1f}","gini":"{:.2f}"}))

# Top country details for WU.
wu=eligible[eligible.provider_canon=="Western Union"].copy()
country_details=[]
for side,col in [("send","send_country"),("receive","receive_country")]:
    x=wu.groupby(col)["revenue_intensity_proxy"].sum().sort_values(ascending=False)
    s=x/x.sum()
    t=pd.DataFrame({"country":x.index,"revenue_intensity_proxy":x.values,"share":s.values})
    t["side"]=side; t["rank"]=np.arange(1,len(t)+1)
    country_details.append(t.head(20))
WU_TOP_COUNTRIES=pd.concat(country_details,ignore_index=True)
display(WU_TOP_COUNTRIES.style.format({"revenue_intensity_proxy":"${:,.0f}","share":"{:.1%}"}))
```

	provider_canon	side	countries	top1_country_share	top5_country_share	top10_country_share	hhi_10000	effective_countries	gini
158	Western Union	send	37	12.8%	44.7%	73.9%	645	15.5	0.60
159	Western Union	receive	70	15.6%	44.6%	64.8%	596	16.8	0.69

	country	revenue_intensity_proxy	share	side	rank
0	Germany	$285,198,754	12.8%	send	1
1	Saudi Arabia	$204,166,397	9.2%	send	2
2	United Arab Emirates	$181,506,310	8.2%	send	3
3	Spain	$161,073,740	7.2%	send	4
4	United Kingdom	$160,906,817	7.2%	send	5
5	Canada	$149,945,579	6.7%	send	6
6	France	$139,474,460	6.3%	send	7
7	Oman	$130,123,269	5.9%	send	8
8	South Africa	$124,353,578	5.6%	send	9
9	Qatar	$104,977,128	4.7%	send	10
10	Italy	$87,664,663	3.9%	send	11
11	Australia	$70,627,849	3.2%	send	12
12	Japan	$62,129,304	2.8%	send	13
13	Israel	$40,957,013	1.8%	send	14
14	Singapore	$36,781,374	1.7%	send	15
15	Malaysia	$30,963,031	1.4%	send	16
16	Kuwait	$30,401,767	1.4%	send	17
17	Cameroon	$28,244,413	1.3%	send	18
18	Austria	$28,027,000	1.3%	send	19
19	Switzerland	$26,604,035	1.2%	send	20
20	India	$346,286,022	15.6%	receive	1
21	Bangladesh	$232,657,149	10.5%	receive	2
22	Hungary	$141,612,358	6.4%	receive	3
23	China	$138,674,085	6.2%	receive	4
24	Morocco	$132,482,205	6.0%	receive	5
25	Nigeria	$111,735,598	5.0%	receive	6
26	Pakistan	$109,371,003	4.9%	receive	7
27	Zimbabwe	$77,633,304	3.5%	receive	8
28	Serbia	$77,054,145	3.5%	receive	9
29	Philippines	$73,719,813	3.3%	receive	10
30	Vietnam	$58,833,351	2.6%	receive	11
31	Ukraine	$56,027,866	2.5%	receive	12
32	Romania	$45,767,149	2.1%	receive	13
33	Lebanon	$43,621,203	2.0%	receive	14
34	Indonesia	$39,444,387	1.8%	receive	15
35	Haiti	$30,805,636	1.4%	receive	16
36	Tunisia	$30,105,689	1.4%	receive	17
37	Jordan	$28,562,976	1.3%	receive	18
38	Nepal	$27,077,869	1.2%	receive	19
39	Sri Lanka	$23,763,374	1.1%	receive	20

In [29]:
```
#@title 27. Sensitivity: how much does WU top-10 concentration depend on the proxy?
SENS=[]
```

```
wu_all=PROVIDER_FLOW_2024[PROVIDER_FLOW_2024.provider_canon=="Western Union"].copy()
for metric,label in [
    ("principal_proxy_equal_provider","Flow × equal provider share"),
    ("principal_proxy_quote","Flow × quote share"),
    ("revenue_intensity_proxy","Flow × quote share × RPW cost"),
]:
    z=conc_stats(wu_all,metric)
    z["scenario"]=label; SENS.append(z)
WU_CONCENTRATION_SENSITIVITY=pd.DataFrame(SENS)
display(WU_CONCENTRATION_SENSITIVITY[["scenario","top1_share","top5_share","top10_share","top20_share","hhi_10000","effective_corridors_1_hhi","gini"]].style.format({
    "top1_share":"{:.1%}","top5_share":"{:.1%}","top10_share":"{:.1%}","top20_share":"{:.1%}","hhi_10000":"{:,.0f}","effective_corridors_1_hhi":"{:.1f}","gini":"{:.2f}"
}))
```

	scenario	top1_share	top5_share	top10_share	top20_share	hhi_10000	effective_corridors_1_hhi	gini
0	Flow × equal provider share	5.7%	20.6%	31.3%	44.2%	159	62.9	0.63
1	Flow × quote share	4.5%	16.7%	26.9%	42.3%	135	74.1	0.62
2	Flow × quote share × RPW cost	5.7%	17.4%	28.5%	42.4%	142	70.4	0.62

In [30]:

```
#@title 28. Competitive intensity vs pricing: corridor-level tests
# One corridor observation; tests whether more providers / WU presence associate with lower costs.
CORRIDOR_COMPETITION=(LATEST_ALL.groupby(["send_country","receive_country","corridor"])
    .agg(providers=("provider_canon","nunique"), quotes=("provider_canon","size"), median_cost=("cost","median"), min_cost=("cost","min"),
         wu_present=("provider_canon",lambda x:(x=="Western Union").any()))
    .reset_index())
CORRIDOR_COMPETITION["log_providers"]=np.log(CORRIDOR_COMPETITION.providers.clip(lower=1))

corr=CORRIDOR_COMPETITION[["providers","median_cost","min_cost"]].corr(method="spearman")
print("Spearman correlations")
display(corr)

# Simple OLS with heteroskedasticity-robust SE when statsmodels exists.
try:
    import statsmodels.formula.api as smf
    reg=CORRIDOR_COMPETITION.dropna(subset=["median_cost","providers"]).copy()
    COMPETITION_OLS=smf.ols("median_cost ~ log_providers + C(wu_present)",data=reg).fit(cov_type="HC3")
    print(COMPETITION_OLS.summary())
except Exception as e:
    print("statsmodels regression skipped:",e)

# Provider relative pricing table.
PROVIDER_PRICING=(PROVIDER_CORRIDOR_LATEST.groupby("provider_canon")
    .agg(corridors=("corridor","nunique"),median_total_cost=("median_cost","median"),
         median_cost_vs_corridor_pp=("cost_vs_corridor_median_pp","median"),median_cost_premium=("cost_premium_pct","median"),
         pct_corridors_below_median=("cost_vs_corridor_median_pp",lambda x:(x<0).mean()))
    .reset_index().sort_values("corridors",ascending=False))
display(PROVIDER_PRICING.head(30).style.format({"median_total_cost":"{:.2f}%","median_cost_vs_corridor_pp":"{:+.2f} pp","median_cost_premium":"{:+.1%}","pct_corridors_below_median":"{:.1%}"}))
```

```
Spearman correlations
```

	providers	median_cost	min_cost
providers	1.000000	-0.180522	-0.439313
median_cost	-0.180522	1.000000	0.567490
min_cost	-0.439313	0.567490	1.000000

OLS Regression Results

Dep. Variable:	median_cost	R-squared:	0.015
Model:	OLS	Adj. R-squared:	0.010
Method:	Least Squares	F-statistic:	2.016
Date:	Tue, 08 Sep 2026	Prob (F-statistic):	0.135
Time:	12:58:54	Log-Likelihood:	-1277.5
No. Observations:	348	AIC:	2561.
Df Residuals:	345	BIC:	2573.
Df Model:	2		
Covariance Type:	HC3		

	coef	std err	z	P>|z|	[0.025	0.975]
Intercept	12.9955	4.394	2.958	0.003	4.384	21.607
C(wu_present)[T.True]	-3.6693	3.620	-1.014	0.311	-10.764	3.426
log_providers	-1.6567	0.840	-1.972	0.049	-3.303	-0.010

Omnibus:	502.471	Durbin-Watson:	1.496
Prob(Omnibus):	0.000	Jarque-Bera (JB):	67234.193
Skew:	7.348	Prob(JB):	0.00
Kurtosis:	69.490	Cond. No.	18.4

Notes:
[1] Standard Errors are heteroscedasticity robust (HC3)

	provider_canon	corridors	median_total_cost	median_cost_vs_corridor_pp	median_cost_premium	pct_corridors_below_median
368	Western Union	336	4.49%	+0.04 pp	+0.9%	36.3%
236	MoneyGram	306	3.91%	+0.00 pp	+0.0%	48.4%
289	Ria	197	4.36%	+0.00 pp	+0.0%	45.7%
287	Remitly	159	3.48%	-0.16 pp	-4.9%	54.1%
370	Wise	127	2.67%	-0.54 pp	-20.1%	69.3%
372	WorldRemit	108	2.98%	-0.72 pp	-15.8%	66.7%
373	Xoom	58	5.94%	+1.33 pp	+29.4%	15.5%
365	Walmart2World	38	5.00%	+1.13 pp	+28.8%	26.3%
267	Paysend	29	2.12%	-1.01 pp	-40.1%	79.3%
110	Dahabshiil	19	6.00%	+1.42 pp	+20.5%	36.8%
9	Ace Money Transfer	19	2.01%	-1.04 pp	-28.9%	73.7%
5	ANZ Bank	18	8.06%	+4.92 pp	+118.5%	11.1%
101	Citibank	18	17.50%	+11.86 pp	+210.6%	33.3%
313	Skrill	17	1.73%	-0.71 pp	-34.5%	70.6%
298	STC Pay	15	3.98%	+0.24 pp	+6.4%	33.3%
138	Extrabanca	14	19.34%	+16.30 pp	+545.8%	0.0%
179	InstaReM	14	0.76%	-2.22 pp	-76.7%	100.0%
343	TransferGo	14	1.11%	-2.47 pp	-67.6%	100.0%
257	Orbit Remit	13	2.46%	-1.22 pp	-40.1%	69.2%
264	Pangea	13	4.23%	-0.23 pp	-4.2%	53.8%
369	Westpac	12	12.00%	+5.92 pp	+119.1%	0.0%
139	Ezremit	11	3.24%	-0.04 pp	-1.2%	63.6%
18	Al Fardan Exchange	11	3.43%	+0.00 pp	+0.0%	45.5%
216	Lari	11	2.79%	-0.25 pp	-8.4%	63.6%
16	Al Ansari	10	3.49%	+0.18 pp	+4.7%	30.0%
325	State Bank of India	10	8.01%	+1.96 pp	+31.5%	40.0%
3	ABSA	10	28.43%	+15.94 pp	+130.2%	20.0%
353	Unicredit Banca	9	27.50%	+24.68 pp	+871.7%	0.0%
60	Bangkok Bank	9	16.27%	+4.51 pp	+38.4%	0.0%
311	Sikhona Money Transfers	9	7.40%	-3.61 pp	-33.4%	88.9%

In [31]:
```
#@title 29. 10-year corridor pricing dynamics for largest observed corridors
# Parse year from RPW period.
r_hist=r.copy()
r_hist["year"] = r_hist.period_raw.astype(str).str.extract(r"(19\d{2}|20\d{2})",expand=False).astype(float)
latest_year=int(np.nanmax(r_hist.year)) if r_hist.year.notna().any() else None
start_year=latest_year-10 if latest_year else None

# Rank corridors by 2024 bilateral flow among RPW-covered corridors.
flow_rank=(PROVIDER_FLOW_2024[["corridor","flow_usd"]].dropna().drop_duplicates("corridor")
           .sort_values("flow_usd",ascending=False))
TOP_FLOW_CORRIDORS=flow_rank.head(20).corridor.tolist()

hist=r_hist[(r_hist.corridor.isin(TOP_FLOW_CORRIDORS)) & (r_hist.year>=start_year)].copy() if start_year else r_hist[r_hist.corridor.isin(TOP_FLOW_CORRIDORS)].copy()
CORRIDOR_PRICE_TREND=(hist.groupby(["corridor","year"])
    .agg(median_cost=("cost","median"),min_cost=("cost","min"),providers=("provider_norm","nunique"),quotes=("provider_norm","size"))
    .reset_index())
```

```
# Start/end/CAGR-like annual pp trend and provider change.
trend_rows=[]
for c,g in CORRIDOR_PRICE_TREND.groupby("corridor"):
    g=g.dropna(subset=["median_cost"]).sort_values("year")
    if len(g)<2:continue
    a,b=g.iloc[0],g.iloc[-1]; years=max(b.year-a.year,1)
    trend_rows.append({"corridor":c,"start_year":int(a.year),"end_year":int(b.year),"start_cost":a.median_cost,"end_cost":b.median_cost,
                       "cost_change_pp":b.median_cost-a.median_cost,"annual_change_pp":(b.median_cost-a.median_cost)/years,
                       "start_providers":a.providers,"end_providers":b.providers,"provider_change":b.providers-a.providers})
CORRIDOR_PRICE_CHANGE=pd.DataFrame(trend_rows).sort_values("cost_change_pp")
display(CORRIDOR_PRICE_CHANGE.style.format({"start_cost":"{:.2f}%","end_cost":"{:.2f}%","cost_change_pp":"{:+.2f} pp","annual_change_pp":"{:+.2f} pp/yr"}))
```

	corridor	start_year	end_year	start_cost	end_cost	cost_change_pp	annual_change_pp	start_providers	end_providers	provider_change
18	United Kingdom → Nigeria	2015	2025	7.99%	1.94%	-6.04 pp	-0.60 pp/yr	13	9	-4
1	Canada → India	2015	2025	6.74%	2.93%	-3.81 pp	-0.38 pp/yr	14	13	-1
0	Australia → India	2015	2025	5.70%	3.04%	-2.66 pp	-0.27 pp/yr	27	22	-5
3	France → Morocco	2015	2025	5.98%	3.67%	-2.31 pp	-0.23 pp/yr	15	18	3
17	United Kingdom → India	2015	2025	4.00%	2.18%	-1.82 pp	-0.18 pp/yr	16	13	-3
13	Spain → Morocco	2015	2025	5.53%	3.88%	-1.64 pp	-0.16 pp/yr	12	9	-3
12	Spain → Colombia	2015	2025	6.10%	4.47%	-1.63 pp	-0.16 pp/yr	11	10	-1
11	Saudi Arabia → Philippines	2015	2025	4.55%	2.97%	-1.58 pp	-0.16 pp/yr	7	6	-1
7	Qatar → India	2015	2025	4.49%	2.99%	-1.50 pp	-0.15 pp/yr	10	5	-5
19	United Kingdom → Pakistan	2015	2025	3.42%	1.96%	-1.46 pp	-0.15 pp/yr	8	8	0
5	Malaysia → Indonesia	2015	2025	4.83%	3.80%	-1.03 pp	-0.10 pp/yr	19	23	4
16	United Arab Emirates → Pakistan	2015	2025	1.50%	0.54%	-0.96 pp	-0.10 pp/yr	10	11	1
2	Canada → Philippines	2015	2025	4.71%	3.77%	-0.94 pp	-0.09 pp/yr	13	14	1
9	Saudi Arabia → India	2015	2025	4.68%	3.77%	-0.91 pp	-0.09 pp/yr	8	8	0
10	Saudi Arabia → Pakistan	2015	2025	3.52%	3.06%	-0.46 pp	-0.05 pp/yr	9	8	-1
6	Oman → India	2016	2025	3.36%	3.15%	-0.21 pp	-0.02 pp/yr	9	8	-1
15	United Arab Emirates → India	2015	2025	2.81%	3.04%	+0.24 pp	+0.02 pp/yr	8	12	4
4	Kuwait → India	2015	2025	2.06%	2.42%	+0.36 pp	+0.04 pp/yr	9	8	-1
8	Saudi Arabia → Bangladesh	2015	2025	3.94%	7.79%	+3.85 pp	+0.39 pp/yr	8	7	-1
14	United Arab Emirates → Bangladesh	2015	2025	2.36%	6.76%	+4.41 pp	+0.44 pp/yr	8	7	-1

In [32]:
```
#@title 30. Cash/digital competitive pricing and provider mix
# Use same heuristic channel classifier, but for all providers.
text_fields=[c for c in ["channel","payment","receive_method","product"] if c in LATEST_ALL.columns]
ct=LATEST_ALL[text_fields].fillna("").astype(str).agg(" | ".join,axis=1).str.lower()
CH=LATEST_ALL.copy()
CH["digital_flag"]=ct.str.contains(r"online|internet|mobile|app|wallet|bank account|account deposit|debit|credit",regex=True)
CH["cash_flag"]=ct.str.contains(r"cash|agent|branch|retail|counter|pickup|pick-up",regex=True)
CH["channel_bucket"]=np.select([CH.cash_flag&~CH.digital_flag,CH.digital_flag&~CH.cash_flag,CH.cash_flag&CH.digital_flag],
                               ["cash/retail","digital/account","hybrid/ambiguous"],default="unclassified")
PROVIDER_CHANNEL=(CH.groupby(["provider_canon","channel_bucket"])
    .agg(quotes=("provider_canon","size"),corridors=("corridor","nunique"),median_cost=("cost","median"))
    .reset_index())
PROVIDER_CHANNEL["provider_channel_share"]=PROVIDER_CHANNEL.quotes/PROVIDER_CHANNEL.groupby("provider_canon").quotes.transform("sum")
# Keep broad providers for readable writeup table.
keep=set(PROVIDER_PRICING.head(15).provider_canon)
PROVIDER_CHANNEL_MAJOR=PROVIDER_CHANNEL[PROVIDER_CHANNEL.provider_canon.isin(keep)].copy()
display(PROVIDER_CHANNEL_MAJOR.style.format({"median_cost":"{:.2f}%","provider_channel_share":"{:.1%}"}))
```

	provider_canon	channel_bucket	quotes	corridors	median_cost	provider_channel_share
9	ANZ Bank	digital/account	33	18	7.84%	82.5%
10	ANZ Bank	hybrid/ambiguous	7	7	12.88%	17.5%
16	Ace Money Transfer	digital/account	34	15	2.08%	63.0%
17	Ace Money Transfer	hybrid/ambiguous	20	8	3.50%	37.0%
162	Citibank	digital/account	21	18	17.50%	70.0%
163	Citibank	hybrid/ambiguous	9	6	22.50%	30.0%
177	Dahabshiil	cash/retail	16	16	5.20%	76.2%
178	Dahabshiil	digital/account	2	1	0.56%	9.5%
179	Dahabshiil	hybrid/ambiguous	3	3	8.71%	14.3%
378	MoneyGram	cash/retail	136	127	5.03%	17.3%
379	MoneyGram	digital/account	171	79	2.07%	21.8%
380	MoneyGram	hybrid/ambiguous	478	225	4.29%	60.9%
429	Paysend	digital/account	64	29	2.16%	97.0%
430	Paysend	hybrid/ambiguous	2	1	2.36%	3.0%
458	Remitly	digital/account	411	128	3.22%	56.0%
459	Remitly	hybrid/ambiguous	323	120	3.66%	44.0%
461	Ria	cash/retail	86	79	5.75%	16.9%
462	Ria	digital/account	134	61	3.04%	26.3%
463	Ria	hybrid/ambiguous	290	140	4.29%	56.9%
478	STC Pay	digital/account	9	8	3.25%	33.3%
479	STC Pay	hybrid/ambiguous	18	15	4.75%	66.7%
501	Skrill	digital/account	30	17	1.97%	100.0%
586	Walmart2World	cash/retail	37	37	9.86%	28.9%
587	Walmart2World	digital/account	1	1	2.00%	0.8%
588	Walmart2World	hybrid/ambiguous	90	31	3.19%	70.3%
592	Western Union	cash/retail	273	258	6.38%	23.9%
593	Western Union	digital/account	215	103	2.85%	18.8%
594	Western Union	hybrid/ambiguous	656	286	4.52%	57.3%
597	Wise	digital/account	267	127	3.13%	100.0%
600	WorldRemit	digital/account	221	76	2.28%	61.7%
601	WorldRemit	hybrid/ambiguous	137	52	4.28%	38.3%
602	Xoom	digital/account	48	23	4.64%	27.0%
603	Xoom	hybrid/ambiguous	130	45	6.25%	73.0%

In [33]:
```
#@title 31. Provider geographic breadth, overlap, and WU competitor set
# Geographic breadth
PROVIDER_BREADTH=(LATEST_ALL.groupby("provider_canon")
    .agg(corridors=("corridor","nunique"),send_countries=("send_country","nunique"),receive_countries=("receive_country","nunique"),quotes=("provider_canon","size"))
    .reset_index())

# Pairwise Jaccard overlap with WU's observed corridor set.
wu_set=set(LATEST_ALL.loc[LATEST_ALL.provider_canon=="Western Union","corridor"])
overlap=[]
for p,g in LATEST_ALL.groupby("provider_canon"):
    s=set(g.corridor); inter=len(wu_set&s); union=len(wu_set|s)
    overlap.append({"provider_canon":p,"corridors":len(s),"overlap_with_wu":inter,"pct_wu_corridors_competed":inter/len(wu_set) if wu_set else np.nan,
```

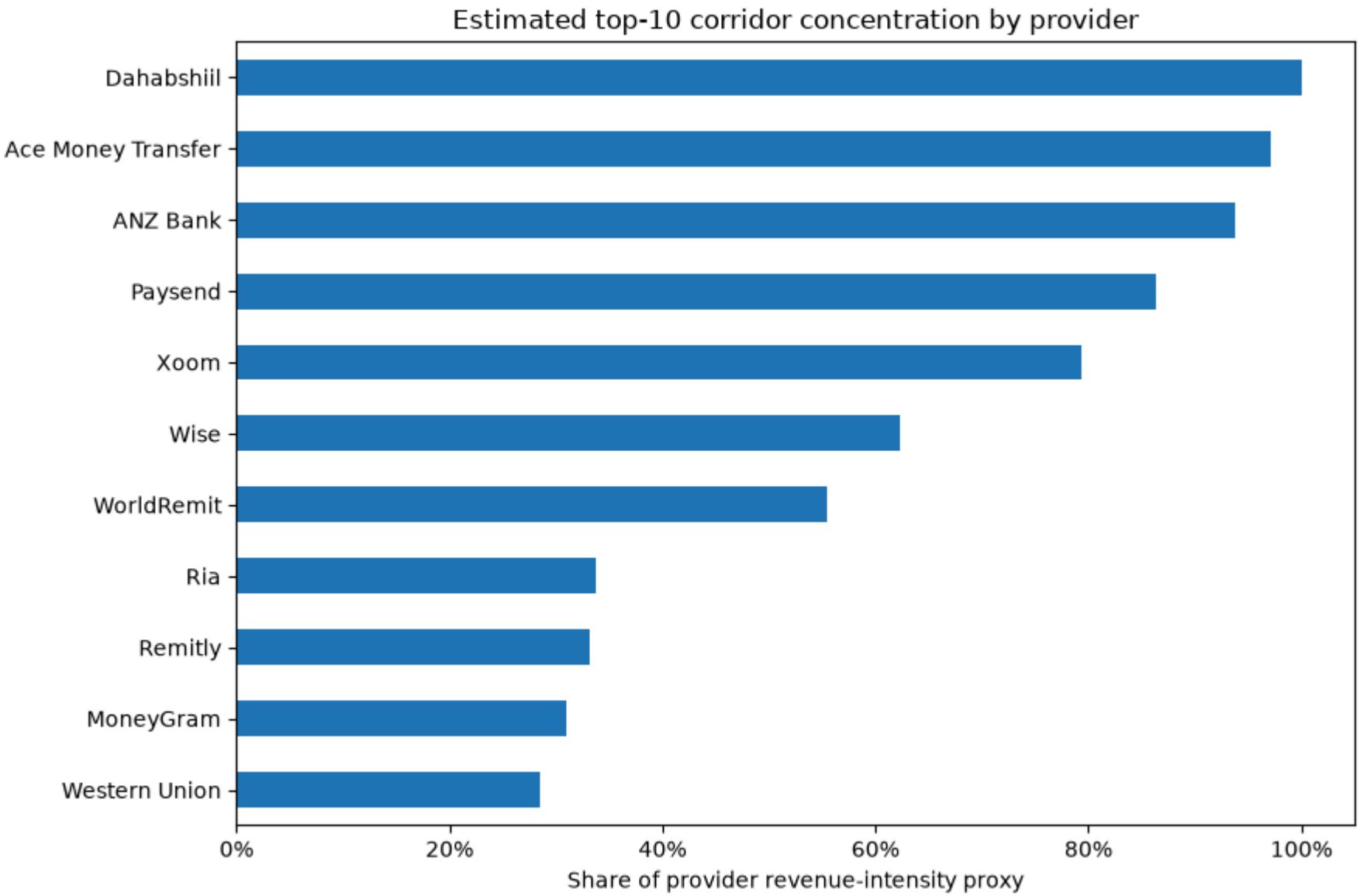
```
                    "pct_provider_corridors_overlapping_wu":inter/len(s) if s else np.nan,"jaccard_vs_wu":inter/union if union else np.nan})
WU_COMPETITOR_OVERLAP=pd.DataFrame(overlap).sort_values(["overlap_with_wu","jaccard_vs_wu"],ascending=False)
display(WU_COMPETITOR_OVERLAP.head(30).style.format({"pct_wu_corridors_competed":"{:.1%}","pct_provider_corridors_overlapping_wu":"{:.1%}","jaccard_vs_wu":"{:.1%}"}))
```

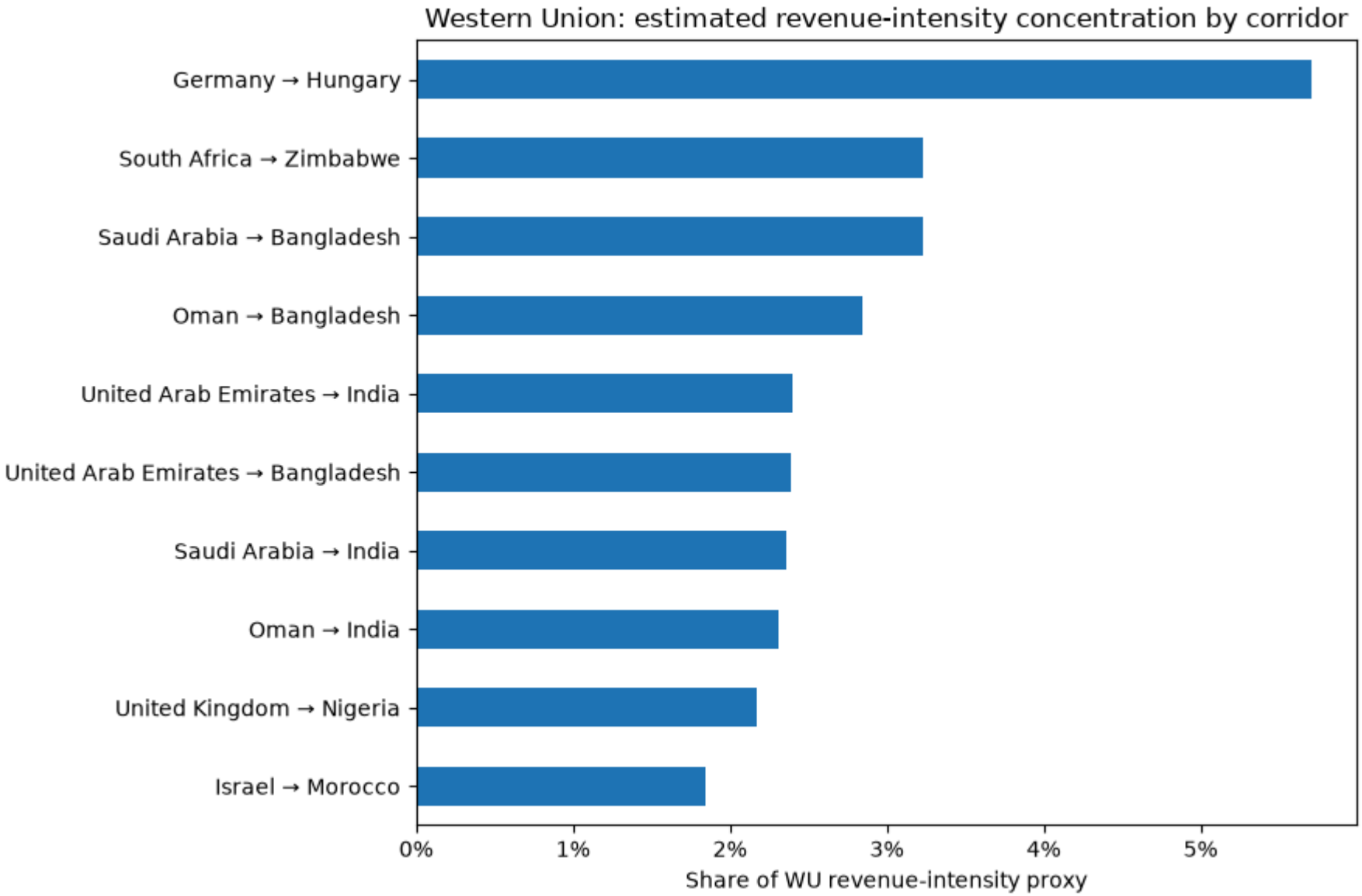
	provider_canon	corridors	overlap_with_wu	pct_wu_corridors_competed	pct_provider_corridors_overlapping_wu	jaccard_vs_wu
368	Western Union	336	336	100.0%	100.0%	100.0%
236	MoneyGram	306	304	90.5%	99.3%	89.9%
289	Ria	197	196	58.3%	99.5%	58.2%
287	Remitly	159	157	46.7%	98.7%	46.4%
370	Wise	127	127	37.8%	100.0%	37.8%
372	WorldRemit	108	105	31.2%	97.2%	31.0%
373	Xoom	58	57	17.0%	98.3%	16.9%
365	Walmart2World	38	36	10.7%	94.7%	10.7%
267	Paysend	29	27	8.0%	93.1%	8.0%
9	Ace Money Transfer	19	19	5.7%	100.0%	5.7%
5	ANZ Bank	18	18	5.4%	100.0%	5.4%
101	Citibank	18	18	5.4%	100.0%	5.4%
313	Skrill	17	17	5.1%	100.0%	5.1%
110	Dahabshiil	19	17	5.1%	89.5%	5.0%
298	STC Pay	15	15	4.5%	100.0%	4.5%
138	Extrabanca	14	14	4.2%	100.0%	4.2%
179	InstaReM	14	14	4.2%	100.0%	4.2%
257	Orbit Remit	13	13	3.9%	100.0%	3.9%
264	Pangea	13	13	3.9%	100.0%	3.9%
369	Westpac	12	12	3.6%	100.0%	3.6%
343	TransferGo	14	12	3.6%	85.7%	3.6%
18	Al Fardan Exchange	11	11	3.3%	100.0%	3.3%
139	Ezremit	11	11	3.3%	100.0%	3.3%
216	Lari	11	11	3.3%	100.0%	3.3%
3	ABSA	10	10	3.0%	100.0%	3.0%
16	Al Ansari	10	10	3.0%	100.0%	3.0%
55	Banca Intesa SanPaolo	9	9	2.7%	100.0%	2.7%
60	Bangkok Bank	9	9	2.7%	100.0%	2.7%
104	Commonwealth Bank	9	9	2.7%	100.0%	2.7%
135	Exact Change	9	9	2.7%	100.0%	2.7%

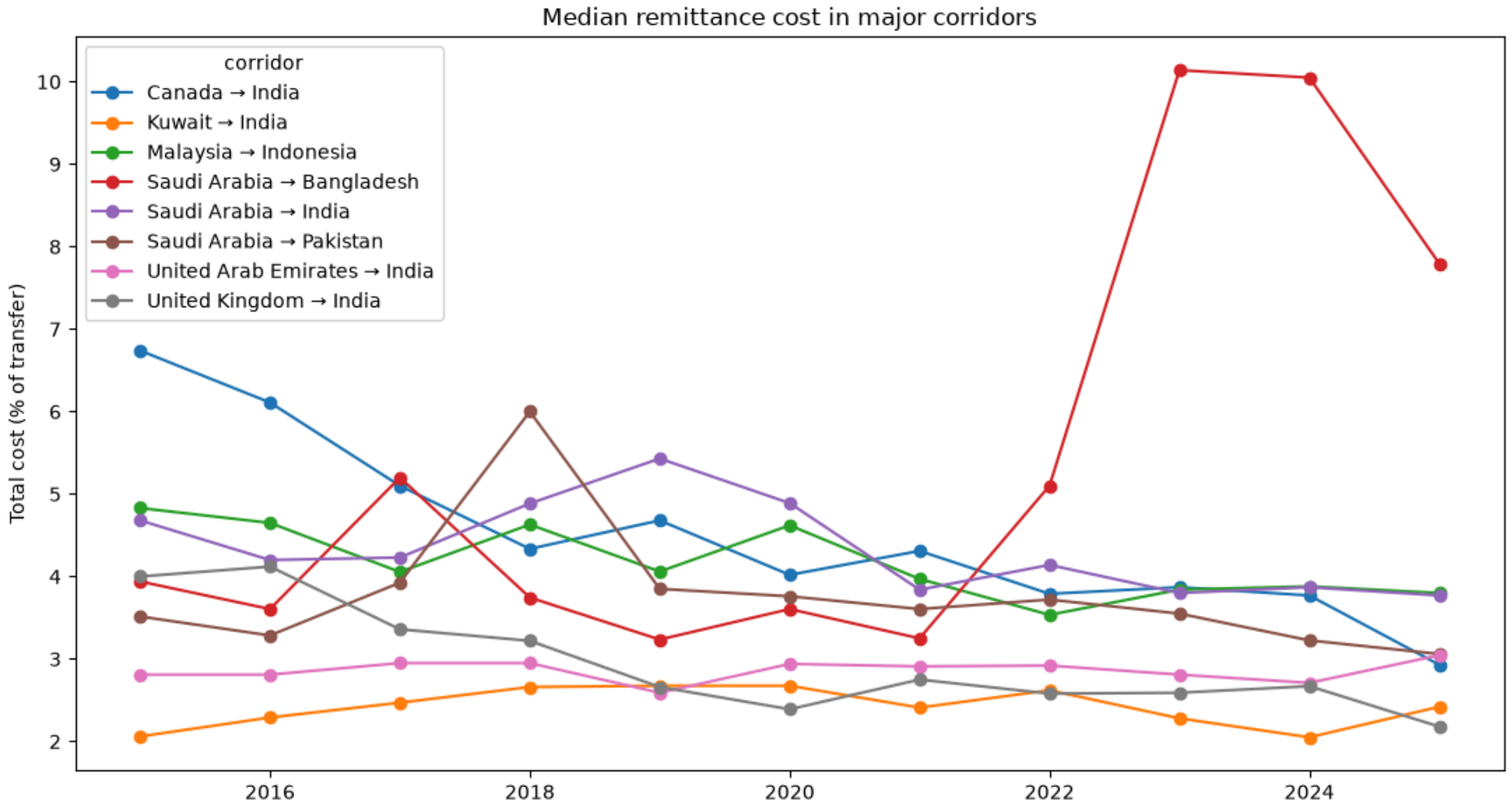
In [34]:

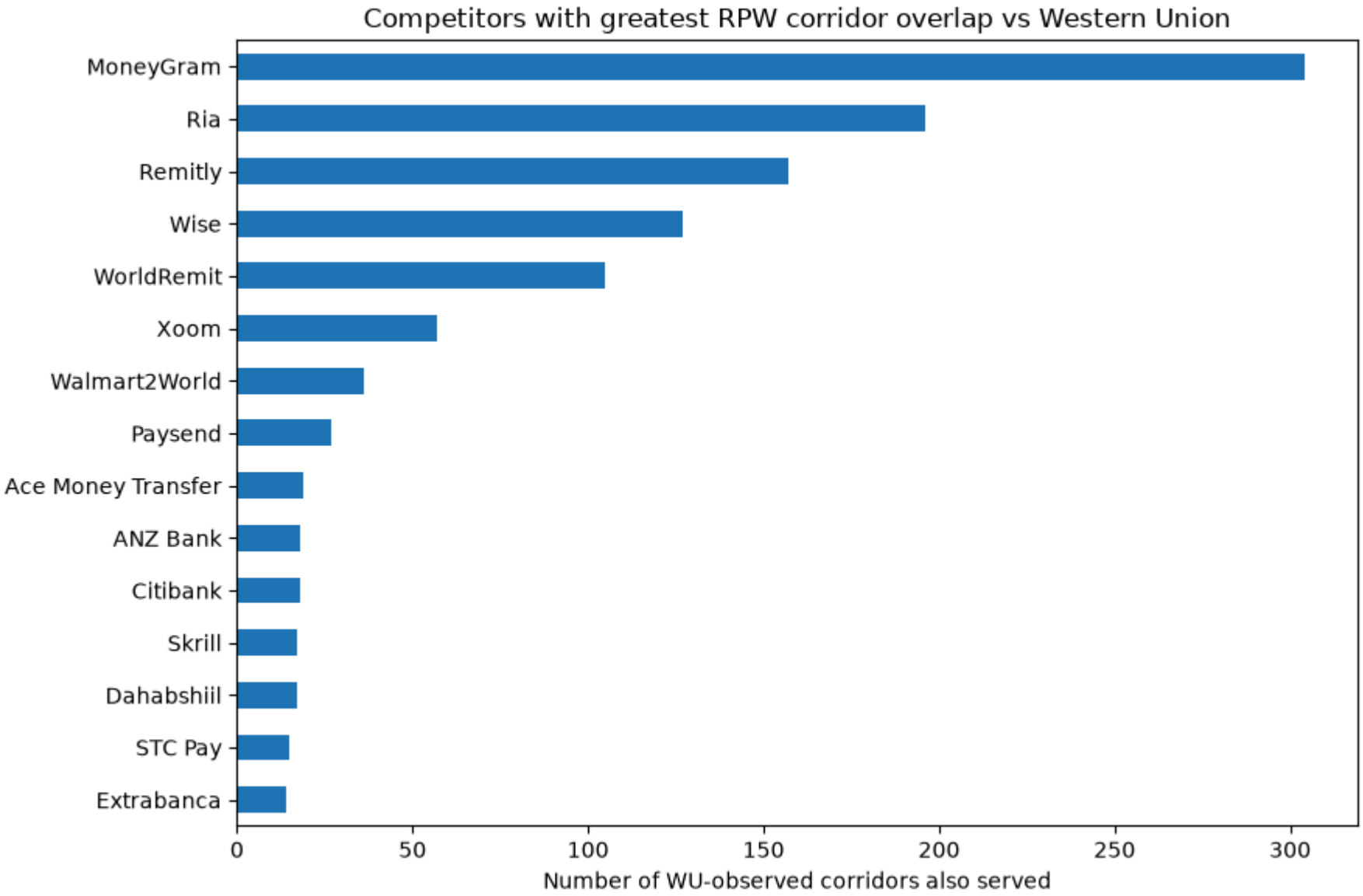
```
#@title 32. Writeup charts — concentration, top corridors, pricing trend, competitor overlap
import matplotlib.pyplot as plt
from pathlib import Path
FIGDIR=Path.cwd() / 'local_outputs' / 'wu_writeup_figures'; FIGDIR.mkdir(exist_ok=True)

# Figure 1: Top-10 corridor concentration by major provider
plot=PROVIDER_CONCENTRATION_PRIMARY[PROVIDER_CONCENTRATION_PRIMARY.provider_canon.isin(majors)].sort_values("top10_share")
ax=plot.plot.barh(x="provider_canon",y="top10_share",legend=False,figsize=(9,6),title="Estimated top-10 corridor concentration by provider")
ax.set_xlabel("Share of provider revenue-intensity proxy"); ax.set_ylabel("");
ax.xaxis.set_major_formatter(lambda x,pos:f"{x:.0%}")
plt.tight_layout(); plt.savefig(FIGDIR/'01_provider_top10_concentration.png',dpi=180,bbox_inches='tight'); plt.show()

# Figure 2: WU top-10 corridors
```

```
wut=top_corridors("Western Union",n=10).sort_values("within_provider_share")
ax=wut.plot.barh(x="corridor",y="within_provider_share",legend=False,figsize=(9,6),title="Western Union: estimated revenue-intensity concentration by corridor")
ax.set_xlabel("Share of WU revenue-intensity proxy"); ax.set_ylabel(""); ax.xaxis.set_major_formatter(lambda x,pos:f"{x:.0%}")
plt.tight_layout(); plt.savefig(FIGDIR/'02_wu_top10_corridors.png',dpi=180,bbox_inches='tight'); plt.show()

# Figure 3: pricing trends for largest 8 RPW-covered corridors
show=TOP_FLOW_CORRIDORS[:8]
piv=CORRIDOR_PRICE_TREND[CORRIDOR_PRICE_TREND.corridor.isin(show)].pivot(index="year",columns="corridor",values="median_cost")
ax=piv.plot(figsize=(11,6),marker='o',title="Median remittance cost in major corridors")
ax.set_ylabel("Total cost (% of transfer)"); ax.set_xlabel("");
plt.tight_layout(); plt.savefig(FIGDIR/'03_major_corridor_pricing_trends.png',dpi=180,bbox_inches='tight'); plt.show()

# Figure 4: competitor overlap with WU
ov=WU_COMPETITOR_OVERLAP.query("provider_canon!='Western Union'").head(15).sort_values("overlap_with_wu")
ax=ov.plot.barh(x="provider_canon",y="overlap_with_wu",legend=False,figsize=(9,6),title="Competitors with greatest RPW corridor overlap vs Western Union")
ax.set_xlabel("Number of WU-observed corridors also served"); ax.set_ylabel("");
plt.tight_layout(); plt.savefig(FIGDIR/'04_wu_competitor_overlap.png',dpi=180,bbox_inches='tight'); plt.show()

print('Saved figures to',FIGDIR)
```


Estimated top-10 corridor concentration by provider
Dahabshiil
Ace Money Transfer
ANZ Bank
Paysend
Xoom
Wise
WorldRemit
Ria
Remitly
MoneyGram
Western Union
0%
20%
40%
60%
80%
100%
Share of provider revenue-intensity proxy


Western Union: estimated revenue-intensity concentration by corridor
Germany → Hungary
South Africa → Zimbabwe
Saudi Arabia → Bangladesh
Oman → Bangladesh
United Arab Emirates → India
United Arab Emirates → Bangladesh
Saudi Arabia → India
Oman → India
United Kingdom → Nigeria
Israel → Morocco
0%
1%
2%
3%
4%
5%
Share of WU revenue-intensity proxy


Median remittance cost in major corridors
corridor
Canada → India
Kuwait → India
Malaysia → Indonesia
Saudi Arabia → Bangladesh
Saudi Arabia → India
Saudi Arabia → Pakistan
United Arab Emirates → India
United Kingdom → India
Total cost (% of transfer)
10
9
8
7
6
5
4
3
2
2016
2018
2020
2022
2024




```
Saved figures to C:\Users\micha\OneDrive\Documents\ChatGPT\alphasensecheck\python_analysis\outputs\colab\local_outputs\wu_writeup_figures
```

In [35]:

```
#@title 33. Writeup-ready summary tables
# Table A: WU concentration summary
wuconc=PROVIDER_CONCENTRATION_PRIMARY.query("provider_canon=='Western Union'").copy()
WRITEUP_WU_CONCENTRATION=wuconc[["provider_canon","corridors","top1_share","top5_share","top10_share","top20_share","hhi_10000","effective_corridors_1_hhi","gini"]]

# Table B: Provider comparison
WRITEUP_PROVIDER_COMPARISON=(PROVIDER_CONCENTRATION_PRIMARY
    .merge(PROVIDER_BREADTH,on="provider_canon",how="left",suffixes=("","_breadth"))
```

```
    .merge(PROVIDER_PRICING[["provider_canon","median_total_cost","median_cost_vs_corridor_pp","pct_corridors_below_median"]],on="provider_canon",how="left")
    .sort_values("corridors_breadth",ascending=False)
    .head(20))

# Table C: WU top 10 corridors
WRITEUP_WU_TOP10=top_corridors("Western Union",n=10).copy()

# Table D: WU top sending/receiving countries
WRITEUP_WU_COUNTRIES=WU_TOP_COUNTRIES.copy()

# Table E: major-corridor 10y pricing change
WRITEUP_CORRIDOR_PRICING=CORRIDOR_PRICE_CHANGE.copy()

for name,obj in [("WU concentration",WRITEUP_WU_CONCENTRATION),("Provider comparison",WRITEUP_PROVIDER_COMPARISON),
                 ("WU top 10 corridors",WRITEUP_WU_TOP10),("WU countries",WRITEUP_WU_COUNTRIES),("Corridor pricing",WRITEUP_CORRIDOR_PRICING)]:
    print("\n",name); display(obj)
```

```
WU concentration
```

	provider_canon	corridors	top1_share	top5_share	top10_share	top20_share	hhi_10000	effective_corridors_1_hhi	gini
247	Western Union	233.0	0.057096	0.17412	0.28486	0.423664	142.073042	70.38633	0.616404

```
Provider comparison
```

	provider_canon	corridors	top1_share	top5_share	top10_share	top20_share	hhi_01	hhi_10000	effective_corridors_1_hhi	gini	entropy	effective_corridors_entropy	metric	corridors_breadth	send_countries	receive_countries	quotes	median_total_cost	median_cost_vs_corridor_pp	pct_corridors_below_median
83	Western Union	233.0	0.057096	0.174120	0.284860	0.423664	0.014207	142.073042	70.386330	0.616404	4.745078	115.016775	revenue_intensity_proxy	336	43	94	1144	4.4900	0.0450	0.363095
82	MoneyGram	211.0	0.063532	0.197203	0.309428	0.459238	0.016404	164.042291	60.959890	0.624675	4.620877	101.583048	revenue_intensity_proxy	306	42	88	785	3.9050	0.0000	0.483660
80	Ria	130.0	0.062304	0.216532	0.337381	0.503562	0.019276	192.762954	51.877188	0.552240	4.326017	75.642416	revenue_intensity_proxy	197	18	77	510	4.3600	0.0000	0.456853
81	Remitly	105.0	0.064020	0.209374	0.331055	0.533456	0.020630	206.301586	48.472725	0.504216	4.213642	67.602269	revenue_intensity_proxy	159	16	67	734	3.4750	-0.1600	0.540881
78	Wise	88.0	0.222928	0.475475	0.623744	0.748472	0.074303	743.034521	13.458325	0.688183	3.423548	30.678062	revenue_intensity_proxy	127	23	30	267	2.6700	-0.5400	0.692913
79	WorldRemit	81.0	0.103435	0.361777	0.554756	0.722212	0.040318	403.176537	24.803031	0.621791	3.662600	38.962510	revenue_intensity_proxy	108	18	35	358	2.9800	-0.7150	0.666667
77	Xoom	23.0	0.189445	0.564766	0.793265	0.987474	0.091328	913.283175	10.949506	0.499196	2.704517	14.947097	revenue_intensity_proxy	58	7	40	178	5.9350	1.3325	0.155172
76	Paysend	20.0	0.218678	0.676428	0.863508	1.000000	0.112823	1128.232496	8.863421	0.543749	2.488423	12.042272	revenue_intensity_proxy	29	11	22	66	2.1200	-1.0100	0.793103
60	Dahabshiil	5.0	0.700718	1.000000	1.000000	1.000000	0.538242	5382.418258	1.857901	0.634090	0.839614	2.315472	revenue_intensity_proxy	19	6	12	21	6.0000	1.4200	0.368421
71	Ace Money Transfer	14.0	0.361463	0.813225	0.971024	1.000000	0.194076	1940.757517	5.152627	0.603650	1.989876	7.314626	revenue_intensity_proxy	19	6	16	54	2.0100	-1.0400	0.736842
12	Citibank	5.0	0.629579	1.000000	1.000000	1.000000	0.449955	4499.549859	2.222445	0.542765	1.073759	2.926360	revenue_intensity_proxy	18	3	12	30	17.5000	11.8600	0.333333
74	ANZ Bank	16.0	0.344504	0.799309	0.938433	1.000000	0.178450	1784.503088	5.603801	0.603549	2.111595	8.261408	revenue_intensity_proxy	18	2	12	40	8.0625	4.9225	0.111111
73	Skrill	14.0	0.320821	0.751612	0.960549	1.000000	0.171139	1711.385314	5.843219	0.547484	2.104786	8.205348	revenue_intensity_proxy	17	8	9	30	1.7250	-0.7100	0.705882
70	STC Pay	13.0	0.410841	0.852668	0.990543	1.000000	0.230682	2306.821896	4.334968	0.632500	1.832129	6.247172	revenue_intensity_proxy	15	1	15	27	3.9800	0.2400	0.333333
37	InstaReM	10.0	0.305744	0.789729	1.000000	1.000000	0.169648	1696.481336	5.894554	0.422519	2.001174	7.397734	revenue_intensity_proxy	14	2	9	15	0.7600	-2.2200	1.000000
75	Extrabanca	13.0	0.169070	0.639167	0.907600	1.000000	0.102615	1026.146094	9.745201	0.318785	2.403589	11.062809	revenue_intensity_proxy	14	1	14	28	19.3400	16.3000	0.000000
6	TransferGo	11.0	0.547878	0.888076	1.000000	1.000000	0.336140	3361.396352	2.974954	0.653090	1.549255	4.707963	revenue_intensity_proxy	14	8	12	42	1.1150	-2.4700	1.000000
72	Orbit Remit	13.0	0.195585	0.713451	0.965924	1.000000	0.125695	1256.947823	7.955780	0.443742	2.228568	9.286555	revenue_intensity_proxy	13	2	8	14	2.4600	-1.2250	0.692308
25	Westpac	10.0	0.242600	0.757959	1.000000	1.000000	0.151154	1511.543588	6.615754	0.379682	2.056788	7.820809	revenue_intensity_proxy	12	2	8	25	12.0025	5.9225	0.000000
18	Al Fardan Exchange	9.0	0.757868	0.964845	1.000000	1.000000	0.587646	5876.458239	1.701705	0.750381	0.960199	2.612217	revenue_intensity_proxy	11	1	11	26	3.4300	0.0000	0.454545

WU top 10 corridors

	provider_canon	corridor	flow_usd	quote_share	providers	median_cost	cost_vs_corridor_median_pp	revenue_intensity_proxy	within_provider_share	cum_share
2032	Western Union	Germany → Hungary	2.454011e+09	0.388889	5	13.300	7.230	1.269269e+08	0.057096	0.057096
2173	Western Union	South Africa → Zimbabwe	2.259411e+09	0.100000	12	31.810	21.530	7.187185e+07	0.032330	0.089426
2136	Western Union	Saudi Arabia → Bangladesh	7.792656e+09	0.111111	7	8.290	0.000	7.177902e+07	0.032289	0.121715
2115	Western Union	Oman → Bangladesh	2.305604e+09	0.333333	7	8.220	0.000	6.317355e+07	0.028417	0.150132
2208	Western Union	United Arab Emirates → India	2.633454e+10	0.083333	12	2.430	-0.615	5.332745e+07	0.023988	0.174120
2206	Western Union	United Arab Emirates → Bangladesh	3.524606e+09	0.222222	7	6.785	-0.365	5.314323e+07	0.023906	0.198026
2139	Western Union	Saudi Arabia → India	1.521617e+10	0.083333	8	4.140	0.585	5.249579e+07	0.023614	0.221640
2116	Western Union	Oman → India	5.977113e+09	0.272727	8	3.150	0.000	5.134883e+07	0.023098	0.244739
2234	Western Union	United Kingdom → Nigeria	4.326655e+09	0.290323	8	3.840	2.830	4.823522e+07	0.021698	0.266436
2048	Western Union	Israel → Morocco	7.236221e+08	0.333333	3	16.980	3.510	4.095701e+07	0.018424	0.284860

```
WU countries
```

	country	revenue_intensity_proxy	share	side	rank
0	Germany	2.851988e+08	0.128292	send	1
1	Saudi Arabia	2.041664e+08	0.091841	send	2
2	United Arab Emirates	1.815063e+08	0.081647	send	3
3	Spain	1.610737e+08	0.072456	send	4
4	United Kingdom	1.609068e+08	0.072381	send	5
5	Canada	1.499456e+08	0.067450	send	6
6	France	1.394745e+08	0.062740	send	7
7	Oman	1.301233e+08	0.058534	send	8
8	South Africa	1.243536e+08	0.055938	send	9
9	Qatar	1.049771e+08	0.047222	send	10
10	Italy	8.766466e+07	0.039434	send	11
11	Australia	7.062785e+07	0.031771	send	12
12	Japan	6.212930e+07	0.027948	send	13
13	Israel	4.095701e+07	0.018424	send	14
14	Singapore	3.678137e+07	0.016545	send	15
15	Malaysia	3.096303e+07	0.013928	send	16
16	Kuwait	3.040177e+07	0.013676	send	17
17	Cameroon	2.824441e+07	0.012705	send	18
18	Austria	2.802700e+07	0.012607	send	19
19	Switzerland	2.660403e+07	0.011967	send	20
20	India	3.462860e+08	0.155771	receive	1
21	Bangladesh	2.326571e+08	0.104657	receive	2
22	Hungary	1.416124e+08	0.063702	receive	3
23	China	1.386741e+08	0.062380	receive	4
24	Morocco	1.324822e+08	0.059595	receive	5
25	Nigeria	1.117356e+08	0.050262	receive	6
26	Pakistan	1.093710e+08	0.049199	receive	7
27	Zimbabwe	7.763330e+07	0.034922	receive	8
28	Serbia	7.705414e+07	0.034661	receive	9
29	Philippines	7.371981e+07	0.033162	receive	10
30	Vietnam	5.883335e+07	0.026465	receive	11
31	Ukraine	5.602787e+07	0.025203	receive	12
32	Romania	4.576715e+07	0.020588	receive	13
33	Lebanon	4.362120e+07	0.019622	receive	14
34	Indonesia	3.944439e+07	0.017743	receive	15
35	Haiti	3.080564e+07	0.013857	receive	16
36	Tunisia	3.010569e+07	0.013543	receive	17
37	Jordan	2.856298e+07	0.012849	receive	18
38	Nepal	2.707787e+07	0.012180	receive	19
39	Sri Lanka	2.376337e+07	0.010690	receive	20

Corridor pricing

	corridor	start_year	end_year	start_cost	end_cost	cost_change_pp	annual_change_pp	start_providers	end_providers	provider_change
18	United Kingdom → Nigeria	2015	2025	7.990	1.945	-6.045	-0.604500	13	9	-4
1	Canada → India	2015	2025	6.740	2.930	-3.810	-0.381000	14	13	-1
0	Australia → India	2015	2025	5.700	3.040	-2.660	-0.266000	27	22	-5
3	France → Morocco	2015	2025	5.980	3.670	-2.310	-0.231000	15	18	3
17	United Kingdom → India	2015	2025	4.000	2.180	-1.820	-0.182000	16	13	-3
13	Spain → Morocco	2015	2025	5.530	3.885	-1.645	-0.164500	12	9	-3
12	Spain → Colombia	2015	2025	6.100	4.470	-1.630	-0.163000	11	10	-1
11	Saudi Arabia → Philippines	2015	2025	4.550	2.970	-1.580	-0.158000	7	6	-1
7	Qatar → India	2015	2025	4.495	2.990	-1.505	-0.150500	10	5	-5
19	United Kingdom → Pakistan	2015	2025	3.415	1.955	-1.460	-0.146000	8	8	0
5	Malaysia → Indonesia	2015	2025	4.830	3.800	-1.030	-0.103000	19	23	4
16	United Arab Emirates → Pakistan	2015	2025	1.500	0.540	-0.960	-0.096000	10	11	1
2	Canada → Philippines	2015	2025	4.705	3.770	-0.935	-0.093500	13	14	1
9	Saudi Arabia → India	2015	2025	4.680	3.770	-0.910	-0.091000	8	8	0
10	Saudi Arabia → Pakistan	2015	2025	3.515	3.060	-0.455	-0.045500	9	8	-1
6	Oman → India	2016	2025	3.360	3.150	-0.210	-0.023333	9	8	-1
15	United Arab Emirates → India	2015	2025	2.810	3.045	0.235	0.023500	8	12	4
4	Kuwait → India	2015	2025	2.060	2.420	0.360	0.036000	9	8	-1
8	Saudi Arabia → Bangladesh	2015	2025	3.940	7.795	3.855	0.385500	8	7	-1
14	United Arab Emirates → Bangladesh	2015	2025	2.355	6.765	4.410	0.441000	8	7	-1

In [36]:

```
#@title 34. Optional WU calibration to reported CMT revenue / principal
# Enter reported values if desired. The concentration shares do not depend on these values.
WU_REPORTED_CMT_REVENUE = 3.507e9   # 2025 CMT revenue; edit if using another period
WU_REPORTED_PRINCIPAL = 107.4e9     # 2025 cross-border principal; edit if using another period

wu_proxy=PROVIDER_FLOW_2024[(PROVIDER_FLOW_2024.provider_canon=="Western Union") & PROVIDER_FLOW_2024.revenue_intensity_proxy.notna()].copy()
if wu_proxy.revenue_intensity_proxy.sum()>0:
    wu_proxy["revenue_proxy_share"] = wu_proxy.revenue_intensity_proxy / wu_proxy.revenue_intensity_proxy.sum()
    wu_proxy["calibrated_cmt_revenue"] = wu_proxy.revenue_proxy_share * WU_REPORTED_CMT_REVENUE
    wu_proxy["calibrated_principal"] = wu_proxy.revenue_proxy_share * WU_REPORTED_PRINCIPAL
    WU_CALIBRATED_CORRIDOR_REVENUE = wu_proxy.sort_values("calibrated_cmt_revenue",ascending=False)
    display(WU_CALIBRATED_CORRIDOR_REVENUE[["corridor","flow_usd","quote_share","median_cost","revenue_proxy_share","calibrated_cmt_revenue","calibrated_principal"]].head(25).style.format({
        "flow_usd":"${:,.0f}","quote_share":"{:.1%}","median_cost":"{:.2f}%","revenue_proxy_share":"{:.1%}","calibrated_cmt_revenue":"${:,.0f}","calibrated_principal":"${:,.0f}"
    }))
    print("WARNING: calibrated dollars are model allocations, not company-disclosed corridor revenue/principal.")
```

	corridor	flow_usd	quote_share	median_cost	revenue_proxy_share	calibrated_cmt_revenue	calibrated_principal
2032	Germany → Hungary	$2,454,010,999	38.9%	13.30%	5.7%	$200,234,980	$6,132,089,206
2173	South Africa → Zimbabwe	$2,259,410,507	10.0%	31.81%	3.2%	$113,382,253	$3,472,270,866
2136	Saudi Arabia → Bangladesh	$7,792,655,611	11.1%	8.29%	3.2%	$113,235,805	$3,467,785,991
2115	Oman → Bangladesh	$2,305,604,044	33.3%	8.22%	2.8%	$99,660,154	$3,052,038,947
2208	United Arab Emirates → India	$26,334,542,340	8.3%	2.43%	2.4%	$84,127,323	$2,576,354,295
2206	United Arab Emirates → Bangladesh	$3,524,606,413	22.2%	6.79%	2.4%	$83,836,711	$2,567,454,457
2139	Saudi Arabia → India	$15,216,172,432	8.3%	4.14%	2.4%	$82,815,339	$2,536,175,480
2116	Oman → India	$5,977,112,744	27.3%	3.15%	2.3%	$81,005,935	$2,480,763,449
2234	United Kingdom → Nigeria	$4,326,654,913	29.0%	3.84%	2.2%	$76,094,027	$2,330,338,881
2048	Israel → Morocco	$723,622,134	33.3%	16.98%	1.8%	$64,612,202	$1,978,714,139
1989	Canada → China	$1,984,574,827	35.7%	5.50%	1.8%	$61,497,634	$1,883,332,148
2182	Spain → Morocco	$3,585,228,881	22.7%	4.57%	1.7%	$58,744,428	$1,799,016,693
2046	Germany → Ukraine	$3,069,189,631	26.3%	4.00%	1.5%	$50,966,685	$1,560,827,472
2042	Germany → Serbia	$2,138,284,877	41.2%	3.56%	1.4%	$49,448,253	$1,514,326,298
1993	Canada → India	$7,594,507,109	15.6%	2.58%	1.4%	$48,297,605	$1,479,088,322
2212	United Arab Emirates → Pakistan	$5,361,259,319	11.8%	4.70%	1.3%	$46,716,441	$1,430,666,040
2021	France → Serbia	$602,285,822	36.4%	12.91%	1.3%	$44,622,164	$1,366,529,904
1988	Cameroon → Nigeria	$1,367,768,172	50.0%	4.13%	1.3%	$44,557,295	$1,364,543,336
2019	France → Morocco	$3,780,143,709	15.4%	4.65%	1.2%	$42,661,309	$1,306,479,773
2127	Qatar → India	$6,551,456,432	28.6%	1.33%	1.1%	$39,274,234	$1,202,752,414
2023	France → Tunisia	$2,000,199,436	22.2%	5.47%	1.1%	$38,356,095	$1,174,634,890
2069	Japan → China	$2,072,868,419	20.0%	5.73%	1.1%	$37,475,084	$1,147,654,426
2178	Spain → Colombia	$3,537,198,867	13.6%	4.77%	1.0%	$36,296,329	$1,111,555,658
2033	Germany → India	$2,543,682,959	14.3%	6.30%	1.0%	$36,115,344	$1,106,013,106
2228	United Kingdom → India	$10,443,491,625	10.7%	2.01%	1.0%	$35,480,626	$1,086,575,196

```
WARNING: calibrated dollars are model allocations, not company-disclosed corridor revenue/principal.
```

In [37]:

```python
#@title 35. Export extended analysis workbook + figure manifest
from pathlib import Path
OUT_EXT=Path.cwd() / 'local_outputs' / 'WU_RPW_Extended_Analysis.xlsx'
OUT_EXT.parent.mkdir(parents=True, exist_ok=True)
with pd.ExcelWriter(OUT_EXT,engine='openpyxl') as xw:
    PROVIDER_CORRIDOR_LATEST.to_excel(xw, sheet_name='provider_corridor_latest', index=False)
    PROVIDER_FLOW_2024.to_excel(xw, sheet_name='provider_flow_2024', index=False)
    PROVIDER_FLOW_COVERAGE.to_excel(xw, sheet_name='flow_coverage', index=False)
    PROVIDER_CONCENTRATION.to_excel(xw, sheet_name='provider_concentration', index=False)
    TOP10_PROVIDER_CORRIDORS.to_excel(xw, sheet_name='provider_top10_corridors', index=False)
    PROVIDER_COUNTRY_CONCENTRATION.to_excel(xw, sheet_name='country_concentration', index=False)
    WU_TOP_COUNTRIES.to_excel(xw, sheet_name='wu_top_countries', index=False)
    WU_CONCENTRATION_SENSITIVITY.to_excel(xw, sheet_name='wu_sensitivity', index=False)
    CORRIDOR_COMPETITION.to_excel(xw, sheet_name='corridor_competition', index=False)
    PROVIDER_PRICING.to_excel(xw, sheet_name='provider_pricing', index=False)
    CORRIDOR_PRICE_TREND.to_excel(xw, sheet_name='corridor_price_trend', index=False)
    CORRIDOR_PRICE_CHANGE.to_excel(xw, sheet_name='corridor_price_change', index=False)
    PROVIDER_CHANNEL.to_excel(xw, sheet_name='provider_channel_mix', index=False)
    PROVIDER_BREADTH.to_excel(xw, sheet_name='provider_breadth', index=False)
    WU_COMPETITOR_OVERLAP.to_excel(xw, sheet_name='wu_competitor_overlap', index=False)
    WRITEUP_WU_CONCENTRATION.to_excel(xw, sheet_name='WRITEUP_wu_concentration', index=False)
    WRITEUP_PROVIDER_COMPARISON.to_excel(xw, sheet_name='WRITEUP_provider_compare', index=False)
    WRITEUP_WU_TOP10.to_excel(xw, sheet_name='WRITEUP_wu_top10', index=False)
```

```
    WRITEUP_WU_COUNTRIES.to_excel(xw, sheet_name='WRITEUP_wu_countries', index=False)
    WRITEUP_CORRIDOR_PRICING.to_excel(xw, sheet_name='WRITEUP_corridor_pricing', index=False)
    if 'WU_CALIBRATED_CORRIDOR_REVENUE' in globals():
        WU_CALIBRATED_CORRIDOR_REVENUE.to_excel(xw, sheet_name='wu_calibrated_model', index=False)
print('Saved:',OUT_EXT)
print('Figures:', sorted(str(x) for x in FIGDIR.glob('*.png')) if 'FIGDIR' in globals() else 'Run chart cell first')
```

```
Saved: C:\Users\micha\OneDrive\Documents\ChatGPT\alphasensecheck\python_analysis\outputs\colab\local_outputs\WU_RPW_Extended_Analysis.xlsx
Figures: ['C:\\Users\\micha\\OneDrive\\Documents\\ChatGPT\\alphasensecheck\\python_analysis\\outputs\\colab\\local_outputs\\wu_writeup_figures\\01_provider_top10_concentration.png',
'C:\\Users\\micha\\OneDrive\\Documents\\ChatGPT\\alphasensecheck\\python_analysis\\outputs\\colab\\local_outputs\\wu_writeup_figures\\02_wu_top10_corridors.png',
'C:\\Users\\micha\\OneDrive\\Documents\\ChatGPT\\alphasensecheck\\python_analysis\\outputs\\colab\\local_outputs\\wu_writeup_figures\\03_major_corridor_pricing_trends.png',
'C:\\Users\\micha\\OneDrive\\Documents\\ChatGPT\\alphasensecheck\\python_analysis\\outputs\\colab\\local_outputs\\wu_writeup_figures\\04_wu_competitor_overlap.png']
```

Interpretation guardrails for the writeup

- RPW is a **pricing/availability sample**, not transaction-level market-share data. Do not call quote share actual market share.
- The 2024 bilateral matrix in this notebook is modeled; label it accordingly and use observed central-bank bilateral data where available as a replacement/validation layer.
- The `revenue_intensity_proxy` is best used for **relative concentration within a provider**. It should not be described as reported revenue.
- WU-calibrated corridor dollars are a scenario allocation to reported CMT totals. Present a range across sensitivity cases rather than a single precise number.
- For Question 10, the most defensible outputs are the sensitivity range for WU top-10 corridor concentration, top-10 sender/receiver-country concentration, and comparisons against large competitors.
- For Question 8, show the 10-year cost paths of major corridors together with provider-count changes; these give a quantitative competition/pricing narrative.
- For Question 2, use provider footprint share, flow-weighted footprint proxy, and reported company principal/revenue market share as **separate definitions** rather than blending them.